# Numerical Methods for Chemical Engineers

Study guide for 6E5X0

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

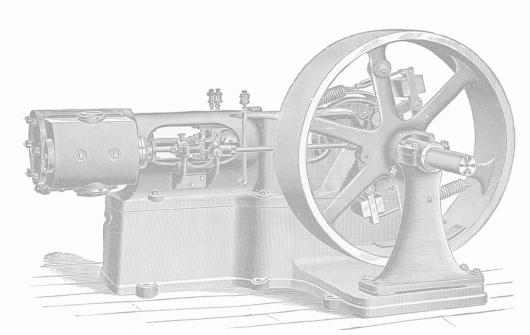
Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Numerical Methods

"Simulation and mathematical modeling will power the twenty-first century the way steam powered the nineteenth."

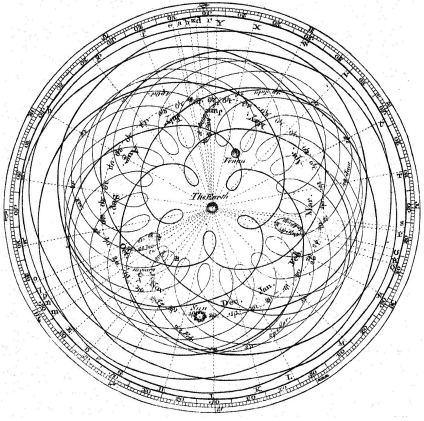
— W.H. Press*



*Author of Numerical recipes, in "The Nature of Mathematical Modeling" by Neil Gershenfeld

# Ptolemy and the almagest





~150 AD. Development of numerical approximations to describe the motions of the heavenly bodies with accuracy matching reality sufficiently.

# Numerical Methods

- Numerical analysis is concerned with obtaining approximate solutions to problems while maintaining reasonable bounds of error...
- ...because it is often impossible to obtain exact answers ...
- Numerical analysis makes use of algorithms to approximate solutions

# Used in many fields... Including Chemical Engineering

- Description of reactors and separators (dynamic and steady state)
- Computational fluid dynamics
- Thermodynamic equations of state
- Optimizing process performance
- Design and synthesis of processes
- Regression of data, e.g. isotherms, kinetics, ...

# Course Schedule

| Lecture | Topic | Teacher |
|---|---|---|
| 1 | Intro + advanced programming | IR |
| 2 | Numerical errors | IR |
| 3 | Linear eqns: direct methods | IR |
| 4 | Linear eqns: iterative methods | IR |
| 5 | Non-linear equations (1) | MSA |
| 6 | Non-linear equations (2) | MSA |
| 7 | Interpolation + integration | IR |
| 8 | ODEs (1) | MSA/IR |
| 9 | ODEs (2) | MSA/IR |
| 10 | PDEs | MSA |
| 11 | Regression and Optimization | IR |
| 12 | Questions/practice | IR |

# Course Objectives

- Gain experience with programming basics and algorithm design (Python, Excel)
- Understand the background of important numerical algorithms used in CEC
- Recognize and solve (systems of) linear, non-linear and differential equations numerically
- Learn about numerical integration, interpolation, optimization
- Apply your skills to solve realistic assignments and small sample problems

# Prerequisites

The following courses should give you enough background knowledge to follow this course comfortably:

- Calculus
- Linear Algebra
- Some basic Python experience
  - We will shortly cover some aspects on Python programming in the first lectures. Detailed documents and courses are provided on Canvas, for your own reference.

You will definitely need a laptop with Python and Excel installed!

# Course Materials

- Lecture slides
- Python scripts
- Additional articles
- There are some useful books for those seeking more in-depth knowledge and alternative methods, not mandatory:
  - Numerical recipes, W.H. Press et al.
  - Numerical methods for chemical engineering, K.J. Beers
  - Numerical methods for chemical engineers, A. Constantinides
  - Python Crash Course, 3rd Edition, Eric Matthes

Look on Canvas/modules for the slides, exercises, scripts, assignments and additional documentation on Python.

# Assessment

## 4 assignments

- Each 20% of the final result
- Done in groups of 2 persons, see Canvas → People → Project Groups
- Short report (template provided, Overleaf and Canvas)
- Assignment 1 graded through peer review, 2–4 graded by instructor

## Final exam

- Practical and theoretical questions, covering *all topics*
- Exam taken on your own computer using Schoolyear software
- You can use the slides and (online) modules documentation
- Sample exam will be released after lecture topics are done
- Grade of the final exam needs to be at least a 5.0

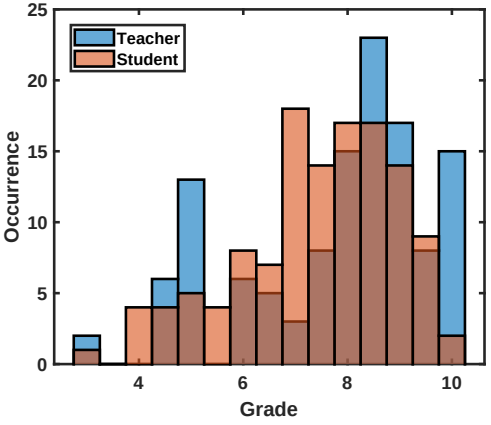
# Assignment grading: peer assessment & feedback (1)

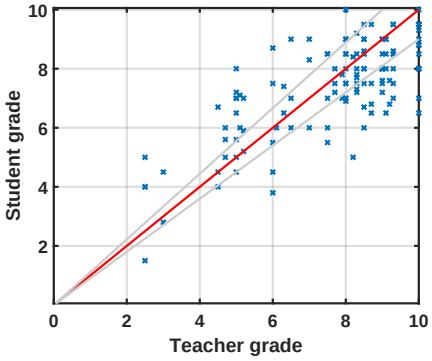
- Assignment 1 is graded through supervised peer-assessment. After the deadline, each *person* will grade 2 other assignments. Rubrics are available to maintain a consistent assessment among different groups. Criteria are:
  - Algorithm design
  - Code quality
  - Verification, validation and analysis
  - Visualisation
  - Report
  - Creativity
- Review are done within 3 days (rubrics+written feedback to establish its validity). We assess the feedback quality by a multiplier (0.8-1.2).
- You can challenge one or more reviews by submitting a rebuttal;
- The final grade will be the averaged grade from the remaining peer-assessments (group), multiplied by the peer-review quality (individual), with a max. of 10.
- When statistics are poor ($\leq 2$ reviews), the assignment will be graded by the instructor, which discards all remaining peer-reviews/grades.

# Assignment grading: peer assessment & feedback (2)

- Along with the rubrics, you will give each other specific comments for improvement: What are you impressed with; why did you score a certain criteria low; how to improve the code or visuals, etc. Give at least 3 tips and 3 tops.
- Grades for an assignment are released only when proper assessment and feedback have been given.
- Rebuttals are turned in through an additional assignment. A rebuttal should convince us and provide evidence and in-depth argumentation why a particular review is flawed. We will evaluate the rebuttals and discard a peer-review if it is indeed disproportionate.
- We are getting help from teaching assistants to make the process go smoothly. I will show a possible solution after the deadline.
- Grading with rubrics: don't be afraid to use the full spectrum.

# Peer assessment in the past



# Peer assessment summary

Complete document can be read through Canvas, here's the summary:

- The assignment and report template are released along with the grading rubrics
- Canvas automatically performs a plagiarism check
- The lecturer will give a short overview of how the assignment could have been solved (point of reference)
- Students have 3 days for double-blind peer assessment
- Students have the opportunity to challenge their reviewers (rebuttal)
- Lecturers and TAs will check the review quality, as well as the reports that have very low or very high marks or large deviations among reviews coarsely. I will provide a full correction when no suitable peer-reviews have been done.
- If a student fails to produce a good, timely peer-review, their grade for the assignment will be marked NA.

# Assignment handout and deadlines

Hand-in your assignments via Canvas

- Deadlines are given on Canvas as well
- Deliver the report in PDF format
- Submit along the scripts + necessities in a .zip (do not put the report in the zip)
- Use your student ID instead of your name for identification purposes
- Be aware that a .docx stores the original author name as metadata.

# So... What about AI?

We understand that you might use AI in the assignments, but note:

- Practice makes perfect; the final exam is without AI assistance
- Always indicate the LLM/AI tool and version and the extent
- AI might be wrong, misleading, or suboptimal
- You take full responsibility for everything you turn in

You can use AI effectively and to your advantage:

- Use it to explain concepts, not to generate the answer. The struggle is part of the learning process.
- First try yourself, let AI comment on your approach
- Be critical of the output, verify and validate
- Numerical methods are about understanding the algorithms, and about validation.

# Course Philosophy

- This is a hard course! It will take a lot of hours, especially if you have little coding experience.
- We are here to help you. To have nice discussions, to show alternative ways, etc. Clarify language subtleties, or suggestions. Not to give away the answers.
- We encourage research and independent learning. It comes down to paying attention, repetition of the concepts, and practice practice practice!
- We try to make the lectures interactive, working on examples and creating scripts as we go. It is advised that you work along with us to get the most out of this course!
- It is ok to discuss the general approach to solving the problem, or to get a hint, or several hints, if you get stuck while solving a problem, but work out the details of the solution in your own group.
- It is **not ok** to take someone else's solution (or an AI/LLMs) and present it as your own work.
- Take regular breaks! Better 6 times per hour a short break than 1 time per hour a long break. Use keyboard shortcuts, or a mouse if you must. Set up screen brightness to a pleasant value. Stretch.

# Contact information


## Ivo Roghair

- E-mail: i.roghair@tue.nl
- Office: STW 0.37

## Martin van Sint Annaland

- E-mail: m.v.sintannaland@tue.nl
- Office: STW 0.39

# Some Acknowledgements



# Some Real Acknowledgements

For inspiration, examples and exercises:

- To Roel Verstappen of Groningen University
- To Johan Hult of Cambridge University
- To Edwin Zondervan of University of Twente
- To edX: MITx course "Introduction to Computer Science and Programming Using Python"

# Python and Programming 1

## Programming basics and algorithms

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

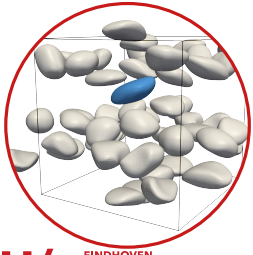
Numerical Methods (6BER03), 2025-2026

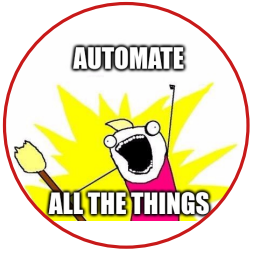
# Today's outline


- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

# Why should you learn something about programming?

- Scientific analyses depend more than ever on computer programs and simulation methods
- Knowledge of programming allows you to automate routine tasks
- Ability to understand algorithms by inspection of the code
- Learn to think by dissecting a problem into smaller, easier to solve, parts










# Introduction to programming

## What is a program?

*A program is a sequence of instructions that is written to perform a certain task on a computer.*

- The computation might be something mathematical, a symbolic operation, image analysis, etc.

## Data → Algorithm → Result

A program typically consists of three parts:

1. Input (Get the radius of a circle)
2. Operations (Compute and store the area of the circle)
3. Output (Print the area to the screen)

# Python help, documentation, resources

- Use the course materials provided for 6BBR06 - Programming and Linear Algebra
- Refer to the Python documentation at Official documentation.
  - Try for instance: `help(print)` or `help(help)`.
- Other packages that we will use:
  - NumPy documentation
  - Matplotlib documentation
  - SciPy documentation
- Python Crash Course, 3rd Edition by Eric Matthes
- A Whirlwind Tour of Python by Jake Vanderplas
- Introduction to Scientific Programming with Python by Joakim Sundnes
- Python Programming And Numerical Methods: A Guide For Engineers And Scientists by Kong, Siauw and Bayen
- Search the web, Reddit, YouTube, etc.

# About your editor

- There are several keyboard shortcuts (vary with editor) that will make coding much more efficient. For VSCode:
  - Typing part of a command and pressing Tab completes the command and lists the possibilities
  - Move lines or selections up and down with Alt + Up / Down
  - Duplicate lines with Shift + Alt + Down
  - You can bind keystrokes such as Ctrl + F5 to run the current script
  - Comment/uncomment lines with Ctrl + /
  - Set up snippets for often used code blocks (e.g. importing packages)
- If a computation takes too long, you can press Ctrl + C to stop the program and return to the command line. Stored variables may contain incomplete results.
- Python files must be in the *current working directory* or in the Python *path*, the locations where Python searches for a command. If you try to run a script that is not in the path, Python will throw an Exception/Error.

# Today's outline


- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

# Terminology

**Variable** Piece of data stored in the computer memory, to be referenced and/or manipulated

**Function** Piece of code that performs a certain operation/sequence of operations on given input

**Operators** Mathematical operators (e.g. `+` `-` `*` or `/`), relational (e.g. `<` `>`or `==`, and logical operators (**`and`**, **`or`**)

**Script** Piece of code that performs a certain sequence of operations without specified input/output

**Expression** A command that combines variables, functions, operators and/or values to produce a result.

# Variables in Python

- Python stores variables in the *namespace*
- You should recognize the difference between the *identifier* of a variable (its name, e.g. `x`, `setpoint_p`), and the data that it actually stores (e.g. 0.5)
- Python also defines a number of functions by default, e.g. `min`, `max` or `sum`.
  - A list of built-in methods is given by `dir(__builtins__)`
- You can assign a variable by the = sign:

```
>>> x = 4*3
>>> x
12
```

- If you don't assign a variable, it will be stored in _
- In most text editors, all variables are cleared automatically before the next execution.

# Datatypes and variables

Python uses different types of variables:

| Datatype | Example |
|---|---|
| `str` | `'Wednesday'` |
| `int` | `15` |
| `float` | `0.15` |
| `list` | `[0.0, 0.1, 0.2, 'Hello', ['Another','List']]` |
| `dict` | `{'name': 'word', "n": 2}` |
| `bool` | `False` |
| `tuple` | `(True, False)` |

Everything in Python is an object. You can use the `dir()` function to query the possible methods on an object of a datatype (e.g. `(dir(list))`, `dir(28)` or `dir("Yes!")`).

# Strings and f-strings

- Strings are sequences of characters, enclosed by either single quotes `'...'` or double quotes `"..."`.
- Methods to manipulate strings include `.upper()`, `.lower()`, `.replace()`, `.split()`, `.index()`, etc.
- Using f-strings are a preferred way to format output (Python 3.6+)
- Readable, compact, explicit
- Formatting affects display, not the numerical value

```
x = 3.1415926535
n = 42
hello_str = "Hello, world!"
value_fstr = f"x is approximate {x:.2f}"

print(f"x = {x}")
print(f"n = {n} and is type {type(n)}")
print(f"{x=}")
print(hello_str)
print(value_fstr)
```

```
x = 3.1415926535
n = 42 and is type <class 'int'>
x=3.1415926535
Hello, world!
x is approximate 3.14
```

# Lists (refresher)

- Ordered, mutable containers
- Methods to add, remove, sort, manipulate elements
- Concatenate lists using `+`, or multiply using `*`
- Zero-based indexing, slicing of multiple elements using `:`
- Typical numerical use: storing samples, time series

```
my_list_of_numbers = [1, 2, 3]
my_list_of_numbers.append(4) # Now: [1, 2, 3, 4]
my_list_of_strings = ["apple", "banana", "cherry"]
my_list_of_strings[1] = "blueberry" # Now: ["apple", "blueberry", "cherry"]
my_list_of_booleans = [True] * 4 # [True, True, True, True]
all_numbers = list(range(5)) + [10, 20, 30] # [0, 1, 2, 3, 4, 10, 20, 30]
my_heterogeneous_list = [1, "apple", 3.14, True, [1, 2], {"a": 1}]
```

While lists *can* be heterogeneous, it's often best to keep them homogeneous for easier processing.

# Exercise: Lists practice

Given a vector

$$x = [2\ 4\ 6\ 8\ 10\ 12\ 14\ 16\ 18\ 20\ 30\ 40\ 50\ 60\ 70\ 80]$$

- Define the vector using `range`'s, without typing all individual elements
- Investigate the meaning of the following commands:

```
x[2]
x[0:5]
x[:-1]
y = x[4:]
y[3]
y.pop(3)
sum(x)
max(x)
min(x)
x[::-1]
```

# Dictionaries (refresher)

- Mutable, unordered storage of key–value pairs
- Fast lookup of arbitrary values (i.e. objects)
- Methods include `.keys`, `.values`, `.items`
- Typical use: parameters, metadata

```
params = {"dt": 0.01, "T": 10.0, "method": "RK4"}
print(f"Time step: {params["dt"]} s") # Access value via key
params["dt"] = 0.02 # Modify value
params["TotalSteps"] = int(params["T"] / params["dt"]) # Add new key--value pair
print(f"New timestep: {params["dt"]} s, total steps: {params["TotalSteps"]}")
```

```
Time step: 0.01 s
New timestep: 0.02 s, total steps: 500
```

# Tuples (refresher)

- Ordered, immutable containers
- Methods to count, find elements
- Zero-based indexing, slicing
- Typical numerical use: fixed collections of related values

```
1 my_tuple = (1, 2, 3)
2 coordinate_point = (3, 4)
3 record = ("CO2", 44.01, "g/mol")
```

Use tuples to store values that should not change during program execution:

```
1 print(my_tuple)
2 my_tuple[1] = 4
```

```
(1, 2, 3)
Traceback (most recent call last):
  File "/home/ivo/Repositories/nm-slides/test_scripts.py", line 5, in <module>
    my_tuple[1] = 4
    ~~~~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

# Loops (refresher)

- `for`: iterate over sequences
- `while`: iterate until condition fails
- `break` to exit loop early, `continue` to skip to next iteration

```
1 for i in range(5):
2     print(f"Iteration {i}")
```

```
Index 0, Value 10
Index 1, Value 20
Index 2, Value 30
Index 3, Value 40
```

# Loops (refresher)

- Use **`enumerate`** to loop over indices and values
- If you want to loop over two or more lists in parallel, use **`zip`**
- Advanced iteration tools are available in the `itertools` module

```
L = [10, 20, 30, 40]
for i, val in enumerate(L):
    print(f"Index {i}, Value {val}")
```

```
Index 0, Value 10
Index 1, Value 20
Index 2, Value 30
Index 3, Value 40
```

```
M_list = ['a','b','c','d','e']
N_list = [1,2,3,4,5]

for m,n in zip(M_list, N_list):
    print(f"Value {m = }, Value {n = }")
```

```
Value m = 'a', Value n = 1
Value m = 'b', Value n = 2
Value m = 'c', Value n = 3
Value m = 'd', Value n = 4
Value m = 'e', Value n = 5
```

# Conditional statements (refresher)

- Control flow based on logic
- Essential for algorithms

```python
temp = 10
if temp < 0:
    state = "solid"
elif 0 <= temp < 100:
    state = "liquid"
else:
    state = "gas"

print(f"At {temp} °C, water is in {state} state." )
```

Check membership:

```python
states = ["Good", "Bad", "Ugly"]
if "Good" in states:
    print("We have a good state!")
else:
    print("No good states found.")
```

Ternary operator:

```python
score = "Good" if points >= 75 else "Bad"
```

# Exercise

- Loop over a list of values (create a list with at least 8 values)
- Count how many are negative
- Stop early if a value exceeds 10
- Print the count of negative values found before stopping

# Functions

Functions help in organizing code, improving readability, efficiency, and avoiding repetition.

- Abstraction makes sure you produce fewer bugs
- Parameters give flexibility, default values make sure that functions can be used with minimal input

```
def f(x, a):
    return a*x**2

y = f(2.0, 3.0) # y = 3 * (2.0)^2 = 12.0
```

Define default parameters to make functions easier to use:

```
def f(x, a=5.0):
    return a*x**2

y = f(2.0) # y = 5 * (2.0)^2 = 20.0
```

# Exercise: Functions practice

Define a function that computes the factorial of a number, $n!$:

$$n! = 1 \times 2 \times 3 \times \ldots \times (n-1) \times n$$

Compute $\exp(x)$ using the Taylor series, iterate until the change is smaller than $1 \cdot 10^{-6}$:

$$\exp(x) = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots + \frac{x^n}{n!}$$

# Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion
- A recursive function includes a call to itself (a function within a function)
  - This could lead to infinite calls;
  - A base case is required so that recursion is stopped;
  - Base case does not call itself, simply returns.



# Advanced topic: Recursion

- In order to understand recursion, one must first understand recursion
- A recursive function includes a call to itself (a function within a function)
  - This could lead to infinite calls;
  - A base case is required so that recursion is stopped;
  - Base case does not call itself, simply returns.



# Recursion: example

```
1 def mystery(a, b):
2   if b == 1:
3       # Base case
4       return a
5   else:
6       # Recursive function call
7       return a + mystery(a, b-1)
```

- What does this function do?
- Can you spot the error?
- How deep can you go? Which values of b don't work anymore?

# Practice

Define a function that computes the factorial of a number, $n!$, using recursion:

$$n! = 1 \times 2 \times 3 \times \ldots \times (n-1) \times n$$

# Lambda functions (1)

Consider the mathematical function $f(x) = x^2 + e^x$ defined as a Python function block:

```
def f(x):
    from math import exp
    return x**2 + exp(x)
```

Lambda functions provide a more compact way of defining simple functions, without the need to create a full function block:

```
f = lambda x: x**2 + exp(x)
f(2.0) # Evaluates to 11.38905609893065
```

This is often used when passing functions as arguments to other functions, e.g. in integration or optimization routines.

# List comprehension basics

Sometimes, loops can be replaced by more compact constructs called *list comprehensions*.

```
L = [1,2,3,4,5]
for i in range(len(L)):
   L[i] = L[i]**2
```

This can be replaced by:

```
L = [x**2 for x in L]
```

List comprehensions can also include conditions:

```
L = [x**2 for x in L if x % 2 == 0]
```

# Python classes

Sometimes, you may end up with many functions and data that are related to each other. In such cases, it is often useful to group them together into a single unit called a class.
A class is a blueprint for creating objects that share common properties and behaviors:

```python
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
        self.cuteness_level = 1

    def purr(self, cute_pts=10):
        print("Purr... =^.^= ")
        self.cuteness_level += cute_pts
        return

    def scratch(self, harshness=5):
        print("Scratch! >.< ")
        self.cuteness_level -= harshness
        return

    def get_cuteness(self):
        return self.cuteness_level
```

```python
my_cat = Cat("Arturo", "Gray")
my_cat.purr()
my_cat.scratch()
result = my_cat.get_cuteness()
print(f"Cuteness of {my_cat.name}: {result}")
```

```python
sweet_cat = Cat("Pinot", "White")
angry_cat = Cat("Rufus", "Black")

angry_cat.scratch(15)
sweet_cat.purr(25)

cats = [my_cat, sweet_cat, angry_cat]
cuteness_levels = [cat.get_cuteness() for cat in
      cats]
print(cuteness_levels)
```

# Exercise: Classes and objects

Create a class `IdealGas` that represents an ideal gas using the ideal gas law. Store the name, number of moles and the gas constant as attributes. Implement a method `compute_pressure(V,T)` that computes the pressure given volume `V` and temperature `T`.

# Today's outline

- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

# Using Modules in Python (1)

Modules are files containing Python code, used to organize functionalities and reuse code across projects. To use a module, it must first be imported using the `import` keyword. Here, we import the entire `math` module:

```
>>> import math
```

Once imported, use the dot notation to access functions and variables defined in the module:

```
>>> math.sqrt(16)
4.0
```

You can import specific attributes from a module using the `from ... import ...` syntax:

```
>>> from math import sqrt
>>> sqrt(16)
4.0
```

# Using Modules in Python (2)

Alias module names using the `as` keyword to shorten module names and avoid naming conflicts:

```
>>> import numpy as np
```

To view the list of all functions and variables in a module, use the `dir()` function:

```
>>> import math
>>> dir(math)
```

Get help on how to use a module or a function using the `help()` function:

```
>>> help(math.sqrt)
```

# The math module

Many mathematical operations and concepts are available in the math module:

```
from math import pi,sin,sqrt,log10\
,exp,floor,ceil,factorial,inf,log
print(pi)
print(sin(0.2*pi))
print(sqrt(2))
print(log10(10_000))
print(exp(1))
print(log(exp(2)))
print(floor(2.57))
print(ceil(2.57))
print(floor(-2.57))
print(round(2.4))
print(round(4.5))
print(factorial(5))
print(f"1 divided by infinity equals {1/inf}")
```

```
3.141592653589793
0.5877852522924731
1.4142135623730951
4.0
2.718281828459045
2.0
2
3
-3
2
4
120
1 divided by infinity equals 0.0
```

# Practice

Use the math module to compute $y = \sin(x)$ for 9 equidistant points $x \in [0, 2\pi]$, including end-points.

- Use *list comprehensions* to generate the lists `x` and `y`

# Today's outline


- Introduction
- Refresher
- New topics in programming
- Modules
- Conclusions
- Exercises

# In conclusion...

- Python: A versatile development language. Easy to use libraries makes this language multi-purpose and easy to use.
- Programming basics: variables, operators and functions, locality of variables, modules and recursive operations
- For now: exercises on slide deck and Python modules

# Practice vectors and arrays

1. Create a list `x` with the elements:
   - `[2, 4, 6, 8, ..., 16]`
   - `[0, 0.5, 2/3, 3/4, ..., 99/100]`
2. Create a list `x` with the elements: $x_n = \frac{(-1)^n}{2n-1}$ for $n = 1, 2, 3, \ldots, 200$. Find the sum of the first 50 elements $x_1, \ldots, x_{50}$.
3. Let `x = list(range(20, 201, 10))`. Create a list `y` of the same length as `x` such that:
   - `y[i] = x[i] - 3`
   - `y[i] = x[i]` for every even index *i* and `y[i] = x[i] + 11` for every odd index *i*.
4. Let `T = np.array([[3, 4, 6], [1, 8, 6], [-4, 3, 6], [5, 6, 6]]).T`. Perform the following operations on T:
   - Retrieve a list consisting of the 2nd and 4th elements of the 3rd row.
   - Find the minimum of the 3rd column.
   - Find the maximum of the 2nd row.
   - Compute the sum of the 2nd column
   - Compute the mean of the row 1 and the mean of row 3

# Practice plotting

1. Plot the functions $f(x) = x$, $g(x) = x^3$, $h(x) = e^x$ and $z(x) = e^{x^2}$ over the interval $[0, 4]$ on the normal scale and on the log-log scale. Use an appropriate sampling to get smooth curves. Describe your plots by using the functions: `plt.xlabel`, `plt.ylabel`, `plt.title` and `plt.legend`.

2. Make a plot of the functions: $f(x) = sin(1/x)$ and $g(x) = cos(1/x)$ over the interval $[0.01, 0.1]$. How do you create `x` so that the plots look sufficiently smooth?

# Practice control flow and loops (1)

1. Write a function that uses two logical input arguments with the following behaviour:

$$f(\text{true}, \text{true}) \mapsto \text{false}$$
$$f(\text{false}, \text{true}) \mapsto \text{true}$$
$$f(\text{true}, \text{false}) \mapsto \text{true}$$
$$f(\text{false}, \text{false}) \mapsto \text{false}$$

2. Write a function that computes the factorial of x:

$$f(x) = x! = 1 \times 2 \times 3 \times 4 \times \ldots \times x$$

- Using a loop-construction
- Using recursion

# Practice control flow and loops (2)

1. Write a function that computes the exponential function using the Taylor series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

   until the last term is smaller than $10^{-6}$.

2. Use a script to compute the result of the following series:

$$f_n = \sum_{n=1}^{\infty} \frac{1}{\pi^2 n^2}$$

   This should give you an indication of the fraction this series converges to.

   - Now plot in two vertically aligned subplots i) The result as a function of $n$, and ii) the difference with the earlier mentioned fraction as a function of n. For the latter, consider carefully the axis scale!

# Practice logical indexing

1. Let `x = np.linspace(-4,4,1000)`, $y_1 = 3x^2 - 4x - 6$ and $y_2 = 1.5x - 1$. Use logical indexing to determine function $y_3 = \max(\max(y_1, y_2), 0)$. Plot the function.
2. Consider these data concerning the age (in years), length (in cm) and weight (in kg) of twelve adult men: `A = [41 25 33 29 64 34 47 38 49 32 26 26]; H = [165 186 177 190 156 174 164 205 184 190 165 171]; W = [75 90 97 60 74 65 101 85 91 75 87 70];`.
   - Calculate the average of all vectors (age, weight and length).
   - Combine the command `length` with logical indexing to determine how many men in the group are taller than 182 cm.
   - What is the average age of men with a body-mass index ($B \equiv \frac{W}{L^2}$ with $W$ in kg and $L$ in m) larger than 25? And for men with a $B < 25$?
   - How many men are older than the average and at the same time have a BMI below 25?

# Practice algorithm: Fourier series for heat equation

The unsteady 1D heat equation in 1D in a slab of material is given as:

$$\frac{\partial T}{\partial t} = k\frac{\partial^2 T}{\partial x^2}$$

We can express the temperature profile $T(x,t)$ in the slab using a Fourier sine series. For an initial profile T(x,0) = 20 and fixed boundary values T(0,t) = T(L,t) = 0, the solution is given as:

$$T(x,t) = \sum_{n=1}^{n=\infty} \frac{40(1-(-1)^n)}{n\pi} \sin\left(\frac{n\pi x}{L}\right) \exp\left(-kt\frac{n\pi^2}{L}\right)$$

- Create a script to solve this equation using loops and/or conditional statements

# Python and Programming 2

## Programming workflow and advanced features

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline

- Scientific computing
- Plotting with Matplotlib
- Coding style
- Debugging and profiling
- Concluding remarks

# Introduction to NumPy (1)

NumPy is a powerful library for numerical computing in Python. It introduces the multidimensional array (`ndarray`) data structure which is central to numerical computing tasks. To start using NumPy, you need to import it first:

```
import numpy as np
```

Creating arrays from Python lists and accessing elements:

```
arr = np.array([1, 2, 3, 4, 5])
arr[0]
1
```

Create arrays with predetermined values:

```
np.zeros(5) # Creates an array of five zeros
np.ones((3, 3)) # Creates a 3x3 array of ones - note that the input argument is a tuple
```

# Introduction to NumPy (2)

Useful array creation functions:

```
arr = np.arange(0, 10, 2) # Creates an array with values from 0 to 10, step 2
arr2 = np.linspace(0, 1, 5) # Creates an array with 5 values evenly spaced between 0 and 1
arr3 = np.logspace(-3,2,6) # Creates an array spaced evenly on a logscale
```

Array operations:

```
arr * 2 # Multiplies every element by 2
arr + np.array([5, 4, 3, 2, 1]) # Element-wise addition
```

Inspecting your array:

```
arr.shape # Returns the dimensions of the array (tuple)
arr.shape[0] # Returns the number of rows ([1] for columns)
len(arr) # Returns the size of the first dimension
arr.size # Returns the total number of elements in arr
```

# Introduction to NumPy (3)

Zeros- or ones-filled arrays:

```
many_zeros = np.zeros_like(arr) # Array with zeros of size like arr, also: np.zeros
just_ones = np.ones((2,4)) # Create array with ones of size (2,4), also: np.ones_like
identity = np.eye((3,3)) # Identity matrix (zeros with ones on the main diagonal)
```

Or random numbers, useful for e.g. stochastic simulations. Note that these are NumPy intrinsic methods, and differ from the ones in the `random` module.

```
np.random.rand() # Single random floating point number [0,1)
0.11879317099184983

np.random.rand(2,3)
array([[0.36482694, 0.47204049, 0.20908193],
       [0.76626339, 0.64075302, 0.82440279]])

np.random.randint(1,10,5) # 5 random integers from 1 (inclusive) until 10 (exclusive)
array([9, 1, 6, 4, 6])
```

# Math with NumPy (1)

NumPy provides a rich set of functions to perform mathematical operations on arrays. Let's explore some categories of these operations.

- Linear Algebra:

```
np.dot(arr, arr) # Dot product (alternative: arr @ arr)
np.linalg.norm(arr) # Euclidean norm (L2 norm)
```

- Trigonometric Functions:

```
np.sin(arr) # Sine of each element
np.cos(arr) # Cosine of each element
```

- Statistical functions to analyze data:

```
np.mean(arr) # Mean value of the array elements
np.std(arr) # Standard deviation of the array elements
```

# Math with NumPy (2)

**Table: Useful NumPy Functions for Beginners**

| Function | Description |
|---|---|
| `np.array` | Create a NumPy array |
| `np.zeros` | Create an array filled with zeros |
| `np.ones` | Create an array filled with ones |
| `np.arange` | Create an array with evenly spaced values |
| `np.linspace` | Create an array with a specified number of evenly spaced values |
| `np.sin` | Sine function |
| `np.cos` | Cosine function |
| `np.tan` | Tangent function |
| `np.exp` | Exponential function |
| `np.log` | Natural logarithm |
| `np.sqrt` | Square root function |
| `np.dot` | Dot product of two arrays |
| `np.linalg.norm` | Calculate the norm of an array |
| `np.mean` | Calculate the mean of an array |
| `np.std` | Calculate the standard deviation of an array |

# Array operations (1)

In Python, NumPy ndarrays enable efficient operations on arrays, particularly mathematical operations associated with linear algebra. Let's look at how we can create and manipulate matrices using NumPy ndarrays:

- Manually creating matrices (2D ndarrays):

```
import numpy as np
matrix = np.array([[1, 2],
                   [3, 4]]) # Matrices are arrays of arrays(rows)
```

- Reshaping, slicing and modifying matrices (try for yourself and print each new result):

```
mat = np.arange(25).reshape(5,5)
col = mat[:,2] # Get column index 2
row = mat[3,:] # Get row index 3
sub = mat[1:4,1:4] # Save submatrix
mat[1,:] = row # Replace row index 1 by 'row'
mat[mat>5] = -1 # Conditional slicing/update of values
```

# Array operations (2)

The real power of NumPy lies in the vectorized computations:

- Omit loops, write computations in natural language
- Shorter code
- Much, much faster

```
import numpy as np
import time

start = time.time()
x = np.linspace(0,2*np.pi,10000000)
y = np.exp(-x) * (2+np.sin(2*np.pi*x))
total_time = time.time() - start

print(f'{total_time = }')
```

```
total_time = 0.35589122772216797
```

# Array operations (3)

If you do need to iterate over the elements:

- Think twice if you *really* have to
- If the answer is still yes, you can use `np.nditer` or `np.ndenumerate`:

```python
import numpy as np

# A 2D array
data = np.array([[11,12,13], [14,15,16]])

# Loop over each element using nditer
for val in np.nditer(data):
    print(f"Value: {val}")

# Loop using ndenumerate (gives index + value)
for i,val in np.ndenumerate(data):
    print(f"Enumerate index {i} has value {val};
          also {data[i]}")
```

```
Value: 11
Value: 12
Value: 13
Value: 14
Value: 15
Value: 16
Enumerate index (0, 0) has value 11; also 11
Enumerate index (0, 1) has value 12; also 12
Enumerate index (0, 2) has value 13; also 13
Enumerate index (1, 0) has value 14; also 14
Enumerate index (1, 1) has value 15; also 15
Enumerate index (1, 2) has value 16; also 16
```

# Practice

Given a function $f(x) = x^2 + 2x - 4$

- Create a large `np.linspace` of $x \in [0, 20]$ e.g. with 1M numbers
- Compute $f(x)$ iteratively (i.e. with a loop)
- Compute $f(x)$ vectorized (i.e. using NumPy operations)
- Compare timings

# Assignment of arrays by reference

Careful with assignment, default is a reference to the same object as observed with lists:

```
arr = np.arange(0,10,1)
print(arr)
[0 1 2 3 4 5 6 7 8 9]
new_arr = arr
new_arr2 = arr.copy()
new_arr[4] = 11
print(new_arr)
[ 0 1 2 3 11 5 6 7 8 9]
print(arr) # :mind_blown: we did not change this, did we?
[ 0 1 2 3 11 5 6 7 8 9]
```

Saving and loading matrices:

```
np.save('my_matrix.npy', matrix) # Save the matrix to a file
loaded_matrix = np.load('my_matrix.npy') # Load the matrix from a file
```

# Today's outline

- Scientific computing
- Plotting with Matplotlib
- Coding style
- Debugging and profiling
- Concluding remarks

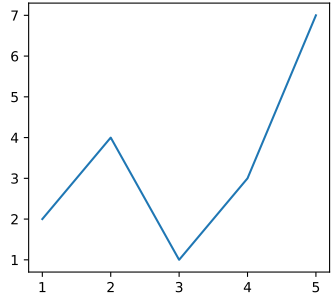
# Introduction to Matplotlib

Matplotlib is a Python library used widely for creating static, animated, and interactive visualizations. To start, we first import the `pyplot` module from `matplotlib`.

```
1 import matplotlib.pyplot as plt
```

Creating a simple line plot:

```
2 x = [1, 2, 3, 4, 5]
3 y = [2, 4, 1, 3, 7]
4 plt.plot(x, y)
5 plt.show()
```




In Jupyter Notebooks, use `%matplotlib` to display figures in a separate window, or `%matplotlib inline` to display in the notebook.

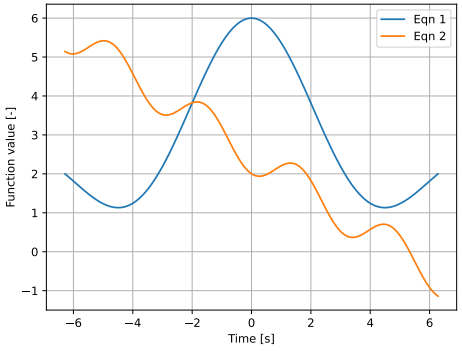
# Brushing up the graph

Adding multiple plots, axis labels and a legend:

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(-2*np.pi, 2*np.pi,101)
y1 = 4*np.sin(t)/t + 2
y2 = np.sin(t)**2 - t/2 + 2

plt.plot(t,y1,label='Eqn 1')
plt.plot(t,y2,label='Eqn 2')
plt.xlabel('Time [s]')
plt.ylabel('Function value [-]')
plt.legend()
plt.grid()
plt.show()
```



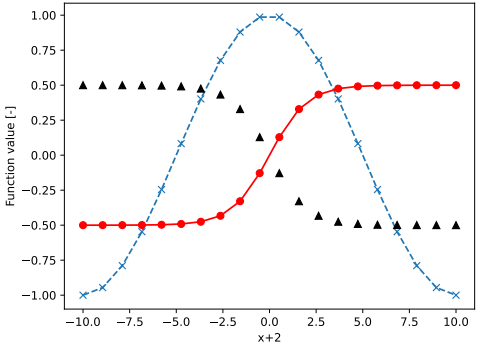
# Line plot styles

Changing markers, line styles:

```
import matplotlib.pyplot as plt
import numpy as np

t = np.linspace(-10.0, 10.0, 20)
p = np.cos(np.pi * t/10)
q = 1/(1+np.exp(-t)) - 0.5

plt.plot(t,p,'--x') # Dashed line, cross
plt.plot(t,q,'r-o') # Red, line, circles
plt.plot(t,-q,'k^') # Black triangle marker
plt.xlabel('x+2')
plt.ylabel('Function value [-]')
plt.show()
```



# Subplots

Multiple plots in a single figure are made by specifying a grid using subplot:

```
x = np.linspace(0,2*np.pi,1000)
y1,y2 = np.sin(x), 2*np.cos(x)

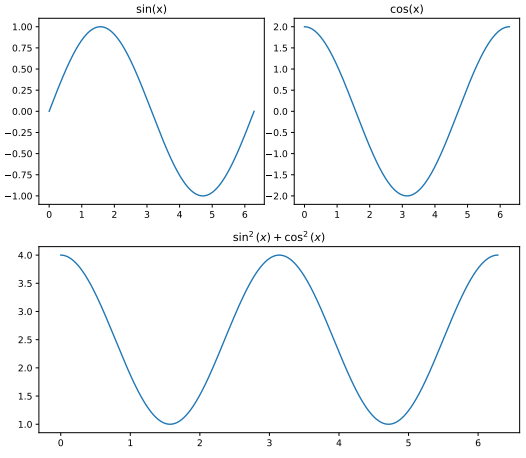
# Specify figure size
fig = plt.figure(figsize=(8, 7))

# First index of 2x2 grid (top-left)
ax1 = plt.subplot(2, 2, 1)
ax1.plot(x,y1)
ax1.set_title('sin(x)')

# Second index of 2x2 grid (top-right)
ax2 = plt.subplot(2, 2, 2)
ax2.plot(x,y2)
ax2.set_title('cos(x)')

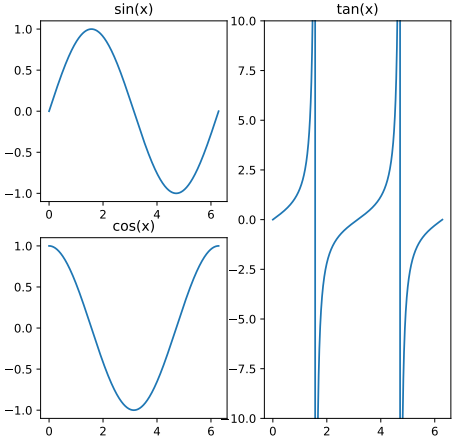
# Second index of 2x1 grid (whole bottom)
ax3 = plt.subplot(2, 1, 2)
ax3.plot(x,y1**2+y2**2)
ax3.set_title(r'$\sin^2(x)+\cos^2(x)$')

plt.tight_layout()
plt.show()
```



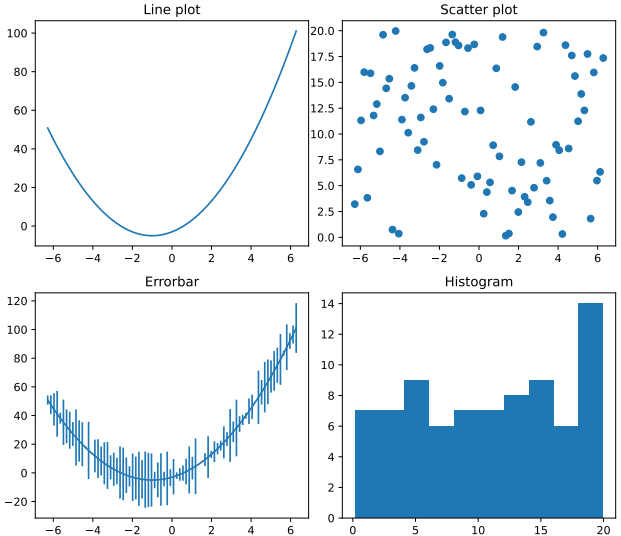
# Practice

Try to create the following figure:



# 2D plot styles

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x = np.linspace(-2*np.pi,2*np.pi,80)
5  y2 = 2*x**2 + 4*x - 3
6  y3 = 20*np.random.rand(y2.size)
7
8  ax1 = plt.subplot(2, 2, 1)
9  ax1.plot(x,y2)
10 ax1.set_title('Line plot')
11
12 ax2 = plt.subplot(2, 2, 2)
13 ax2.scatter(x,y3)
14 ax2.set_title('Scatter plot')
15
16 ax3 = plt.subplot(2, 2, 3)
17 ax3.errorbar(x,y2,yerr=y3)
18 ax3.set_title('Errorbar')
19
20 ax4 = plt.subplot(2,2,4)
21 ax4.hist(y3)
22 ax4.set_title('Histogram')
23
24 plt.show()
```



# Surface plot

We draw a surface plot of $f(x,y) = xye^{(-x^2-y^2)}$:

```
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
import matplotlib.pyplot as plt
import numpy as np

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

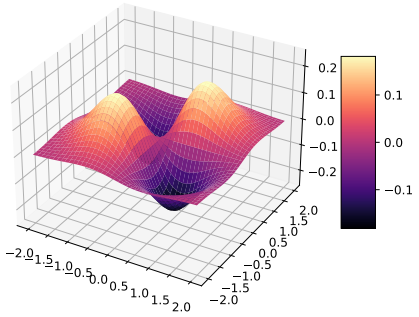
# Make data.
x = np.arange(-2, 2, 0.025)
y = np.arange(-2, 2, 0.025)
x,y = np.meshgrid(x, y)
z = x * y * np.exp(-x**2 - y**2)

# Plot the surface.
surf = ax.plot_surface(x,y,z,
      cmap=cm.magma,linewidth=0, antialiased=False)

# Customize the z axis.
ax.set_zlim(-0.25, 0.25)

# Add a color bar which maps values to colors.
fig.colorbar(surf, shrink=0.5, aspect=5)

plt.show()
```



# Today's outline

- Scientific computing
- Plotting with Matplotlib
- Coding style
- Debugging and profiling
- Concluding remarks

## Make a habit of the following adage

MAKE IT **WORK**

MAKE IT **RIGHT**

MAKE IT **FAST**

# Make it work

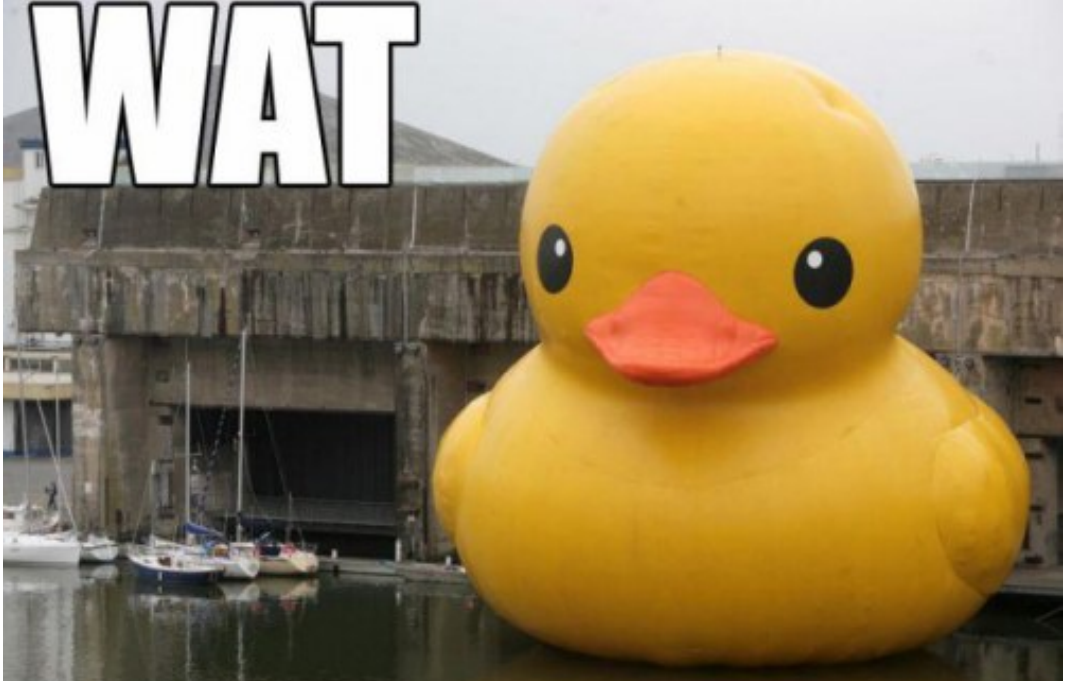
Use the building blocks of previous lecture to create an algorithm:

1. *Problem analysis*
   Contextual understanding of the nature of the problem to be solved
2. *Problem statement*
   Develop a detailed statement of the mathematical problem to be solved with the program
3. *Processing scheme*
   Define the inputs and outputs of the program
4. *Algorithm*
   A step-by-step procedure of all actions to be taken by the program (*pseudo-code*)
5. *Program the algorithm*
   Convert the algorithm into a computer language, and debug until it runs
6. *Evaluation*
   Test all of the options and conduct a validation study

Now it's time to make it right!

# Interpret the following code

```
s = checksc()
if s:
    a = cb()
    b = cfrsp()
    if a < 5:
        if b > 5:
            a = gtbs()
        if a > b:
            ubx()
else:
    brn()
    gtbs()
```


WAT

# Enhancing Readability with Proper Indentation

Proper indentation is not just about syntax in Python; it significantly impacts the readability of the code. Python recommends 4 spaces for indentation. Let's see the difference:

```
s = checksc()
if s:
a = cb()
b = cfrsp()
if a < 5:
if b > 5:
a = gtbs()
if a > b:
ubx()
else:
brn()
gtbs()
```

```
s = checksc()
if s:
    a = cb()
    b = cfrsp()
    if a < 5:
        if b > 5:
            a = gtbs()
        if a > b:
            ubx()
else:
    brn()
    gtbs()
```

# Readable Variables and Function Names

Making use of descriptive variable and function names enhances the readability and understanding of the code.

```
s = checksc()
if s:
    a = cb()
    b = cfrsp()
    if a < 5:
        if b > 5:
            a = gtbs()
        if a > b:
            ubx()
else:
    brn()
    gtbs()
```

```
isAvailable = checkSchedule()
if isAvailable:
    bookCount = countBooks()
    freeShelfSpace = checkFreeShelfSpace()
    if bookCount < 5:
        if freeShelfSpace > 5:
            bookCount = visitBookStore()
        if bookCount > freeShelfSpace:
            useStorageBox()
else:
    burnBooks()
    visitBookStore()
```

# Avoiding Magic Numbers in the Code

Magic numbers are constant values without a name, which can reduce code readability.
Replacing them with named constants can enhance the understanding of the code.

```
isAvailable = checkSchedule()
if isAvailable:
    bookCount = countBooks()
    freeShelfSpace = checkFreeShelfSpace()
    if bookCount < 5:
        if freeShelfSpace > 5:
            bookCount = visitBookStore()
        if bookCount > freeShelfSpace:
            useStorageBox()
else:
    burnBooks()
    visitBookStore()
```

```
MAX_SHELF_SPACE = 5
MIN_BOOKS_REQUIRED = 5

isAvailable = checkSchedule()
if isAvailable:
    bookCount = countBooks()
    freeShelfSpace = checkFreeShelfSpace()
    if bookCount < MAX_SHELF_SPACE:
        if freeShelfSpace > MIN_BOOKS_REQUIRED:
            bookCount = visitBookStore()
        if bookCount > freeShelfSpace:
            useStorageBox()
else:
    burnBooks()
    visitBookStore()
```

# Now, That's More Like It!

Demonstrating the evolution of the script to a more readable and maintainable version.

```
s = checksc()
if s:
    a = cb()
    b = cfrsp()
    if a < 5:
        if b > 5:
            a = gtbs()
        if a > b:
            ubx()
else:
    brn()
    gtbs()
```

```
MAX_SHELF_SPACE = 5
MIN_BOOKS_REQUIRED = 5

isAvailable = checkSchedule()
if isAvailable:
    bookCount = countBooks()
    freeShelfSpace = checkFreeShelfSpace()
    if bookCount < MAX_SHELF_SPACE:
        if freeShelfSpace > MIN_BOOKS_REQUIRED:
            bookCount = visitBookStore()
        if bookCount > freeShelfSpace:
            useStorageBox()
else:
    burnBooks()
    visitBookStore()
```

# Writing readable code

- Good code reads like a book.
  - Write code as if it were paragraphs in a story
- Use comments to document design and purpose, not mechanics
- If you have $> 15$ lines of code, you can probably replace it by one or more functions: don't repeat yourself.
- Use meaningful variable and function names
  - `calculate_average_temperature()`
  - `grid_size = (100,100)`
- Document functions using docstrings
- Generalize: separate data from algorithms

# Writing efficient code

Numerical methods are about efficient techniques to compute your answer. A couple of guidelines:

- Avoid loops
- Especially avoid nested loops
- Use inherent matrix operations when possible
- Reduce IO (i.e. reading / writing to and from files)
- Don’t run scripts from network disks
- Pre-allocate your arrays, that means, making it as large as the maximum required size for your particular problem
- Use Python’s built-in modules for performance profiling, such as cProfile or timeit, to test the execution times

# Efficiency: Measuring execution time example

```python
import numpy as np
import time

x = np.linspace(0, 10, 1000001)
start_time = time.time()

y = []
for cr in range(len(x)):
    y.append(np.sin(2 * np.pi * x[cr]))

end_time = time.time()
print(f'Execution time: {end_time - start_time}
    seconds')
```

```python
import numpy as np
import time

x = np.linspace(0, 10, 1000001)
start_time = time.time()

y = np.sin(2 * np.pi * x)

end_time = time.time()
print(f'Execution time: {end_time - start_time}
    seconds')
```

## Make a habit of the following adage

MAKE IT **WORK**

MAKE IT **RIGHT**

MAKE IT **FAST**

# Make a habit of the following adage

1. *Make it work*
   Create an algorithm that does the intended job. Make sure it works, and works repeatedly. Test and verify frequently. Add *todo* comments when you're not sure about a certain decision.
2. *Make it right*
   Refactor the code to improve the code design. Insert functions, comments, compartmentalize it. Get rid of magic numbers, use sensible variable names. Check input. Test and verify. Align with the team!
3. *Make it fast*
   Measure and tune the performance of your code (profiling tool). In Python, vectorized calculations are much (!) faster than for-loops. Use sensible numerical techniques (e.g. higher-order integration).

Program by iterating over these aspects multiple times, starting at the fine-grained level, working your way up.

# Today's outline

- Scientific computing
- Plotting with Matplotlib
- Coding style
- Debugging and profiling
- Concluding remarks

# Errors in computer programs

The following symptoms can be distinguished:

- Unable to execute the program
- Program crashes, warnings or error messages
- Never-ending loops
- Wrong (unexpected) result

Three error categories:

| | |
|---|---|
| Syntax errors | You did not obey the language rules. These errors prevent running or compilation of the program. |
| Runtime errors | Something goes wrong during the execution of the program resulting in an error message (problem with input, division by zero, loading of non-existent files, memory problems, etc.) |
| Semantic errors | The program does not do what you expect, but does what have told it to do. |

# Validation

- Testcases: run the program with parameters such that a known result is (should be) produced.
- Testcases: what happens when unforeseen input is encountered?
  - More or fewer arguments than anticipated? (Python uses `*args` and `**kwargs` to create a varying number of input arguments, and to check the number of given input arguments
  - Other data types than anticipated? How does the program handle this? Warnings, error messages (crash), NaN or worse: a program that silently continues?
- For physical modeling, we typically look for analytical solutions
  - Sometimes somewhat stylized cases
  - Possible solutions include Fourier-series
  - Experimental data

But: validation can only tell you *if* something is wrong, not *where* it went wrong.

# The debugger (1)

- No-one can write a 1000-line code without making errors
  - If you can, please come work for us
- One of the most important skills you will acquire is debugging.
- Although it can be frustrating, debugging is one of the most intellectually rich, challenging, and interesting parts of programming.
- In some ways, debugging is like detective work. You are confronted with clues, and you have to infer the processes and events that led to the results you see.
- Actually, you are the detective, the murderer and the victim at the same time.

*"When you have eliminated the impossible, whatever remains, however improbable, must be the truth."*

— A. Conan Doyle, The Sign of Four

# The debugger (2)

A debugger can help you to:

- Pause a program at a certain line: set a *breakpoint*
- Check the values of variables during the program
- Controlled execution of the program:
  - One line at a time
  - Run until a certain line
  - Run until a certain condition is met (conditional breakpoint)
  - Run until the current function exits
- Note: You may end up in the source code of Python functions!
- Check Canvas (Python Crash Course section) for a demonstration of the debugger.

# Recursive Fibonacci

- Create a program that computes the $n$-th Fibonacci number using recursion:
  $F_n = F_{n-1} + F_{n-2}$ with $F_1 = 1$ and $F_2 = 1$

```
1  def fibonacci_recursive(N):
2      """
3      Prints out the Nth Fibonacci number to the screen.
4      SYNTAX: fibonacci_recursive(N)
5      """
6      if N > 2:
7          Nminus1 = fibonacci_recursive(N-1)
8          Nminus2 = fibonacci_recursive(N-2)
9          out = Nminus1 + Nminus2
10     elif N == 1 or N == 2:
11         out = 1
12     else:
13         raise ValueError('Input argument was invalid')
14     return out
```

- Place a breakpoint line 5 (click on dash or press F12), run `fibonacci_recursive(5)`
- Explore the function of step F10, step into F11, and how the local workspace changes
- Stop the debugger (red stop button on top, or Shift + F5)
- Right-click the breakpoint, select *Set/modify condition*, enter `N==2`, run again.

# Today's outline

- Scientific computing
- Plotting with Matplotlib
- Coding style
- Debugging and profiling
- Concluding remarks

# Advanced concepts

- Object oriented programming: classes and objects
- Memory management: some programming languages require you to allocate computer memory yourself (e.g. for arrays)
- External libraries: in many cases, someone already built the general functionality you are looking for
- Compiling and scripting (“interpreted”); compiling means converting a program to computer-language before execution. Interpreted languages do this on the fly.
- Parallellization: Distributing expensive calculations over multiple processors or GPUs.

## Make a habit of the following adage

MAKE IT **WORK**

MAKE IT **RIGHT**

MAKE IT **FAST**

# Make a habit of the following adage

1. *Make it work*
   Create an algorithm that does the intended job. Make sure it works, and works repeatedly. Test and verify frequently. Add *todo* comments when you're not sure about a certain decision.
2. *Make it right*
   Refactor the code to improve the code design. Insert functions, comments, compartmentalize it. Get rid of magic numbers, use sensible variable names. Check input. Test and verify. Align with the team!
3. *Make it fast*
   Measure and tune the performance of your code (profiling tool). In Python, vectorized calculations are much (!) faster than for-loops. Use sensible numerical techniques (e.g. higher-order integration).

Program by iterating over these aspects multiple times, starting at the fine-grained level, working your way up.

# Numerical errors in computer simulations

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

# Example 1

Start your spreadsheet program (Excel, ...)

Enter:

| Cell | Value |
|---|---|
| A1 | `0.1` |
| A2 | `=(A1*10)-0.9` |
| A3 | `=(A2*10)-0.9` |
| A4 | `=(A3*10)-0.9` |

(repeat until A30)

What's happening?

Enter:

| Cell | Value |
|---|---|
| A1 | `2` |
| A2 | `=(A1*10)-18` |
| A3 | `=(A2*10)-18` |
| A4 | `=(A3*10)-18` |

(repeat until A30)

# Example 2

Start Python

Investigate the result of `np.sin(1e40 * np.pi)`

Create a vector `v` containing the powers of 10, e.g., from $10^0$ up to $10^{40}$, and solve `np.sin(v * np.pi)`:

```
import numpy as np
import matplotlib.pyplot as plt

v = np.logspace(0, 40, 41)
y = np.sin(v * np.pi)
plt.loglog(v, np.abs(y)) # Double log plot, values on y-axis must be positive.
plt.show()
```

# Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters
- Roundoff- and truncation errors
- Break errors

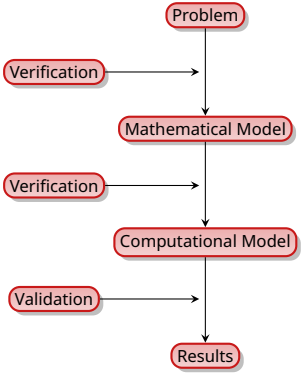
# Verification and validation

**Verification**

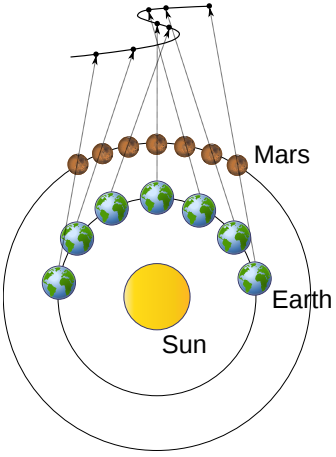
Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.

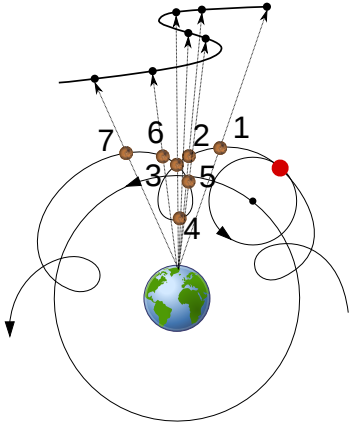
**Validation**

Validation is the process of determining the degree to which a model is an accurate representation of the real world from the perspective of the intended uses of the model



# Verification of the physical model




- The perceived orbit of Mars from Earth shows a zig-zag (in contrast to the Sun, Mercury, Venus)
- Even though they were not 'right', Earth-centered models (Ptolemy) were still valid

# Be aware of your uncertainties

**Aleatory uncertainty**

Uncertainty that arises due to inherent randomness of the system, features that are too complex to measure and take into account

**Epistemic uncertainty**

Uncertainty that arises due to lack of knowledge of the system, but could in principle be known

# Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters
- Roundoff- and truncation errors
- Break errors

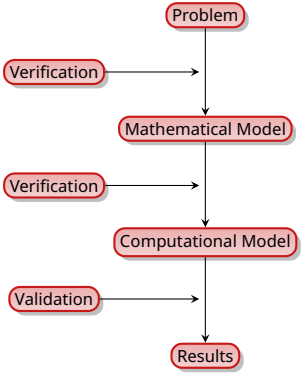
# Verification and validation

**Verification**

Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.

**Validation**

Validation is the process of determining the degree to which a model is an accurate representation of the real world from the perspective of the intended uses of the model



# Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters
- Roundoff- and truncation errors
- Break errors

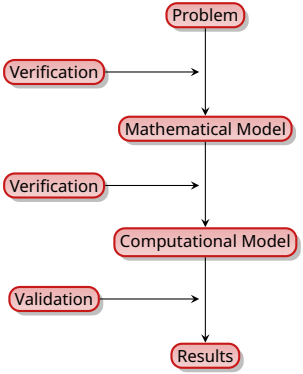
# Verification and validation

**Verification**

Verification is the process of mathematically and computationally assuring that the model computes the equations you intended to implement.

**Validation**

Validation is the process of determining the degree to which a model is an accurate representation of the real world from the perspective of the intended uses of the model



# Errors in computer simulations

In this lecture I will outline different numerical errors that can appear in computer simulations, and how these errors can affect the simulation results.

- Errors in the mathematical model (physics)
- Errors in the program (implementation)
- Errors in the entered parameters
- Roundoff- and truncation errors
- Break errors

# Significant digits

A numerical result $\tilde{x}$ is an approximation of the real value $x$.

- Absolute error

$$\delta = |\tilde{x} - x|, x \neq 0$$

- Relative error

$$\frac{\delta}{\tilde{x}} = |\frac{\tilde{x} - x}{\tilde{x}}|$$

- Error margin

$$\tilde{x} - \delta \leq x \leq \tilde{x} + \delta$$

$$x = \tilde{x} \pm \delta$$

# Significant digits

- $\tilde{x}$ has $m$ significant digits if the absolute error in $x$ is smaller or equal to 5 at the $(m+1)$-th position:

$$10^{q-1} \leq |\tilde{x}| \leq 10^{q}$$

$$|x - \tilde{x}| = 0.5 \times 10^{q-m}$$

- For example:

$$x = \frac{1}{3}, \tilde{x} = 0.333 \Rightarrow \delta = 0.00033333\ldots$$

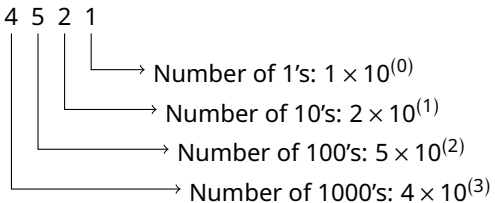
3 significant digits

# Today's outline

- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

# Representation of numbers

- Computers represent a number with a finite number of digits: each number is therefore an approximation due to roundoff and truncation errors.
- In the decimal system, a digit $c$ at position $n$ has a value of $c \times 10^{n-1}$




$$(4521)_{10} = 4 \times 10^3 + 5 \times 10^2 + 2 \times 10^1 + 1 \times 10^0$$

# Representation of numbers

- You could use another basis, computers often use the basis 2:

$$
\begin{aligned}
(4521)_{10} &= 1\times 2^{12} + 0\times 2^{11} + 0\times 2^{10} + 0\times 2^{9} + 1\times 2^{8} + \dots \\
&\dots 1\times 2^{7} + 0\times 2^{6} + 1\times 2^{5} + 0\times 2^{4} + \dots \\
&\dots 1\times 2^{3} + 0\times 2^{2} + 0\times 2^{1} + 1\times 2^{0} \\
&= (1000110101001)_2
\end{aligned}
$$

- In general:

$$(c_m \dots c_1 c_0)_q = c_0 q^0 + c_1 q^1 + \dots + c_m q^m, c \in \{0, 1, 2, \dots, q-1\}$$

# Representation of numbers

- Numbers are stored in binary in the memory of a computer, in segments of a specific length (called a *word*).
- We distinguish multiple types of numbers:
  - Integers: $-301, -1, 0, 1, 96, 2293, \ldots$
  - Floating points: $-301.01, 0.01, 3.14159265, 14498.2$
- A binary integer representation looks like the following bit sequence:

$$z = \sigma\left(c_0 2^0 + c_1 2^1 + \ldots + c_{\lambda-1} 2^{\lambda-1}\right)$$

  $\sigma$ is the sign of $z$ (+ or -), and $\lambda$ is the length of the word
- Endianness: the order of bits stored by a computer

# Excercise

- Convert the following decimal number to base-2: 214

$$214_{10} = 11010110_2$$

- Excel:
  - Decimal: `=DEC2BIN(214)`
  - Octal: `=DEC2OCT(214)`
  - Hexadecimal: `=DEC2HEX(214)`
- Python:
  - Decimal: `bin(214)`
  - Octal: `oct(214)`
  - Hexadecimal: `hex(214)`

# Try yourself

Try the following commands in Python:

| Command | Result |
|---|---|
| `np.iinfo(np.int32).min` | `-2147483648` |
| `np.iinfo(np.int32).max` | `2147483647` |
| `i = np.int16(np.iinfo(np.int32).max)` | `i = 32767` |
| `type(i)` | `<class 'numpy.int16'>` |
| `i = i + 100` | `i = 32767` |
| `np.finfo(np.float64).max` | `1.7976931348623157e+308` |
| `f = 0.1` | |
| `type(f)` | `<class 'float'>` |
| `print(":.16f".format(f))` | `0.1000000000000000` |
| `print(":.20f".format(f))` | `0.10000000000000000555` |

# Representation of integer numbers

- In Python, integers are typically represented with at least 32 bits (though this can be larger depending on the Python implementation and system architecture). For a typical 64-bit Python installation, the integer type would have 64 bits.
- The set of numbers that a 32-bit integer 'int32' can represent in numpy is:

$$-2^{31} \leq z \leq 2^{31} - 1 \approx 2 \times 10^9$$

  where $z$ is a variable representing an `np.int32` type value.
- If, during a calculation, an integer number in Python exceeds its maximum limit, Python automatically promotes it to a larger type if possible. In the case of numpy, an overflow error will be thrown if a number exceeds the maximum value representable by the 'np.int32' type.
- How can a computer identify an overflow? Usually through error reporting mechanisms within the language. Python's numpy library, for instance, will throw an OverflowError when an operation results in a number larger than what can be represented by the current data type.

# Representation of real (floating point) numbers

- Formally, a real number is represented by the following bit sequence

$$x = \sigma\left(2^{-1} + c_2 2^{-2} + \ldots + c_m 2^{-m}\right) 2^{e-1023}$$

  Here, $\sigma$ is the sign of $x$ and $e$ is an integer value.

- A floating point number hence contains sections that contain the sign, the exponent and the mantissa

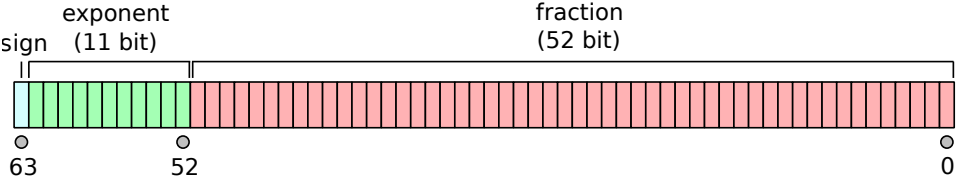



Image: Wikimedia Commons CC by-SA

# Roundoff error: truncation vs rounding

Real numbers are stored with a limited number of digits (or bits). When a number does not fit, the representation must be *approximated*.

Truncation (chopping)

- Extra digits are simply discarded.
- Always biases the result in the same direction.

$$\pi = 3.14159265 \implies 3.14$$

Rounding (to nearest)

- Last retained digit is rounded up or down.
- Error is smaller and approximately unbiased.

$$\pi = 3.14159265 \implies 3.14, \qquad 3.145 \Rightarrow 3.15$$

In binary floating-point, this corresponds to choosing the last bit as 0 or 1 when the exact value lies in between.

# Truncation vs rounding: numerical example

Simulate decimal arithmetic with 3 significant digits.

```python
import math

x = math.pi

# chopping to 3 significant digits
chop = float(f"{x:.3g}")

# rounding to nearest (default Python behavior)
rounding = round(x, 3)

print(f"'exact' : {x}...")
print(f"chopped : {chop}")
print(f"rounded : {rounding}")
print(f"errors : {chop-x}, {rounding-x}")
```

```
'exact' : 3.141592653589793...
chopped : 3.14
rounded : 3.142
errors : -0.0015926535897929917,
  0.0004073464102067881
```

- Chopping always underestimates here.
- Rounding alternates between over- and underestimation.

# Exercise: binary representation

Consider a binary floating point number, here represented as $x = 1.101101011_2$.

- The digits before the binary point represent powers of $2^0, 2^1, \ldots$ (from right to left)
- The digits after the binary point represent powers of $2^{-1}, 2^{-2}, \ldots$

Write a function to convert a string representation of a binary number (e.g. `'1.101101011'`) to its decimal value.

```
def binary_to_float(_bin: str) -> float:
    _pos,_neg = _bin.split('.')
    _float = 0
    # Count positive powers
    for i in range(len(_pos)):
        _float += int(_pos[i]) * 2**i
    # Count negative powers
    for i in range(1,len(_neg)+1):
        _float += int(_neg[i-1]) * 2**(-i)
    return _float

my_bin = '1.101'
print(binary_to_float(my_bin))
```

# Binary truncation (chopping)

Example: store $x = 1.101101011_2$ using only $p = 5$ bits.

$$x = 1.\underline{10110}1011_2 \implies \mathrm{fl}_{\mathrm{chop}}(x) = 1.10110_2$$

Properties:

- Always rounds *towards zero*.
- Error has fixed sign.
- Bias accumulates in long computations.

Relative error:

$$\frac{|x - \mathrm{fl}(x)|}{|x|} < 2^{-p}.$$

# Binary rounding (to nearest)

Same number, same precision ($p = 5$ bits):

$$x = 1.\underline{10110}1011_2$$

Look at the first discarded bit:

$$\mathrm{fl}_{\text{round}}(x) = 1.10111_2$$

Properties:

- Error is smaller than with chopping.
- Over- and underestimation occur with similar probability.
- Expected rounding error is approximately zero.

Bound on relative error:

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \leq \tfrac{1}{2}2^{-p}.$$

# Today's outline


- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

# Trigonometric, Logarithmic, and Exponential computations

- Processors can do logic and arithmetic instructions
- Trigonometric, logarithmic and exponential calculations are “higher-level” functions: exp, sin, cos, tan, sec, arcsin, arccos, arctan, log, ln, …
- Such functions can be performed using these “low level” instructions, for instance using a Taylor series:

$$\sin(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

# Trigonometric, Logarithmic, and Exponential computations

- These operations involve many multiplications and additions, and are therefore *expensive*
- Computations can only take finite time, for infinite series, calculations are interrupted at $N$

$$\sin(x) = \sum_{n=0}^{N} \frac{(-1)^n}{(2n+1)!} x^{2n+1} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots + \frac{(-1)^N}{(2N+1)!} x^{2N+1}$$

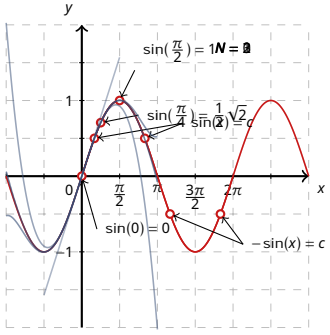
$$e^x = \sum_{n=0}^{N} \frac{x^n}{x!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \ldots + \frac{x^N}{N!}$$

- This results in a *break error*

# Algorithm for sine-computation

A computer may use a clever algorithm to limit the number of operations required to perform a higher-level function. A (fictional!) example for the computation of $\sin(x)$:

1. Use periodicity so that $0 \leq x \leq 2\pi$
2. Use symmetry ($0 \leq x \leq \frac{\pi}{2}$)
3. Use lookup tables for known values
4. Perform taylor expansion



# Today's outline


- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

# Arithmetic operations with binary numbers

Addition:

$0+0=0$
$0+1=1$
$1+0=1$
$1+1=0$ (carry one)

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ + & & 2 & 3 \\ \hline & 1 & 6 & 8 \end{array} \qquad \begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ + & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}$$

Subtraction:

$0-0=0$
$1-0=1$
$1-1=0$
$0-1=1$ (borrow one)

$$\begin{array}{rrrr} & 1 & 4 & 5 \\ - & & 2 & 3 \\ \hline & 1 & 2 & 2 \end{array} \qquad \begin{array}{rrrrrrrrr} & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ - & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ \hline & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \end{array}$$

- Multiplication and division are more expensive, and more elaborate

# Loss of digits

- During operations such as $+, -, \times, \div$, an error can add up
- Consider the summation of $x$ and $y$

$$\tilde{x} - \delta \leq x \leq \tilde{x} + \delta \quad \text{and} \quad \tilde{y} - \varepsilon \leq y \leq \tilde{y} + \varepsilon$$

$$(\tilde{x} + \tilde{y}) - (\delta + \varepsilon) \leq x + y \leq (\tilde{x} + \tilde{y}) + (\delta + \varepsilon)$$

- Consider the subtraction of $x$ and $y$

$$(\tilde{x} - \tilde{y}) - (\delta + \varepsilon) \leq x - y \leq (\tilde{x} - \tilde{y}) + (\delta + \varepsilon)$$

# Loss of digits: Example 1

$$\left.\begin{aligned} x = \pi, \tilde{x} = 3.1416 \\ y = 22/7, \tilde{y} = 3.1429 \end{aligned}\right\} \Rightarrow \left.\begin{aligned} \delta = \tilde{x} - x = 7.35 \times 10^{-6} \\ \varepsilon = \tilde{y} - y = 4.29 \times 10^{-5} \end{aligned}\right\}$$

$$x + y = \tilde{x} + \tilde{y} \pm (\delta + \varepsilon) \approx 6.2845 - 5.025 \times 10^{-5}$$

$$x - y = \tilde{x} - \tilde{y} \pm (\delta + \varepsilon) \approx -0.0013 + 3.55 \times 10^{-5}$$

- The absolute error is small ($\approx 10^{-5}$), but the relative error is much bigger (0.028).
- Adding up the errors results in a loss of significant digits!

# Catastrophic cancellation: concept

Catastrophic cancellation occurs when subtracting nearly equal numbers.
Relative error scales as

$$\frac{|\delta a - \delta b|}{|a - b|}.$$

When $|a - b|$ is small, relative error becomes large.

# Loss of digits: Summation order matters

Example:

$$10^8 + \sum_{i=1}^{N} 10^{-8}.$$

```
N = 1000000
big = 1e8
tiny = 1e-8

s1 = big
for _ in range(N):
    s1 += tiny

s2 = 0.0
for _ in range(N):
    s2 += tiny
s2 += big

print(s1, s2)
```

# Exercise: summation of many numbers

```python
import numpy as np
np.random.seed(42) # Make sure that the results are reproducible
x = 10.0**np.random.uniform(-12, 12, size=1_000_000)
```

- Sum the numbers in `x` in the given order.
- Now sort the numbers in ascending order and sum them again.
- Finally, sort the numbers in descending order and sum them again.
- What do you observe?

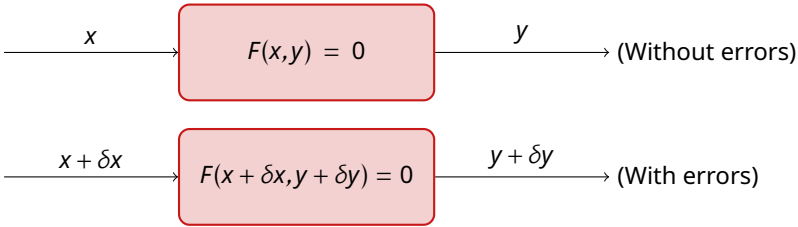
Specialized algorithms (Kahan summation, pairwise summation) can reduce the error.

# Loss of digits: Example 3

- Calculate $e^{-5}$
  - Use the Taylor series we programmed before ($e^x \approx \sum_{n=0}^{N} \frac{x^n}{n!}$ )
  - Calculate the first 26 terms ($N = 26$)
- Now repeat the calculation, but use for each calculation only 4 digits. What do you find? Use: `round(term, 4)`
- Without errors you would find: $e^{-5} = 0.006738$
- If you only use 4 digits in the calculations, you'll find 0.00998

# Badly (ill) conditioned problems

We consider a system $F(x,y)$ that computes a solution from input data. The input data may have errors:




$$y(x+\delta x) - y(x) \approx y'(x)\delta x$$

Propagated error on the basis of Taylor expansion

$$C = \max_{\delta x}\left(\left|\frac{\delta y/y}{\delta x/x}\right|\right)$$

Condition criterion, $C < 10$ error development small

# Badly (ill) conditioned problems: Example

Solve the following linear system in Python using double and single precision:

$$A = \begin{bmatrix} 1.2969 & 0.8648 \\ 0.2161 & 0.1441 \end{bmatrix}, \quad x = \begin{bmatrix} 0.8642 \\ 0.1440 \end{bmatrix}, \quad y = \begin{bmatrix} 2.0 \\ -2.0 \end{bmatrix}$$

## Double precision

```
import numpy as np

# Double precision
A = np.array([[1.2969, 0.8648], [0.2161,
     0.1441]], dtype=np.float64)
x = np.array([0.8642, 0.1440], dtype=np.float64)
y = np.linalg.solve(A, x)
print("y =")
print(y)
```

## Single precision

```
import numpy as np

# Single precision
A = np.array([[1.2969, 0.8648], [0.2161,
     0.1441]], dtype=np.float32)
x = np.array([0.8642, 0.1440], dtype=np.float32)
y = np.linalg.solve(A, x)
print("y =")
print(y)
```

# Badly (ill) conditioned problems: Example

- Python does not automatically warn about bad condition number. You need to compute and check it manually using NumPy:

```
if np.linalg.cond(A)> threshold: raise("Ill conditioned error")
```

- The `cond` number is the condition number computed using NumPy.
- A small error in $A$ (the matrix) results in a big error in the solution. This is called an ill-conditioned problem.

# Today's outline


- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

# (Un)stable methods

- The condition criterion does not tell you anything about the quality of a numerical solution method!
- It is very well possible that a certain solution method is more sensitive for one problem than another
- If the method propagates the error, we call it an *unstable method*. Let's look at an example.

# Error propagation: concept

Errors propagate through numerical algorithms.
For an iteration

$$x_{n+1} = g(x_n),$$

the error satisfies

$$e_{n+1} \approx g'(x^*)e_n + \delta_n.$$

If $|g'(x^*)| < 1$, errors decay. If $|g'(x^*)| > 1$, errors amplify.

# Error propagation: numerical differentiation

Forward difference:

$$f'(x) \approx \frac{f(x+h)-f(x)}{h}.$$

Truncation error is $O(h)$, roundoff error behaves like $O(\varepsilon/h)$.
Example: $f(x) = \sin x$ at $x = 1$.

```
import numpy as np

x = 1.0
true = np.cos(x)

for k in range(1,17):
    h = 10.0**(-k)
    approx = (np.sin(x+h)-np.sin(x))/h
    print(h, abs(approx-true))
```

# Error propagation: mitigation

- Use higher-order schemes (e.g. central differences).
- Choose $h$ where truncation and roundoff balance.
- Avoid subtraction when possible (symbolic or automatic differentiation).

Smaller step sizes are not always better.

# Example 1: Explanation

Recall example 1, where the errors blew up our computation of 0.1, whereas they did not for 2. Why did we see these results?

- The number 0.1 is not exactly represented in binary
  - A tiny error can accumulate up to catastrophic proportions!
- The number 2 does have an exact binary representation

# Example 2 (large sine series)

The `np.sin(1e40*np.pi)` result gives poor results because 1e40 has an error margin on the order of floating-point machine epsilon, which is roughly $1 \times 10^{-16}$ in Python (double-precision floating-point format). In Python, as in many computing environments, the number of $2 \cdot \pi$ cycles is still much larger than $10^{40} \cdot 10^{-16}$. Also, $\pi$ is not stored with an infinite number of digits, which further contributes to the imprecision.

# Example 3

Start your calculation program of choice (Excel, Python, ...)

Calculate the result of $y$:

$$y = e^{\pi} - \pi = 19.999099979 \neq 20$$

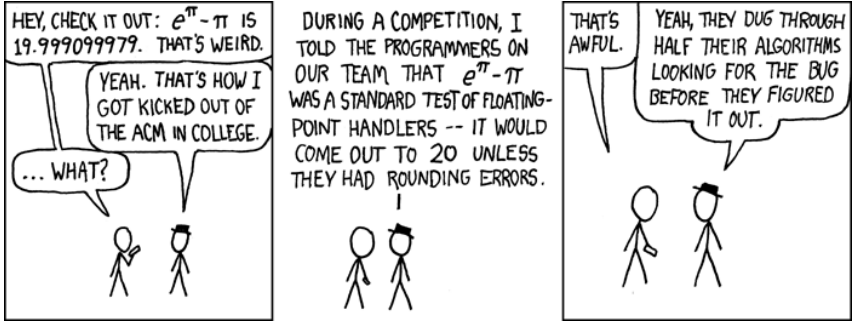





Image: xkcd

# Today's outline


- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

# Symbolic math packages

**Definition**

The use of computers to manipulate mathematical equations and expressions in symbolic form, as opposed to manipulating the numerical quantities represented by those symbols.

- Symbolic integration or differentiation, substitution of one expression into another
- Simplification of an expression, change of subject etc.
- Packages and toolboxes:

# Symbolic math packages

| | |
|---|---|
| Mathematica | Well known software package, license available via TU/e |
| Maple | Well known, license available via TU/e |
| Wolfram\|Alpha | Web-based interface by Mathematica developer. Less powerful in mathematical respect, but more accessible and has a broad application range (unit conversion, semantic commands). |
| Sage | Open-source alternative to Maple, Mathematica, Magma, and MATLAB. |
| Matlab | Symbolic math toolbox |
| Python | SymPy |

# Symbolic math: simplify

$$f(x) = (x-1)(x+1)(x^2+1)+1$$

```python
from sympy import symbols, simplify

x = symbols('x') # Alt: from sympy.abc import x,y,z
f = (x - 1)*(x + 1)*(x**2 + 1) + 1
f_simplified = simplify(f)
print(f_simplified)
```

```
f_simplified = x**4
```

# Symbolic math: integration and differentiation

$$f(x) = \frac{1}{x^3 + 1}$$

```
from sympy import symbols, simplify, integrate, diff

x = symbols('x')
f = 1/(x**3+1)
my_f_int = integrate(f, x)
my_f_diff = diff(my_f_int, x)
my_f_diff_simplified = simplify(my_f_diff)
print(my_f_diff_simplified)
```

```
my_f_diff_simplified = 1/(x**3 + 1)
```

# Symbolic math: exercises

## Exercise 1

Simplify the following expression:

$$f(x) = \frac{2\tan x}{(1+\tan^2 x)} = \sin 2x$$

```
1 from sympy import symbols, trigsimp, tan
2
3 x = symbols('x')
4 expr = 2*tan(x)/(1 + tan(x)**2)
5 simplified_expr = trigsimp(expr)
```

# Symbolic math: exercises

## Exercise 2

Calculate the *value* of $p$:

$$p = \int_0^{10} \frac{e^x - e^{-x}}{\sinh x} dx$$

```
from sympy import exp, sinh, integrate, symbols

x = symbols("x")
f = ((exp(x)- exp(-x))/sinh(x)).simplify()
p = integrate(f, (x, 0, 10))
print(p)
```

```
p = 20
```

# Symbolic math: root finding

A root finding method searches for the values where a function reaches zero. We will cover the numerical methods later, here we show how to use root finding with symbolic math in Python.

## Symbolic math function

$$f(x) = \frac{3}{x^2 + 3x} - 2$$

```
1 from sympy import solve, symbols
2
3 x = symbols("x")
4 f = 3 / (x**2 + 3*x) - 2
5 solutions = solve(f, x)
6 print(solutions)
```

```
solutions = [-3/2 + sqrt(15)/2, -sqrt(15)/2 - 3/2]
```

# Today's outline


- Introduction
- Roundoff errors
- Break errors
- Loss of digits
- (Un)stable methods
  - Error propagation
- Symbolic math
- Summary

# Summary

- Numerical errors may arise due to truncation, roundoff and break errors, which may seriously affect the accuracy of your solution
- Errors may propagate and accumulate, leading to smaller accuracy
- Ill-conditioned problems and unstable methods have to be identified so that proper measures can be taken
- Symbolic math computations may be performed to solve certain equations algebraically, bypassing numerical errors, but this is not always possible.

# Linear equations 1

## Linear algebra basics

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Matrix inversion
- Solving a linear system
- Towards larger systems
- Summary

# Overview

## Goals

- Different ways of looking at a system of linear equations
- Determination of the inverse, determinant and the rank of a matrix
- The existence of a solution to a set of linear equations

# Different views of linear systems

- Separate equations:

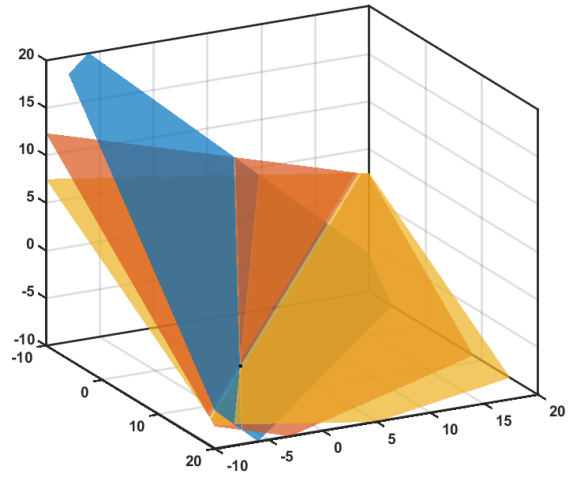
$$x + y + z = 4$$
$$2x + y + 3z = 7$$
$$3x + y + 6z = 5$$

- Matrix mapping $Mx = b$:

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 3 \\ 3 & 1 & 6 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \\ 5 \end{bmatrix}$$

- Linear combination:

$$x \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + y \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + z \begin{bmatrix} 1 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \\ 5 \end{bmatrix}$$



# Today's outline


- Introduction
- Matrix inversion
- Solving a linear system
- Towards larger systems
- Summary

# Inverse of a matrix

- The inverse $M^{-1}$ is defined such that:

$$MM^{-1} = I \quad \text{and} \quad M^{-1}M = I$$

- Use the inverse to solve a set of linear equations:

$$\begin{aligned} M\boldsymbol{x} &= \boldsymbol{b} \\ M^{-1}M\boldsymbol{x} &= M^{-1}\boldsymbol{b} \\ I\boldsymbol{x} &= M^{-1}\boldsymbol{b} \\ \boldsymbol{x} &= M^{-1}\boldsymbol{b} \end{aligned}$$

# How to calculate the inverse?

- The inverse of an $N \times N$ matrix can be calculated using the co-factors of each element of the matrix:

$$M^{-1} = \frac{1}{\det|M|} \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix}^T$$

- $\det|M|$ is the *determinant* of matrix $M$.
- $C_{ij}$ is the *co-factor* of the $ij^{\text{th}}$ element in $M$.

# Computing the co-factors

Consider the following example matrix: $M = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 3 \\ 3 & 1 & 6 \end{bmatrix}$

A co-factor (e.g. $C_{11}$) is the determinant of the elements left over when you cover up the row and column of the element in question, multiplied by $\pm 1$, depending on the position.

$$\begin{bmatrix} 1 & \times & \times \\ \times & 1 & 3 \\ \times & 1 & 6 \end{bmatrix} \qquad \begin{bmatrix} + & - & + \\ - & + & - \\ + & - & + \end{bmatrix}$$

$$C_{11} = +1 \cdot \det \begin{vmatrix} 1 & 3 \\ 1 & 6 \end{vmatrix}$$

$$= 6 \times 1 - 3 \times 1 = 3$$

# Computing the co-factors

Back to our example:

$$M^{-1} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 3 \\ 3 & 1 & 6 \end{bmatrix}^{-1} = \frac{1}{\det|M|} \begin{bmatrix} 3 & -3 & -1 \\ -5 & 3 & 2 \\ 2 & -1 & -1 \end{bmatrix}^{T}$$

- The determinant is very important
- If $\det|M| = 0$, the inverse does not exist (singular matrix)

# Calculating the determinant

Compute the determinant by multiplication of each element on a row (or column) by its cofactor and adding the results:

$$\det\begin{vmatrix}\mathbf{1} & \mathbf{1} & \mathbf{1}\\ 2 & 1 & 3\\ 3 & 1 & 6\end{vmatrix} = +\det\begin{vmatrix}1 & 3\\ 1 & 6\end{vmatrix} - \det\begin{vmatrix}2 & 3\\ 3 & 6\end{vmatrix} + \det\begin{vmatrix}2 & 1\\ 3 & 1\end{vmatrix} = -1$$

$$\det\begin{vmatrix}1 & 1 & \mathbf{1}\\ 2 & 1 & \mathbf{3}\\ 3 & 1 & \mathbf{6}\end{vmatrix} = +\det\begin{vmatrix}2 & 1\\ 3 & 1\end{vmatrix} - 3\det\begin{vmatrix}1 & 1\\ 3 & 1\end{vmatrix} + 6\det\begin{vmatrix}1 & 1\\ 2 & 1\end{vmatrix} = -1$$

# Today's outline


- Introduction
- Matrix inversion
- Solving a linear system
- Towards larger systems
- Summary

# Solving a linear system

- Our example:

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 1 & 3 \\ 3 & 1 & 6 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \\ 5 \end{bmatrix}$$

- The solution is:

$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = M^{-1} b = \frac{1}{-1} \begin{bmatrix} 3 & -5 & 2 \\ -3 & 3 & -1 \\ -1 & 2 & -1 \end{bmatrix} \begin{bmatrix} 4 \\ 7 \\ 5 \end{bmatrix} = \frac{1}{-1} \begin{bmatrix} -13 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} 13 \\ -4 \\ -5 \end{bmatrix}$$

- The inverse exists, because $\det |M| = -1$.

# Solving a linear system in Python using the inverse

- Create the matrix:

```
>>> A = np.array([[1, 1, 1], [2, 1, 3], [3, 1, 6]])
```

- Create solution vector:

```
>>> b = np.array([4, 7, 5])
```

- Get the matrix inverse:

```
>>> Ainv = np.linalg.inv(A)
```

- Compute the solution:

```
>>> x = np.dot(Ainv, b)
```

- Python's internal direct solver:

```
>>> x = np.linalg.solve(A, b)
```

- These are black boxes! We are going over some methods later!

# Exercise: performance of inverse computation

Create a script that generates matrices with random elements of various sizes $N \times N$ (e.g. values of $N \in \{10, 20, 50, 100, 200, \ldots, 5000, 10000\}$). Compute the inverse of each matrix, and use `tic` and `toc` to see the computing time for each inversion. Plot the time as a function of the matrix size $N$.

---

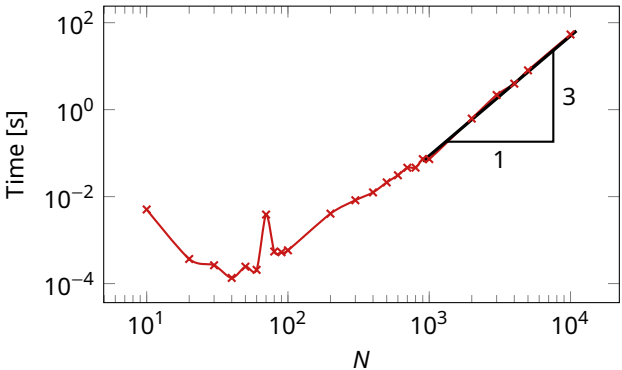
Hints:

- Create an array that contains the sizes of the systems $n$
- Loop over the array elements to:
  - Create a random matrix of size $n \times n$
  - Perform the matrix inversion
  - Record the time required
- Plot the time required for inversion vs size of the system on a double-log scale

---

# Exercise: sample results

Each computer produces slightly different results because of background tasks, different matrices, etc. This is especially noticable for small systems.




The time increases by 3 orders of magnitude, for every magnitude in $N$. The *computational complexity* of matrix inversion scales with $\mathcal{O}(N^3)$!

# Today's outline

- Introduction
- Matrix inversion
- Solving a linear system
- Towards larger systems
- Summary

# Towards larger systems

Computation of determinants and inverses of large matrices in this way is too difficult (slow), so we need other methods to solve large linear systems!

# Towards larger systems

- Determinant of upper triangular matrix:

$$\det\left|M_{\text{tri}}\right| = \prod_{i=1}^{n} a_{ii} \qquad M = \begin{bmatrix} 5 & 3 & 2 \\ 0 & 9 & 1 \\ 0 & 0 & 1 \end{bmatrix} \Rightarrow \det\left|M\right| = 5 \times 9 \times 1 = 45$$

- Matrix multiplication:

$$\det\left|AM\right| = \det\left|A\right| \times \det\left|M\right|$$

- When $A$ is an identity matrix ($\det\left|A\right| = 1$):

$$\det\left|AM\right| = \det\left|A\right| \times \det\left|M\right| = 1 \times \det\left|M\right|$$

- With rules like this, we can use row-operations so that we can compute the determinant more cheaply.

# Solutions of linear systems

Rank of a matrix: the number of linearly independent columns (columns that can not be expressed as a linear combination of the other columns) of a matrix.

$$M = \begin{bmatrix} 5 & 3 & 2 \\ 0 & 9 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

- 3 independent columns
- In Python:

```
1 >>> numpy.linalg.matrix_rank(M)
```

$$M = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- col 2 = 2× col 1
- col 4 = col 3 − col 1
- 2 independent columns: rank = 2

# Solutions of linear systems

The solution of a system of linear equations may or may not exist, and it may or may not be unique. Existence of solutions can be determined by comparing the rank of the Matrix $M$ with the rank of the augmented matrix $M_a$:

```
>>> numpy.linalg.matrix_rank(A)
>>> numpy.linalg.matrix_rank(np.column_stack((A,b))) # Concatenated matrices
```

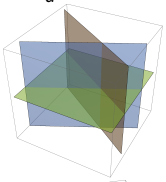
Our system: $Mx = b$

$$M = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix}, b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \Rightarrow M_a = \begin{bmatrix} M_{11} & M_{12} & M_{13} & b_1 \\ M_{21} & M_{22} & M_{23} & b_2 \\ M_{31} & M_{32} & M_{33} & b_3 \end{bmatrix}$$

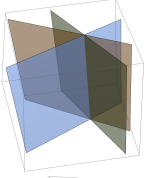
# Existence of solutions for linear systems

For a matrix $M$ of size $n \times n$, and augmented matrix $M_a$:

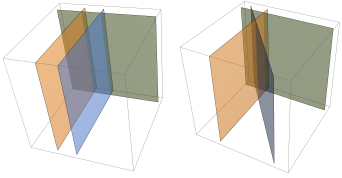
- $\text{Rank}(M) = n$:
  Unique solution



- $\text{Rank}(M) = \text{Rank}(M_a) < n$:
  Infinite number of solutions



- $\text{Rank}(M) < n$, $\text{Rank}(M) < \text{Rank}(M_a)$:
  No solutions



# Two examples

$$M = \begin{bmatrix} 1 & 1 & 2 \\ 0 & 3 & 1 \\ 0 & 0 & 2 \end{bmatrix} \quad b = \begin{bmatrix} 17 \\ 11 \\ 4 \end{bmatrix} \Rightarrow M_a = \begin{bmatrix} 1 & 1 & 2 & 17 \\ 0 & 3 & 1 & 11 \\ 0 & 0 & 2 & 4 \end{bmatrix}$$

$\text{rank}(M) = 3 = n \Rightarrow$ Unique solution

$$M = \begin{bmatrix} 1 & 1 & 2 \\ 0 & 3 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad b = \begin{bmatrix} 17 \\ 11 \\ 0 \end{bmatrix} \Rightarrow M_a = \begin{bmatrix} 1 & 1 & 2 & 17 \\ 0 & 3 & 1 & 11 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$\text{rank}(M) = \text{rank}(M_a) = 2 < n \Rightarrow$ Infinite number of solutions

# Today's outline


- Introduction
- Matrix inversion
- Solving a linear system
- Towards larger systems
- Summary

# Summary

- Linear equations can be written as matrices
- Using the inverse, the solution can be determined
  - Inverse via cofactors
  - Inverse and solution in Python
- Introduced the concept of computational complexity: matrix inversion scales with $N^3$
- A solution depends on the rank of a matrix

# Linear equations 2

## Direct methods

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Gauss elimination
- Partial Pivoting
- LU decomposition
- Summary

# Overview

## Goals

Today we are going to write a program, which can solve a set of linear equations

- The first method is called Gaussian elimination
- We will encounter some problems with Gaussian elimination
- Then LU decomposition will be introduced

# Today's outline


- Introduction
- Gauss elimination
- Partial Pivoting
- LU decomposition
- Summary

# Define the linear system

Consider the system:

$$Ax = b$$

In general:

$$\begin{bmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \\ A_{20} & A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}$$

Desired solution:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \end{bmatrix}$$

# Using row operations

- Use row operations to simplify the system. Eliminate element $A_{10}$ by subtracting $A_{10}/A_{00} = d_{10}$ times row 1 from row 2.
- In this case, Row 1 is the pivot row, and $A_{00}$ is the pivot element.

$$\left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ A_{10} & A_{11} & A_{12} & b_1 \\ A_{20} & A_{21} & A_{22} & b_2 \end{array}\right] \longrightarrow \left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ 0 & A'_{11} & A'_{12} & b'_1 \\ A_{20} & A_{21} & A_{22} & b_2 \end{array}\right]$$

# Using row operations

Eliminate element $A_{10}$ using $d_{10} = A_{10}/A_{00}$.

$$\left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ A_{10} & A_{11} & A_{12} & b_1 \\ A_{20} & A_{21} & A_{22} & b_2 \end{array}\right] \longrightarrow \left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ 0 & A'_{11} & A'_{12} & b'_1 \\ A_{20} & A_{21} & A_{22} & b_2 \end{array}\right]$$

- $d_{10} \longrightarrow A_{10}/A_{00}$
- $A_{10} \longrightarrow A_{10} - A_{00}d_{10}$
- $A_{11} \longrightarrow A_{11} - A_{01}d_{10}$
- $A_{12} \longrightarrow A_{12} - A_{02}d_{10}$
- $b_1 \longrightarrow b_1 - b_0 d_{10}$

```
d10 = A[1,0] / A[0,0]

A[1,0] = A[1,0] - A[0,0] * d10
A[1,1] = A[1,1] - A[0,1] * d10
A[1,2] = A[1,2] - A[0,2] * d10

b[1] = b[1] - b[0] * d10
```

# Using row operations

Eliminate element $A_{20}$ using $d_{20} = A_{20}/A_{00}$.

$$\left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ 0 & A'_{11} & A'_{12} & b'_1 \\ \hline A_{20} & A_{21} & A_{22} & b_2 \end{array}\right] \longrightarrow \left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ 0 & A'_{11} & A'_{12} & b'_1 \\ \hline 0 & A'_{21} & A'_{22} & b'_2 \end{array}\right]$$

- $d_{20} \longrightarrow A_{20}/A_{00}$
- $A_{20} \longrightarrow A_{20} - A_{00}d_{20}$
- $A_{21} \longrightarrow A_{21} - A_{01}d_{20}$
- $A_{22} \longrightarrow A_{22} - A_{02}d_{20}$
- $b_2 \longrightarrow b_2 - b_0 d_{20}$

```
1 d20 = A[2, 0] / A[0, 0]
2
3 A[2, 0] = A[2, 0] - A[0, 0] * d20
4 A[2, 1] = A[2, 1] - A[0, 1] * d20
5 A[2, 2] = A[2, 2] - A[0, 2] * d20
6 b[2] = b[2] - b[0] * d20
```

# Using row operations

Eliminate element $A'_{21}$ using $d_{21} = A'_{21}/A'_{11}$. Note that now the second row has become the pivot row.

$$\left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ 0 & A'_{11} & A'_{12} & b'_1 \\ 0 & A'_{21} & A'_{22} & b'_2 \end{array}\right] \longrightarrow \left[\begin{array}{ccc|c} A_{00} & A_{01} & A_{02} & b_0 \\ 0 & A'_{11} & A'_{12} & b'_1 \\ 0 & 0 & A''_{22} & b''_2 \end{array}\right]$$

- $d_{21} \rightarrow A_{21}/A'_{11}$
- $A_{21} \rightarrow A_{21} - A'_{11} d_{21}$
- $A_{22} \rightarrow A_{22} - A'_{12} d_{21}$
- $b_2 \rightarrow b_2 - b'_2 d_{21}$

```
d21 = A[2, 1] / A[1, 1]
A[2, 1] = A[2, 1] - A[1, 1] * d21
A[2, 2] = A[2, 2] - A[1, 2] * d21
b[2] = b[2] - b[1] * d21
```

The matrix is now a triangular matrix — the solution can be obtained by back-substitution.

# Backsubstitution

The system now reads:

$$\begin{bmatrix} A_{00} & A_{01} & A_{02} \\ 0 & A'_{11} & A'_{12} \\ 0 & 0 & A''_{22} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_0 \\ b'_1 \\ b''_2 \end{bmatrix}$$

Start at the last row $N$, and work upward until row 1.

$$x_2 = b''_2/A''_{22}$$
$$x_1 = (b'_1 - A'_{12}x_2)/A'_{11}$$
$$x_0 = (b_0 - A_{01}x_1 - A_{02}x_2)/A_{00}$$

```
x = np.empty_like(b)
x[2] = b[2] / A[2,2]
x[1] = (b[1] - A[1,2] * x[2]) / A[1,1]
x[0] = (b[0] - A[0,1] * x[1] - A[0,2] * x[2]) / A[0,0]
```

In general:

$$x_N = \frac{b_N}{A_{NN}} \qquad x_i = \frac{b_i - \sum_{j=i+1}^{N} A_{ij}x_j}{A_{ii}}$$

# Writing the program

- Create a function that provides the framework: take matrix $A$ and vector $b$ as an input, and return the solution $x$ as output:

```
def gaussian_eliminate(A, b):
    pass # Your implementation here
```

- We will use *for-loops* instead of typing out each command line.
- Useful Python (with NumPy) shortcuts:
  - `A[0, :]` = $[A_{00}, A_{01}, A_{02}]$
  - `A[:, 1]` = $[A_{01}, A_{11}, A_{21}]$
  - `A[0, 1:]` = $[A_{01}, A_{02}]$
- A row operation could look like:

```
A[i, :] = A[i, :] - d * A[0, :]
```

# The program: elimination step

An initial draft could look like:

```
def gaussian_eliminate_draft(A,b):
   """Perform elimination to obtain an upper triangular matrix"""
   A = np.array(A,dtype=np.float64)
   b = np.array(b,dtype=np.float64)

   assert A.shape[0] == A.shape[1], "Coefficient matrix should be square"

   N = len(b)
   for col in range(N-1): # Select pivot
      for row in range(col+1,N): # Loop over rows below pivot
         d = A[row,col] / A[col,col] # Choose elimination factor
         for element in range(row,N): # Elements from diagonal to right
            A[row,element] = A[row,element] - d * A[col,element]
         b[row] = b[row] - d * b[col]

   return A,b
```

# The program: elimination step

Employing some of the row operations to create `gaussian_eliminate_v1`:

```
for element in range(row,N):
   A[row,element] = A[row,element] - d * A[col,element]
```

```
A[row,:] = A[row,:] - d * A[col,:]
```

```
def gaussian_eliminate_v1(A,b):
   A = np.array(A,dtype=np.float64)
   b = np.array(b,dtype=np.float64)

   assert A.shape[0] == A.shape[1], "Coefficient matrix should be square"

   N = len(b)
   for col in range(N-1):
      for row in range(col+1,N):
         d = A[row,col] / A[col,col]
         A[row,:] = A[row,:] - d * A[col,:]
         b[row] = b[row] - d * b[col]

   return A,b
```

# Testing

Let's try to eliminate our linear system! If you create/downloaded our file `gaussjordan.py`, you can access the functions by importing them. The file should be stored where your own code/notebook is:

```
from gaussjordan import gaussian_eliminate_draft,gaussian_eliminate_v1
import numpy as np

A = np.array([[1, 1, 1], [2, 1, 3], [3, 1, 6]])
b = np.array([4, 7, 5])

Aprime,bprime = gaussian_eliminate_draft(A,b)
print(Aprime)
print(bprime)
```

# The program: Backsubstitution

Now we have elimination working, let's create a back substitution algorithm too. Recall:

$$x_N = \frac{b_N}{A_{NN}} \qquad x_i = \frac{b_i - \sum_{j=i+1}^{N} A_{ij} x_j}{A_{ii}}$$

```
def backsubstitution_draft(A, b):
    """Back substitutes an upper triangular matrix to
           find x in Ax=b"""
    x = np.copy(b)
    N = len(b)

    for row in range(N-1, -1, -1):
        for i in range(row+1, N):
            x[row] = x[row] - A[row, i] * x[i]
        x[row] = x[row] / A[row, row]

    return x
```

```
def backsubstitution_v1(A,b):
    """Back substitutes an upper triangular matrix to find x in Ax=b"""
    x = np.empty_like(b)
    N = len(b)

    for row in range(N)[::-1]:
        x[row] = (b[row] - np.sum(A[row,row+1:] * x[row+1:])) / A[row,row]

    return x
```

# A full Gauss Elimination solver

- The functions we just built are distributed via Canvas too
- Use `help(gaussian_eliminate_v1)` to find out how it works
- Solve the following system of equations:

$$\begin{bmatrix} 9 & 9 & 5 & 2 \\ 6 & 7 & 1 & 3 \\ 6 & 4 & 3 & 5 \\ 2 & 6 & 2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 7 \\ 4 \\ 10 \\ 1 \end{bmatrix}$$

- Compare your solution with `np.linalg.solve(A,b)`

# Today's outline


- Introduction
- Gauss elimination
- Partial Pivoting
- LU decomposition
- Summary

# Partial pivoting

- Now try to run the algorithm with the following system:

$$\begin{bmatrix} 0 & 2 & 1 \\ 3 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \\ 10 \end{bmatrix}$$

- It does not work! Division by zero, due to $A_{11} = 0$.
- Solution: Swap rows to move largest element to the diagonal.

# Partial pivoting: implementing row swaps

- Find maximum element row below pivot in current column

```
index = np.argmax(np.abs(A[col:, col])) + col
```

- Store current row

```
temp = A[column,:]
```

- Swap pivot row and desired row in `A`

```
A[column,:] = A[index,:]
A[index,:] = temp
```

- Do the same for `b` — store and swap

```
temp = b[column]
b[column] = b[index]
b[index] = temp
```

# Adding the partial pivoting rules

```python
def gaussian_eliminate_partial_pivot(A,b):
    A = np.array(A,dtype=np.float64)
    b = np.array(b,dtype=np.float64)

    assert A.shape[0] == A.shape[1], "Coefficient matrix should be square"

    N = len(b)
    for col in range(N-1):
        index = np.argmax(np.abs(A[col:, col])) + col
        temp = A[col,:]
        A[col,:] = A[index,:]
        A[index,:] = temp

        temp = b[col]
        b[col] = b[index]
        b[index] = temp
        for row in range(col+1,N):
            d = A[row,col] / A[col,col]
            A[row,:] = A[row,:] - d * A[col,:]
            b[row] = b[row] - d * b[col]

    return A,b
```

# Improve the program by using re-usable functions

```python
def swap_rows(mat,i1,i2):
    """Swap two rows in a matrix/vector"""
    temp = mat[i1,...].copy()
    mat[i1,...] = mat[i2,...]
    mat[i2,...] = temp
```

```python
def gaussian_eliminate_v2(A,b):
    A = np.array(A,dtype=np.float64)
    b = np.array(b,dtype=np.float64)

    assert A.shape[0] == A.shape[1], "Coefficient matrix should be square"

    N = len(b)
    for col in range(N-1):
        index = np.argmax(np.abs(A[col:, col])) + col
        swap_rows(A,col,index)
        swap_rows(b,col,index)
        for row in range(col+1,N):
            d = A[row,col] / A[col,col]
            A[row,:] = A[row,:] - d * A[col,:]
            b[row] = b[row] - d * b[col]

    return A,b
```

# Alternatives to this program

- Python can compute the solution to Ax=b with `scipy.linalg.solve` or `numpy.linalg.solve` solvers (more efficient)
- Too many loops. Loops make Python slow.
- There are fundamental problems with Gaussian elimination
  - You can add a counter to the algorithm to see how many subtraction and multiplication operations it performs for a given size of matrix A.
  - The number of operations to perform Gaussian elimination is $\mathcal{O}(2N^3)$ (where $N$ is the number of equations)
  - Exercise: verify this for our script
  - LU decomposition takes $\mathcal{O}(2N^3/3)$ flops, 3 times less!
  - Forward and backward substitution each take $\mathcal{O}(N^2)$ flops (both cases)

# Today's outline


- Introduction
- Gauss elimination
- Partial Pivoting
- LU decomposition
- Summary

# LU Decomposition

Suppose we want to solve the previous set of equations, but with several right hand sides:

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} \begin{bmatrix} \vdots & \vdots & \vdots \\ x_1 & x_2 & x_3 \\ \vdots & \vdots & \vdots \end{bmatrix} = \begin{bmatrix} \vdots & \vdots & \vdots \\ b_1 & b_2 & b_3 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Factor the matrix A into two matrices, L and U, such that $A = LU$:

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \times & 1 & 0 \\ \times & \times & 1 \end{bmatrix} \begin{bmatrix} \times & \times & \times \\ 0 & \times & \times \\ 0 & 0 & \times \end{bmatrix}$$

Now we can solve for each right hand side, using only a forward followed by a backward substitution!

# Substitutions

- Define a lower and upper matrix $L$ and $U$ so that $A = LU$
- Therefore $LUx = b$
- Define a new vector $y = Ux$ so that $Ly = b$
- Solve for $y$, use $L$ and forward substitution
- Then we have $y$, solve for $x$, use $Ux = y$
- Solve for $x$, use $U$ and backward substitution
- But how to get L and U?

# Decomposing the matrix (1)

When we eliminate the element $A_{21}$ we can keep multiplying by a matrix that undoes this row operations, so that the product remains equal to $A$.

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ d_{21} & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ 0 & A_{22} - d_{21}A_{12} & A_{23} - d_{21}A_{13} \\ A_{31} & A_{32} & A_{33} \end{bmatrix}$$

# Decomposing the matrix (2)

When we eliminate the element $A_{31}$ we can keep multiplying by a matrix that undoes this row operations, so that the product remains equal to $A$.

$$A = \begin{bmatrix} 1 & 0 & 0 \\ d_{21} & 1 & 0 \\ d_{31} & 0 & 1 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ 0 & A'_{22} = A_{22} - d_{21}A_{12} & A'_{23} = A_{23} - d_{21}A_{13} \\ 0 & A'_{32} = A_{32} - d_{31}A_{12} & A'_{33} = A_{33} - d_{31}A_{21} \end{bmatrix}$$

# Decomposing the matrix (3)

When we eliminate the element $A_{32}$ we can keep multiplying by a matrix that undoes this row operations, so that the product remains equal to $A$.

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ d_{21} & 1 & 0 \\ d_{31} & d_{32} & 1 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ 0 & A'_{22} & A'_{23} \\ 0 & 0 & A''_{33} = A'_{33} - d_{32}A'_{23} \end{bmatrix}$$

We now have a lower matrix $L$ and an upper matrix $U$. This finishes the LU decomposition!

# Pivoting during decomposition

Suppose we have arrived at the situation below, where $A'_{32} > A'_{22}$:

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ d_{21} & 1 & 0 \\ d_{31} & 0 & 1 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ 0 & A'_{22} & A'_{23} \\ 0 & A'_{32} & A'_{33} \end{bmatrix}$$

Exchange rows 2 and 3 to get the largest value on the main diagonal. Use a permutation matrix to store the swapped rows:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ d_{31} & 0 & 1 \\ d_{21} & 1 & 0 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ 0 & A'_{32} & A'_{33} \\ 0 & A'_{22} & A'_{23} \end{bmatrix}$$

Multiplying with a permutation matrix will swap the rows of a matrix. The permutation matrix is just an identity matrix, whose rows have been interchanged.

# Recipe for LU decomposition

When decomposing matrix $A$ into $A = LU$, it may be beneficial to swap rows to get the largest values on the diagonal of $U$ (pivoting). A permutation matrix $P$ is used to store row swapping such that:

$$PA = LU$$

- Write down a permutation matrix and the linear system
- Promote the largest value in the column diagonal
- Eliminate all elements below diagonal
- Move on to the next column and move largest elements to diagonal
- Eliminate elements below diagonal
- Repeat 5 and 6
- Write down L,U and P

# LU decomposition example (1)

Write down a permutation matrix, which starts as the identity matrix, and the linear system:

$$PA = LU$$

$$\begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix}\begin{bmatrix}0&1&1\\2&1&1\\1&2&0\end{bmatrix} = \begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix}\begin{bmatrix}0&1&1\\2&1&1\\1&2&0\end{bmatrix}$$

Promote the largest value into the diagonal of column 1 — swap row 1 and 2:

$$\begin{bmatrix}0&1&0\\1&0&0\\0&0&1\end{bmatrix}\begin{bmatrix}0&1&1\\2&1&1\\1&2&0\end{bmatrix} = \begin{bmatrix}1&0&0\\0&1&0\\0&0&1\end{bmatrix}\begin{bmatrix}2&1&0\\0&1&1\\1&2&0\end{bmatrix}$$

# LU decomposition example (2)

Eliminate all elements below the diagonal — row 2 already contains a zero in column 1, row 3 = row 3 - 0.5 row 1. Record the multiplier 0.5 in $L$:

$$\begin{bmatrix}0&1&0\\1&0&0\\0&0&1\end{bmatrix}\begin{bmatrix}0&1&1\\2&1&1\\1&2&0\end{bmatrix}=\begin{bmatrix}1&0&0\\0&1&0\\0.5&0&1\end{bmatrix}\begin{bmatrix}2&1&1\\0&1&1\\0&1.5&-0.5\end{bmatrix}$$

Elimination of column 1 is done. Now step to the next column, and move the largest value below/on the diagonal to the diagonal swap rows 2 and 3). Adjust $P$ and the lower triangle of $L$ accordingly:

$$\begin{bmatrix}0&1&0\\0&0&1\\1&0&0\end{bmatrix}\begin{bmatrix}0&1&1\\2&1&0\\1&2&0\end{bmatrix}=\begin{bmatrix}1&0&0\\0.5&1&0\\0&0&1\end{bmatrix}\begin{bmatrix}2&1&1\\0&1.5&-0.5\\0&1&1\end{bmatrix}$$

# LU decomposition example (3)

Eliminate all elements below the diagonal — row 3 = row 3 - $\frac{2}{3}$ row 2. Record the multiplier $\frac{2}{3}$ in $L$:

$$\begin{bmatrix}0&1&0\\0&0&1\\1&0&0\end{bmatrix}\begin{bmatrix}0&1&1\\2&1&0\\1&2&0\end{bmatrix}=\begin{bmatrix}1&0&0\\0.5&1&0\\0&\frac{2}{3}&1\end{bmatrix}\begin{bmatrix}2&1&1\\0&1.5&-0.5\\0&0&\frac{4}{3}\end{bmatrix}$$

We have obtained the matrices from $PA = LU$:

$$P=\begin{bmatrix}0&1&0\\0&0&1\\1&0&0\end{bmatrix}\quad L=\begin{bmatrix}1&0&0\\0.5&1&0\\0&\frac{2}{3}&1\end{bmatrix}\quad U=\begin{bmatrix}2&1&1\\0&1.5&-0.5\\0&0&\frac{4}{3}\end{bmatrix}$$

Proceed with solving for $x$.

# Substitutions

$$Ax = b \quad \Rightarrow \quad PAx = Pb \equiv d$$
$$PA = LU \quad \Rightarrow \quad LUx = d$$

- Define a new vector $y = Ux$
  - $Ly = b \quad \Rightarrow \quad Ly = d$
  - Solve for $y$, forward substitution:

$$y_0 = \frac{d_0}{L_{00}}$$
$$y_i = \frac{d_i - \sum_{j=0}^{i} L_{ij} y_j}{L_{ii}}$$

- Then solve $Ux = y$:
  - Solve for $x$, backward substitution:

$$x_N = \frac{y_N}{U_{NN}}$$
$$x_i = \frac{y_i - \sum_{j=i+1}^{N} U_{ij} x_j}{U_{ii}}$$

# How to use the solver in Python

```python
import numpy as np
from scipy.linalg import lu
from gaussjordan import backsubstitution_v1 as backwardSub
from gaussjordan import forwardsubstitution as forwardSub

# Example usage
A = np.random.rand(5, 5) # Get random matrix
P, L, U = lu(A) # Get L, U and P
b = np.random.rand(5) # Random b vector
d = P @ b # Permute b vector
y = forwardSub(L, d) # Can also do y=L\d
x = backwardSub(U, y) # Can also do x=U\y
rnorm = np.linalg.norm(A @ x - b) # Residual
```

- Use this as a basis to create a function that takes $A$ and $b$, and returns $x$.
- Use the function to check the performance for various matrix sizes and inspect the performance.

# Today's outline


- Introduction
- Gauss elimination
- Partial Pivoting
- LU decomposition
- Summary

# Summary

- This lecture covered direct methods using elimination techniques.
- Gaussian elimination can be slow ($\mathcal{O}(N^3)$)
- Back substitution is often faster ($\mathcal{O}(N^2)$)
- LU decomposition means that we don't have to do Gaussian elimination every time (saves time and effort), but the matrix has to stay the same.
- Python's libraries have built in routines for solving linear equations and LU decomposition.
- Advanced techniques such as (preconditioned) conjugate gradient or biconjugate gradient solvers are also available.
- Next part covers iterative approaches

# Linear equations 3

## Iterative methods

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Sparse matrices
- Laplace's equation
- Creating a sparse system
- Iterative methods
- Summary

# Sparse matrices

- In many engineering cases, we deal with sparse matrices (as opposed to dense matrices)
- A matrix is sparse when it mostly consists of zeros
- Linear systems where equations depend on a limited number of variables (e.g. spatial discretization)
- Storing zeros is not very efficient:

```
import numpy as np
from scipy.sparse import csr_matrix

A = np.eye(10000)
print(A.nbytes)

S = csr_matrix(A)
print(S.data.nbytes)
```

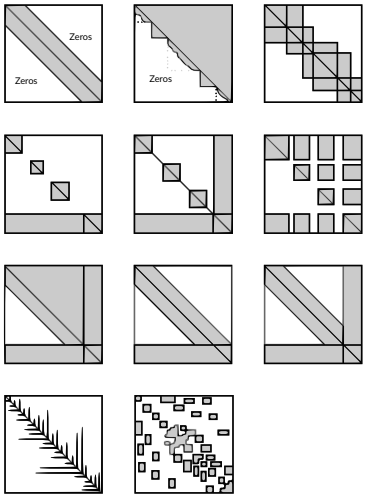
- Can you think of a way to achieve this?
- Sparse matrix formats: Yale, CRS, CCS

# Sparse matrix storage format

- Example: Yale storage format, storing 3 vectors:
  - `A = [5 8 3 6]`
  - `IA = [0 1 2 3 4]`
  - `JA = [0 1 2 1]`
  - `A` stores the non-zero values
  - `IA` stores the index in A of the first non-zero in row i
  - `JA` stores the column index

$$A = \begin{bmatrix} 5 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 6 & 0 & 0 \end{bmatrix}$$

# Sparse matrix layout examples



# Today's outline

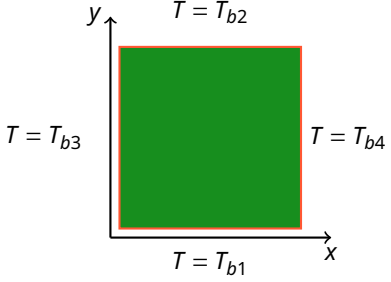
- Introduction
- Sparse matrices
- Laplace's equation
- Creating a sparse system
- Iterative methods
- Summary

# Laplace's equation

$$\frac{\partial T}{\partial t} = \alpha \nabla^2 T$$

$T$ = Temperature

$\alpha$ = Thermal diffusivity

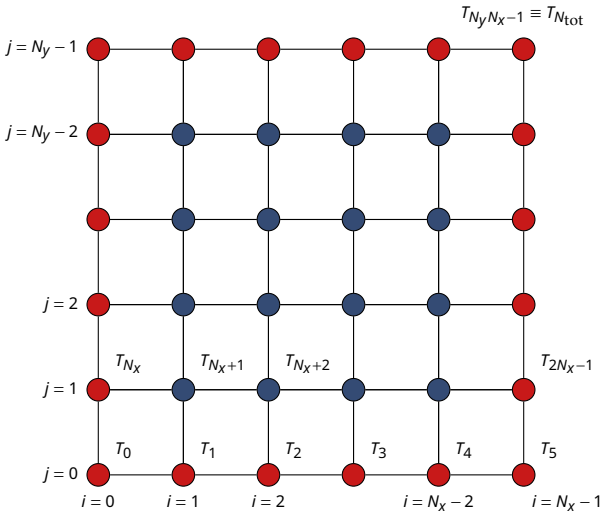



In steady state:

$$\nabla^2 T = 0$$

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

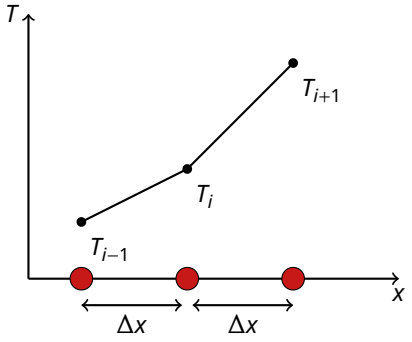
# Discretization of Laplace's equation (I)




- Define a grid of points in $x$ and $y$
- Index of the grid points using 2D coordinates $i$ and $j$
- Set up the equations using a 1D index system: $T_{i,j} \longrightarrow T_{i+jNx}$

# Discretization of Laplace's equation (II)

Estimate the second-order differentials: assume a piece-wise linear profile in the temperature:




$$\frac{\partial^2 T}{\partial x^2} \approx \frac{\left.\frac{\partial T}{\partial x}\right|_{i+\frac{1}{2}} - \left.\frac{\partial T}{\partial x}\right|_{i-\frac{1}{2}}}{\Delta x}$$

$$\approx \frac{\frac{(T_{i+1,j}-T_{i,j})}{\Delta x} - \frac{(T_{i,j}-T_{i-1,j})}{\Delta x}}{\Delta x}$$

$$= \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{(\Delta x)^2}$$

# Discretization of Laplace's equation (III)

The $y$-direction is derived analogously, so that the 2D Laplace's equation is discretized as:

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{(\Delta x)^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{(\Delta y)^2} = 0$$

Use a single index counter $k = i + N_x(j-1)$, so that the equation becomes:

$$\frac{T_{k+1} - 2T_k + T_{k-1}}{(\Delta x)^2} + \frac{T_{k+N_x} - 2T_k + T_{k-N_x}}{(\Delta y)^2} = 0$$

For an equal spaced grid $\Delta x = \Delta y = 1$:

$$T_{k-N_x} + T_{k-1} - 4T_k + T_{k+1} + T_{k+N_x} = 0$$

$$\Rightarrow AT = b$$

# Today's outline

- Introduction
- Sparse matrices
- Laplace's equation
- Creating a sparse system
- Iterative methods
- Summary

# Creating the linear system

$$T_{k-N_x} + T_{k-1} - 4T_k + T_{k+1} + T_{k+N_x} = 0$$

Create a *banded* matrix $A$: the main diagonal $k$ contains -4, whereas the bands at $k-1$, $k+1$, $k-N_x$ and $k+N_x$ contain a 1. Boundary cells just contain a 1 on the main diagonal so that the temperature is equal to $T_b$ (e.g. $T_1 = 1T_b$).

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ \cdots & 1 & \cdots & 1 & -4 & 1 & \cdots & 1 & \ddots & 0 \\ 0 & \cdots & 1 & \cdots & 1 & -4 & 1 & \cdots & 1 & \vdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} T_0 \\ T_1 \\ \vdots \\ T_k \\ T_k + 1 \\ \vdots \\ T_{N_y N_x - 2} \\ T_{N_y N_x - 1} \end{bmatrix} = \begin{bmatrix} T_b \\ T_b \\ \vdots \\ 0 \\ 0 \\ \vdots \\ T_b \\ T_b \end{bmatrix}$$

# Creating the linear system

$$T_{k-N_x} + T_{k-1} - 4T_k + T_{k+1} + T_{k+N_x} = 0$$

Create a *banded* matrix $A$ in Python, by setting the coefficients for the internal cells:

```
import numpy as np
from scipy.sparse import diags

Nx,Ny = 50,50 # Number of grid points along x,y direction
Nc = Nx*Ny # Total number of points

e = np.ones(Nc)
A = diags([e, e, -4*e, e, e], [-Nx, -1, 0, 1, Nx], shape=(Nc,Nc))
b = np.zeros(Nc)
```

The function `diags` uses the following arguments:

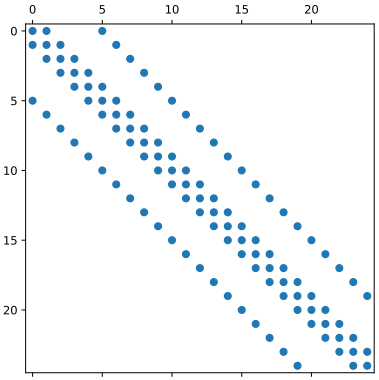
- The coefficients that have to be put on the diagonals arranged as columns in a matrix
- The position of the bands with respect to the main diagonal
- Size of the resulting matrix (in our case square $N_xN_y \times N_xN_y$)

# Matrix sparsity

- Let's check the matrix layout by adding:

```
print(A)
plt.spy(A, marker='o',markersize=6)
```

- The *sparse* structure stores/prints only the nonzero elements
- `spy` shows the location of the nonzero values in the matrix
- Apart from the main diagonal, there are offset bands!



# About boundary conditions

- For the nodes on the boundary, we have a simple equation:

$$T_{k,\text{boundary}} = \text{Some fixed value}$$

- However, we have set all nodes to be a function of their neighbors
- Solution: Determine the boundary node indices $k$ and set the coefficients accordingly

```
bnd_bottom = np.arange(Nx)
bnd_left = np.arange(Ny) * Nx
bnd_right = bnd_left + Nx - 1
bnd_top = bnd_bottom + Nx*(Ny-1)
```

- Reset each row $k$ in $A$ to zeros, then set element $A_{kk} = 1$
- Set values in rhs: $b_k = T_{\text{boundary}}$
- Boundary conditions are often more elaborate to implement!

# Implementation of the boundary conditions

A (shortened) version of the `set_boundary_conditions(A,b,Tb,Nx,Ny)` function:

```
def set_boundary_conditions(A, b, Tb, Nx, Ny):

    A = lil_matrix(A) # Required for efficient modification of the sparsity

    # Select nodes that lie at the boundaries
    bnd_bottom = np.arange(Nx)
    bnd_left = np.arange(Ny) * Nx
    bnd_right = bnd_left + Nx - 1
    bnd_top = bnd_bottom + Nx*(Ny-1)

    bnd_all = np.unique(np.concatenate((bnd_bottom,bnd_left,bnd_right,bnd_top)))

    # Reset the coefficient row to zero, add a 1 only on the main diagonal
    A[bnd_all,:] = 0
    A[bnd_all,bnd_all] = 1

    b[bnd_bottom] = Tb['bottom']
    b[bnd_left] = Tb['left']
    b[bnd_right] = Tb['right']
    b[bnd_top] = Tb['top']

    return A.tocsr(), b
```

# How applying boundary conditions affects the linear system

Using the functions provided in `laplace_demo.py`:

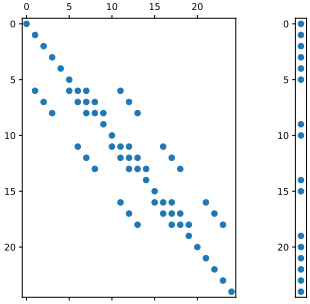
```
Nx = Ny = 5 # number of internal grid cells over x/y-direction

T_boundary = {'bottom': 300, 'left': 1000, 'right': 1000, 'top': 500}

A,b = create_laplace_coefficient_matrix(Nx,Ny)
A,b = set_boundary_conditions(A, b, T_boundary, Nx, Ny)
```

Check the new structure of the matrix and the right hand side:

```
plt.subplot(121); plt.spy(A2);
plt.subplot(122); plt.spy(b[:,None]);
```



# A full program, including solver

The program and auxiliary functions are on Canvas (`laplace_demo.py`)

```
import numpy as np
from scipy.sparse.linalg import spsolve
from matplotlib import cm
import matplotlib.pyplot as plt

Nx = Ny = 20

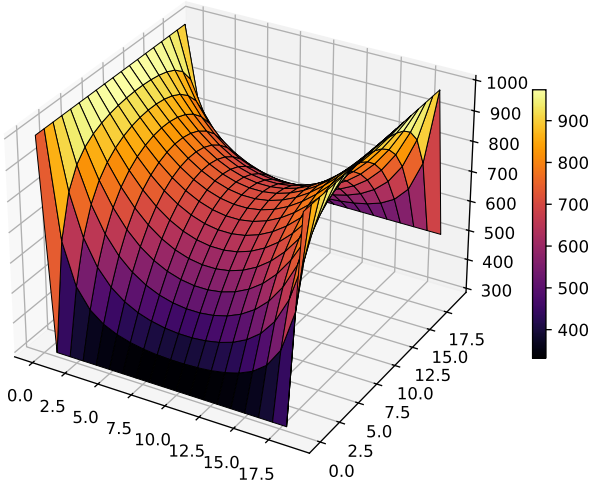
T_boundary = {'bottom': 300, 'left': 1000, 'right': 1000, 'top': 500}

A,b = create_laplace_coefficient_matrix(Nx,Ny)
A,b = set_boundary_conditions(A, b, T_boundary, Nx, Ny)

T = spsolve(A,b).reshape((Nx,Ny))

fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
x,y = np.meshgrid(np.arange(Nx),np.arange(Ny))
surf = ax.plot_surface(x,y,T,cmap=cm.inferno)
fig.colorbar(surf, shrink=0.5)
plt.show()
```

# Sample results



# Exercise: Verify the numerical solution using Fourier-series

A Fourier-series expansion for the steady-state heat conduction in a flat plate is given for a domain: $x, y \in [0, 1]$, with fixed-temperature boundaries $T\big|_{x=0} = T\big|_{x=1} = T\big|_{y=0} = 0$ and $T\big|_{y=1} = 1$:

$$T = \frac{4}{\pi} \sum_{n=1}^{\infty} \frac{\sin(m\pi x)\sinh(m\pi y)}{m \sinh(m\pi)} \quad \text{with} \quad m = 2n - 1$$

Compute and plot the exact temperature profile in the 2D plate, and compare it with the numerical solution:

---

Hints:

- Use meshgrid to create a mesh in $x$ and $y$
- Compute the temperature using the Fourier series, use vectorised computations over $x$ and $y$ so that only 1 loop (over n) is required.
- Solve the numerics for the same problem (note the boundary conditions)
- Compare the numerical and exact solutions (e.g. a plot).

---

# LU decomposition of a sparse matrix

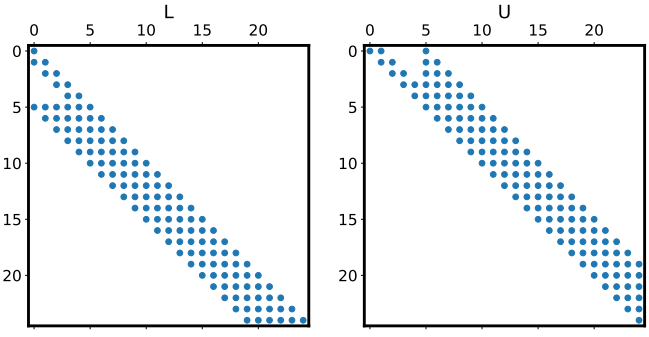
```python
import numpy as np
from scipy.linalg import lu
import matplotlib.pyplot as plt
from laplace_demo import
      create_laplace_coefficient_matrix

A,b = create_laplace_coefficient_matrix(5,5)

# Perform LU decomposition
# Note: lu does not work on sparse arrays,
# so we map to a full array
P,L,U = lu(A.toarray())

# Plot the sparsity patterns of L and U
plt.subplot(121)
plt.spy(L)
plt.title('L')
plt.subplot(122)
plt.spy(U)
plt.title('U')
plt.tight_layout()
```

- With LU decomposition we produce matrices that are less sparse than the original matrix.
- Sparse storage often required, and also numerical techniques that fully utilizes this!



# LU decomposition

- LU decomposition and Gaussian elimination on a matrix like $A$ requires more memory (with 3D problems, the offset in the diagonal would even be bigger!)
- In general extra memory allocation will not be a problem for Python
- Python is clever, in that sense that it attempts to reorder equations, to move elements closer to the diagonal)

Alternatives for elimination methods

- Use iterative methods when systems are large and sparse.
- Often such systems are encountered when we want to solve PDE's of higher dimensions

# Today's outline


- Introduction
- Sparse matrices
- Laplace's equation
- Creating a sparse system
- Iterative methods
- Summary

# Examples of iterative methods

- Jacobi method
- Gauss-Seidel method
- Succesive over relaxation

- `bicg` — Bi-conjugate gradient method
- `pcg` — preconditioned conjugate gradient method
- `gmres` — generalized minimum residuals method
- `bicgstab` — Bi-conjugate gradient method

# The Jacobi method

- In our example we derived the following equation:

$$T_{k-N_x} + T_{k-1} - 4T_k + T_{k+1} + T_{k+N_x} = 0$$

- Rearranging gives:

$$T_k = \frac{T_{k-N_x} + T_{k-1} + T_{k+1} + T_{k+N_x}}{4}$$

- In the Jacobi scheme the iteration proceeds as follows:
  1. Start with an initial guess for the values of $T$ at each node
  2. Compute updated values and store a new vector:

$$T_k^{\text{new}} = \frac{T_{k-N_x}^{\text{old}} + T_{k-1}^{\text{old}} + T_{k+1}^{\text{old}} + T_{k+N_x}^{\text{old}}}{4}$$

  3. Do this for all nodes
  4. Repeat the procedure until converged

# Jacobi method for Laplace's equation

See `laplace_jacobi.py` for animation included (from Canvas)

```
import numpy as np
import matplotlib.pyplot as plt

# Set grid resolution
nx = 40
ny = 40

# Set old solution array
T = np.zeros((nx,ny))

# Set boundary conditions
T[0,:] = 40 # Left
T[nx-1,:] = 60 # Right
T[:,0] = 20 # Bottom
T[:,ny-1] = 30 # Top

# Set new solution array (inc bnd
      conditions)
Tnew = T.copy()
```

```
# Create grid for plotting
x,y = np.meshgrid(np.arange(1,nx+1), np.arange(1,ny+1))

# Perform iterations
for iter in range(1,1001):
   for i in range(1,nx-1):
      for j in range(1,ny-1):
         # Calculate new solution
         Tnew[i,j] = \
            (T[i-1,j]+T[i+1,j]+T[i,j-1]+T[i,j+1])/4.0
   T = Tnew.copy()
```

→ Try to modify this script so that 1 cell/block of cells in the center is kept at 100 degrees

# About the straightforward implementation

- The method as implemented works fine for a simple Laplace equation
- For generic systems of linear equations, the implementation cannot be used.

We will now introduce the Jacobi method so it can be used for generic systems of linear equations.

# The Jacobi method with matrices

We can split our (banded) matrix $A$ into a diagonal matrix $D$ and a remainder $R$:

$$A = D + R$$



# Jacobi method: solving a system

- We can solve $AT = b$, now written generally as $Ax = b$, by:

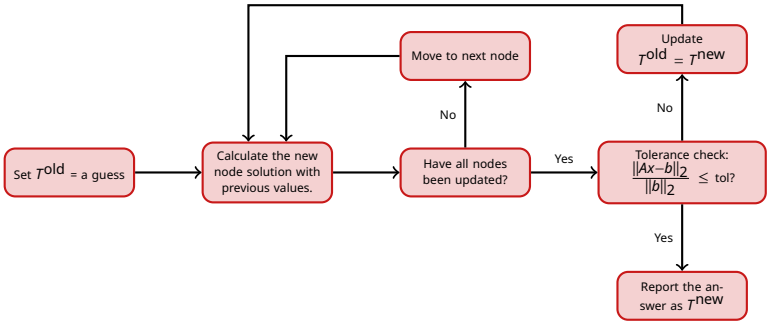
$$\begin{aligned} Ax &= b \\ (D+R)x &= b \\ Dx &= b - Rx \\ Dx^{\text{new}} &= b - Rx^{\text{old}} \\ x^{\text{new}} &= D^{-1}(b - Rx^{\text{old}}) \end{aligned}$$

- Using the $n$ and $n+1$ notation for old and new time steps, we find in general:

$$x^{n+1} = D^{-1}\left(b - Rx^{n}\right)$$

$$x_i^{n+1} = \frac{1}{A_{ii}}\left(b_i - \sum_{j \neq i} A_{ij} x_j^n\right)$$

# Diagram of the Jacobi method



# The core of the solver

The full function `jacobi(A, b, tol=1e-2)` is on Canvas, see `it_methods.py`. The gist is:

```
# While not converged or max_it not reached
while (x_diff > tol and it_jac < 1000):
   x_old = x.copy()
   for i in range(N):
      s = 0
      for j in range(N):
         if j != i:
            # Sum off-diagonal*x_old
            s += A[i,j] * x_old[j]
      # Compute new x value
      x[i] = (b[i] - s) / A[i,i]

   # Increase number of iterations
   it_jac += 1
   x_diff = norm(A@x - b)/norm(b)
```

Try to call it from the `laplace_demo.py` file, instead of using `spsolve`.

# A few details on this algorithm

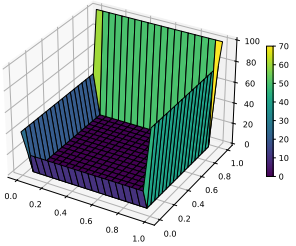
- The while loop holds two aspects
  - A convergence criterion (`norm(A@x - b)/norm(b)> tol`). Some considerations are:
    - $L_1$-norm (sum)
    - $L_2$-norm (Euclidian distance)
    - $L_\infty$-norm (max)
  - Protection against infinite loops (no convergence)
- Reset the sum for each row, before summing for the new unknown node

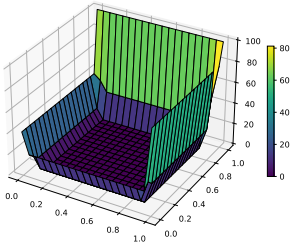
- Start vector x is not shown in the example, but should be there!
- It can have huge impact on performance!
- The for-loops also have a large performance penalty!

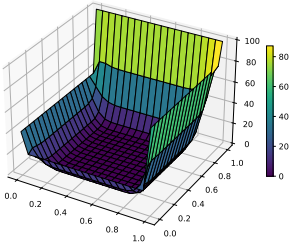
# The solver using array indices

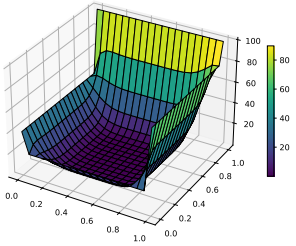
Make a copy of the Jacobian solver, and replace the for-loop on `j` by a vector-operation in a new function `jacobi_vec(A, b, tol=1e-2)`:

# Iterations 1, 2, 5 and 10



# Gauss-Seidel method

The Gauss-Seidel method is quite similar to Jacobi method

- The only difference is that the new estimate $x^{\text{new}}$ is returned to the solution $x^{\text{old}}$ as soon as it is completed
- For following nodes, the updated solution is used immediately
- Our straightforward script (from the Jacobi method) is therefore changed easily:
  - Do not create a `Tnew` array (save memory!)
  - Do not store the solution in `Tnew`, but simply in `T`
  - Do not perform the update step `T=Tnew`
  - See `gaussseidel(A, b, tol=1e-2)` for the algorithm.
- The straightforward script works well for the current Laplace equation, but we define the generic Gauss-Seidel algorithm on the following slides.

# Gauss-Seidel method

- Define a lower and strictly upper triangular matrix, such that $A = L + U$
- Now we can solve AT=b by:

$$\begin{aligned}(L+U)T &= b \\ LT &= b - UT \\ LT^{\text{new}} &= b - UT^{\text{old}} \\ T^{\text{new}} &= L^{-1}(b - UT^{\text{old}})\end{aligned}$$

- Using the $n$ and $n+1$ notation for old and new time steps, we find in for the general Gauss-Seidel method:

$$x^{n+1} = L^{-1}\left(b - Ux^{n}\right)$$

$$x_i^{n+1} = \frac{1}{A_{ii}}\left(b_i - \sum_{j<i} A_{ij}x_j^{n+1} - \sum_{j>i} A_{ij}x_j^{n}\right)$$

# Create yourself: Gauss-Seidel method

- Create a copy of the `jacobi` method and rename it to `gaussseidel`
- Rework the inner algorithm to reflect the changes for the Gauss-Seidel method
- Test! Perform a timing check and check if the solution is correct.
- Next, create a new copy of the just created method and vectorize it, analogous to our vectorized Jacobi method

# Today's outline

- Introduction
- Sparse matrices
- Laplace's equation
- Creating a sparse system
- Iterative methods
- Summary

# Summary

- Partial differential equations can be discretized into sparse systems of linear equations
- Sparse matrices can be stored in memory efficiently using specialised formats (e.g. compressed row storage)
- The Jacobi and Gauss–Seidel methods were introduced as iterative methods; other methods are based on the same principle (successive over-relaxation method, for example)
- Various implementation issues were discussed, e.g. vectorised computing, convergence tolerances

# Direct methods vs. Iterative methods

- Iterative methods converge *gradually* to a solution while direct methods (possibly with partial pivoting) factorise a (set of) matrix(ces) which allow to compute the solution by *substitution*.
- Direct methods generally use more memory, since they need to store also the result matrices.
- A strictly (or irreducibly) diagonally dominant matrix is a prerequisite for convergence of the Jacobi and Gauss-Seidel method.
- For real-life situations; 1D problems are generally solved with direct methods (LU decomposition). If you have systems of more than 1 dimension, a direct method still can be used, if there are no memory issues, otherwise an iterative method would be more attractive.

# Non-linear equations

## One dimensional case

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

S

# Today's outline


- Introduction
  - General
- Direct Iteration Method
  - Passing functions
- Bracketing
- Bisection method
- Secant/False Position
- Brent's method

# Content

## Root finding

How to solve $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ for arbitrary functions $\mathbf{f}$ (i.e., $\mathbf{f}(\mathbf{x})$ move all terms to the left)

- One-dimensional case: 'Bracket' or 'trap' a root between bracketing values, then hunt it down like a rabbit.
- Multi-dimensional case:
  - $N$ equations in $N$ unknowns: You can only hope to find a solution.
  - It may have no (real) solution, or more than one solution!
  - Much more difficult!! "You never know whether a root is near, unless you have found it"

# Outline

**One-dimensional case:**

- Direct iteration method
- Bisection method
- Secant and false position method
- Brent's method
- Newton-Raphson method

**Multi-dimensional case:**

- Newton-Raphson method
- Broyden's method

**In this course we will:**

- Introduction to underlying ideas and algorithms
- Exercises in how to program the methods in Excel and Python.

**Warning**

Do not use routines as black boxes without understanding them!!!

# Today's outline


- Introduction
  - General
- Direct Iteration Method
  - Passing functions
- Bracketing
- Bisection method
- Secant/False Position
- Brent's method

# General Idea

Root finding proceeds by iteration:

- Start with a good initial guess (crucially important!!)
- Use an algorithm to improve the solution until some predetermined convergence criterion is satisfied

Pitfalls:

- Convergence to the wrong root…
- Fails to converge because there is no root
- Fails to converge because your initial estimate was not close enough…

Tips:

- It never hurts to inspect your function graphically
- Pay attention to carefully select initial guesses

**Hamming's motto**

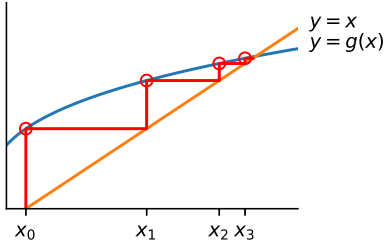
The purpose of computing is insight, not numbers!!

# Direct Iteration Method/Successive Substitutions

Rewrite $f(x) = 0 \Rightarrow x = g(x)$

- Start with an initial guess: $x_0$
- Calculate new estimate with: $x_1 = g(x_0)$
- Continue iteration with: $x_2 = g(x_1)$
- Proceed until: $|x_{i+1} - x_i| < \varepsilon$

When the process converges, taking a smaller value for $x_{i+1} - x_i$ results in a more accurate solution, but more iterations need to be performed.



# Direct Iteration Method - Exercise 1

Find the root of

$$f(x) = x^3 - 3x^2 - 3x - 4$$

## Attempt 1

Rewrite as $x = (3x^2 + 3x + 4)^{(1/3)}$

- Solve in Excel
- Solve in Python

## Attempt 2

Rewrite as: $x = (x^3 - 3x^2 - 4)/3$

- Solve in Excel
- Solve in Python

# Intermezzo: Functions Revisited

- In Python, you can define your own functions to reuse certain functionalities. We can define a mathematical function at the top of a file, or in a separate file with `.py` extension:

```
def demo_f1(x):
    return x**2 + np.exp(x)
```

- The first line contains the function name, in this case `demo_f1`
- The return statement defines the output, `x` is defined as input
- It can use `x` as a scalar as well as a vector by using NumPy: e.g. `np.exp()`
  - If `x` is a vector, the output is also a vector.
- In case you define your function in a separate file, e.g. `nonlin_functions.py`, you can import the function into another file through:

```
from nonlin_functions import demo_f1
```

# Passing Functions in Python

- To solve $f(x) = x^2 - 4x + 2 = 0$ numerically, we can write a function that returns the value of $f(x)$:

```
def MyFunc(x): # Note: case sensitive!!
    return x**2 - 4*x + 2
```

- The function can be assigned to a variable as an alias:

```
f = MyFunc
a = 4
b = f(a)
```

```
2
```

- We can then call a solving routine (e.g., `fsolve` from SciPy):

```
from scipy.optimize import fsolve
ans = fsolve(MyFunc, 5)
ans = fsolve(lambda x: x**2 - 4*x + 2, 5)
```

```
array([3.41421356])
array([3.41421356])
```

# Passing Functions in Python

- We can also make our own function, that takes another function as an argument:

```
import matplotlib.pyplot as plt
import numpy as np

def draw_my_function(func):
    # Draws a function in the range [0, 10] using 20 data points.
    # 'func' is a function that can be any actual function.
    x = np.linspace(0, 10, 20)
    y = func(x)
    plt.plot(x, y, "-o")
    plt.show()
```

- Now we can call the function with another function, either a lambda function or a common function:

```
f = lambda x: x**2 - 4*x + 2
draw_my_function(f)
```

# Direct Iteration Method - Exercise 1

Find the root of

$$f(x) = x^3 - 3x^2 - 3x - 4$$

## Attempt 1

Rewrite as $x = (3x^2 + 3x + 4)^{(1/3)}$

- Solve in Excel
- Solve in Python

## Attempt 2

Rewrite as: $x = (x^3 - 3x^2 - 4)/3$

- Solve in Excel
- Solve in Python

# Direct Iteration Method - Exercise 1

Find the root of $f(x) = x^3 - 3x^2 - 3x - 4$ with the direct iteration method in Excel:

First attempt:

| Iteration | Formula | Result |
|---|---|---|
| 1 | $(3x^2 + 3x + 4)^{(1/3)}$ | 2 |
| 2 | | 3.115 |
| 3 | | 3.489 |
| ⋮ | | ⋮ |
| 10 | | 3.990 |

**Converges!**

Second attempt:

| Iteration | Formula | Result |
|---|---|---|
| 1 | $x = (x^3 - 3x^2 - 4)/3$ | −1 |
| 2 | | −2.375 |
| 3 | | −11.439 |
| ⋮ | | ⋮ |
| 10 | | #NUM! |

**Diverges!**

# Direct Iteration Method - Exercise 1

Find the root of $f(x) = x^3 - 3x^2 - 3x - 4 = 0$ with the direct iteration method in Python:
A simple script:

```
x = 2.5
print(f"i: {0}, x: {x:.6e}")
for i in range(1, 21):
    x = (3*x**2 + 3*x + 4)**(1/3)
    print(f"i: {i}, x: {x:.6e}")
```

```
i: 0, x: 2.500000e+00
i: 1, x: 3.115840e+00
i: 2, x: 3.489024e+00
...
i: 19, x: 3.999970e+00
i: 20, x: 3.999983e+00
```

**Lesson**

Not very flexible/reusable → use functions

# Direct Iteration Method - Exercise 1

Find the root of the equation $f(x) = x^3 - 3x^2 - 3x - 4 = 0$ using the direct iteration method in Python.

- First, define the functions.

```
def MyFnc1(x):
    return (3*x**2 + 3*x + 4)**(1/3)

def MyFnc2(x):
    return (x**3 - 3*x**2 - 4) / 3
```

- Then, create a function to carry out the Direct Iteration algorithm.

# Direct Iteration Method - Exercise 1

Find the root of the equation $f(x) = x^3 - 3x^2 - 3x - 4 = 0$ using the direct iteration method in Python.

- Finally, call the Direct Iteration function with the appropriate parameters.

```
1 DirectIterationMethod(MyFnc1, 2.5, 1e-3)
2 DirectIterationMethod(MyFnc2, 2.5, 1e-3)
```

```
i: 0, x: 2.500000e+00
i: 1, x: 3.115840e+00
i: 2, x: 3.489024e+00
i: 3, x: 3.708113e+00
...
i: 9, x: 3.990573e+00
i: 10, x: 3.994696e+00
i: 11, x: 3.997016e+00
i: 12, x: 3.998321e+00
```

```
i: 0, x: 2.500000e+00
i: 1, x: -2.375000e+00
i: 2, x: -1.143945e+01
i: 3, x: -6.311875e+02
i: 4, x: -8.421961e+07
i: 5, x: -1.991216e+23
i: 6, x: -2.631687e+69
Traceback (most recent
     call last):
```

**Thinking**

Discuss why it converges with **`MyFnc1`** and diverges with **`MyFnc2`**

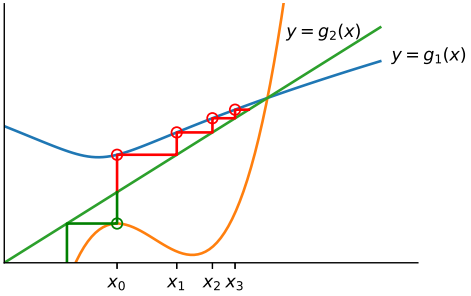
# Direct Iteration Method

- Exercise 1: Find the root of the equation

$$f(x) = x^3 - 3x^2 - 3x - 4 = 0$$

using the direct iteration method.

- Observe that the method only works effectively when $g'(x_i) < 1$. Even then, it may not converge quickly.




**Point**

The iterations can be represented using the following relations:

$$x_{i+1} = g(x_i) + g'(x_i)(x - x_i)$$
$$x_{i+2} = g(x_{i+1}) + g'(x_{i+1})(x_{i+1} - x_i)$$
$$|x_{i+2} - x_{i+1}| = |g'(x_i)||x_{i+1} - x_i|$$

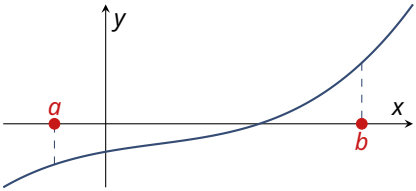
Convergence if $|g'(x_i)| \leq 1$

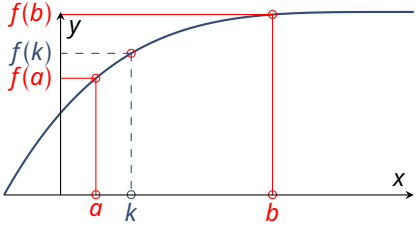
# Today's outline


- Introduction
  - General
- Direct Iteration Method
  - Passing functions
- Bracketing
- Bisection method
- Secant/False Position
- Brent's method

# Bracketing

Bracketing a root involves identifying an interval $(a, b)$ within which the function changes its sign.




- If $f(a)$ and $f(b)$ have opposite signs, it indicates that at least one root lies in the interval $(a, b)$, assuming the function is continuous in the interval.

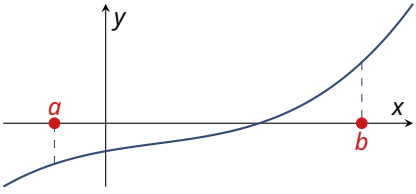
**Intermediate value theorem**

States that if $f(x)$ is continuous on $[a, b]$ and $k$ is a constant lying between $f(a)$ and $f(b)$, then there exists a value $x \in [a, b]$ such that $f(x) = k$.

# Bracketing

**What's the point?**

Bracketing a root = Understanding that the function changes its sign in a specified interval, which is termed as bracketing a root.

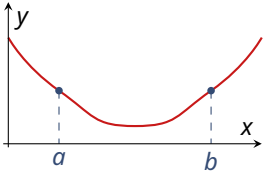



**General best advice**:

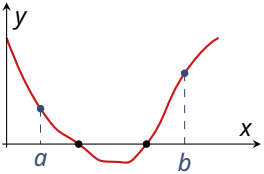
- Always bracket a root before attempting to converge on a solution.
- Never allow your iteration method to get outside the best bracketing bounds...

# General Idea

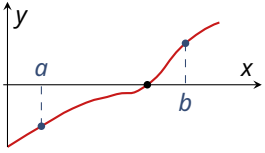
Potential issues to be cautious of while bracketing:



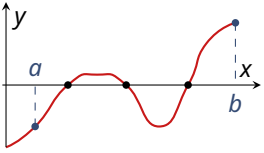

No answer (no root found)




Oops! Encountering two roots




Ideal scenario with one root found




Finding three roots (might work temporarily)

# Bracketing - exercise 2

1. Write a Python function to bracket a function, starting with an initially guessed range $x_1$ and $x_2$ through the expansion of the interval.
2. Develop a program to ascertain the minimum number of roots existing within the $x_1$ and $x_2$ interval.
3. Note: These functions can be integrated to formulate a function that yields bracketing intervals for diverse roots.
4. Test the function for $f(x) = x^2 - 4x + 2$

# Bracketing - exercise 2

- Initially, if feasible, draft a graph using the following Python commands:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 5, 50)
y = x**2 - 4*x + 2
plt.figure()
plt.plot(x, y, x, np.zeros(len(x)))
plt.axis('tight')
plt.grid(True)
plt.show()
```

- This graphical representation instantly reveals the existence of two roots, evaluated as:

$$x_1 = 2 - \sqrt{2} \approx 0.59 \quad , \qquad x_2 = 2 + \sqrt{2} \approx 3.41$$

# Bracketing - exercise 2

```python
def find_root_by_bracketing(func, x1, x2, tol=1e-6, max_iter=1000):
    # Ensure the bracket is valid
    if func(x1) * func(x2) > 0:
        print('The bracket is invalid. The function must have opposite signs at the two endpoints.')
        return False

    # Loop until we find the root or exceed the maximum number of iterations
    for i in range(max_iter):
        # Find the midpoint
        x_mid = (x1 + x2) / 2

        # Check if we found the root
        if abs(func(x_mid)) < tol:
            print(f'Root found: {x_mid}')
            return True

        # Narrow down the bracket
        if func(x_mid) * func(x1) < 0:
            x2 = x_mid
        else:
            x1 = x_mid

    # If we reach here, we did not find the root within the maximum number of iterations
    print('Failed to find the root within the maximum number of iterations.')
    return False
```

**Steps:**

- Formulate a function to augment the interval $(x_1, x_2)$ up to a maximum of 250 iterations or until a root is discovered.
- The function should:
  - Return true if a root is found, and false otherwise.
  - Showcase the results.

# Bracketing

### Exercise 2: Function to Bracket a Function

```
def brak(func, x1, x2, n):
    nroot = 0
    dx = (x2 - x1) / n
    xb1 = []
    xb2 = []

    x = x1
    for i in range(n):
        x += dx
        if func(x) * func(x - dx) <= 0:
            nroot += 1
            xb1.append(x - dx)
            xb2.append(x)

    for i in range(nroot):
        print(f'Root {i+1} in bracketing interval
              [{xb1[i]}, {xb2[i]}]')
    else:
        if nroot == 0:
            print('No roots found!')
```

**Steps:**

- The function subdivides the interval $(x_1, x_2)$ into $n$ parts to check for at least one root.
- It returns the left and right boundaries of the intervals where roots are found in arrays xb1 and xb2.

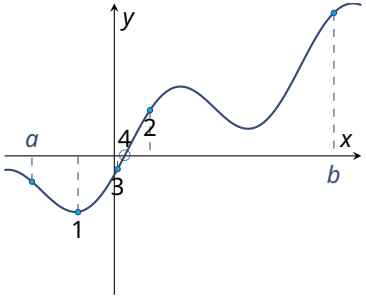
# Today's outline


- Introduction
  - General
- Direct Iteration Method
  - Passing functions
- Bracketing
- Bisection method
- Secant/False Position
- Brent's method

# Bisection Method

Bisection Algorithm:

- Within a certain interval, the function crosses zero, indicated by a change in sign.
- Evaluate the function value at the midpoint of the interval and examine its sign.
- The midpoint then supersedes the limit sharing its sign.




**Properties**

- Pros: The method is infallible.
- Cons: Convergence is relatively slow.

# Bisection Method

**Exercise 3**

- Write a function in Excel to find a root of a function using the bisection method.
- Assume that an initial bracketing interval $(x_1, x_2)$ is provided.
- Specify the required tolerance.
- Output the required number of iterations.
- Implement the same in Python.

# Exercise 3

**Bisection Method in Excel:**

| it | $x_1$ | $x_2$ | $f_1$ | $f_2$ | xmid | fmid | Interval Size |
|---|---|---|---|---|---|---|---|
| 0 | -2 | 2 | 14 | -2 | 0 | 2 | 4 |
| 1 | 0 | 2 | 2 | -2 | 1 | -1 | 2 |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| 25 | 0.585786 | 0.585786 | $1 \times 10^{-7}$ | $-6.8 \times 10^{-8}$ | 0.585786 | $1.58 \times 10^{-8}$ | $5.96 \times 10^{-8}$ |

Note: The table represents a sequence of iterations showing how the bisection method converges to a root with each step, demonstrating variable updates and interval size reduction.

# Bisection Method

## Exercise 3: Python Implementation

```python
def bisection(func, a, b, tol, maxIter):
    if func(a) * func(b) > 0:
        print('Error: f(a) and f(b) must have different signs.'
              )
        return None

    iter = 0
    while (b - a) / 2 > tol:
        iter += 1
        if iter >= maxIter:
            print('Maximum iterations reached')
            return None

        c = (a + b) / 2
        print(f'Iteration {iter}: Current estimate: {c}')

        if func(c) == 0:
            return c

        if np.sign(func(c)) != np.sign(func(a)):
            b = c
        else:
            a = c

    return (a + b) / 2
```

- Criterion used for both the function value and the step size.
- While loop usually requires protection for a maximum number of iterations.
- Bisection is sure to converge.
- Root found in 25 iterations. Can we optimize it further?

# Bisection Method

**Required Number of Iterations:**

- Interval bounds containing the root decrease by a factor of 2 after each iteration.

$$\varepsilon_{n+1} = \frac{1}{2}\varepsilon_n \quad \Rightarrow \quad \boxed{n = \log_2 \frac{\varepsilon_0}{tol}}$$

$\varepsilon_0$ = initial bracketing interval,
$tol$ = desired tolerance.

- After 50 iterations, the interval is decreased by a factor of $2^{50} = 10^{15}$.
- Consider machine accuracy when setting tolerance.
- Order of convergence is 1:

$$\boxed{\varepsilon_{n+1} = K\varepsilon_n^m}$$

- $m = 1$: linear convergence.
- $m = 2$: quadratic convergence.

- Bisection method will:
  - Find one of the roots if there is more than one.
  - Find the singularity if there is no root but a singularity exists.

# Today's outline

- Introduction
  - General
- Direct Iteration Method
  - Passing functions
- Bracketing
- Bisection method
- Secant/False Position
- Brent's method

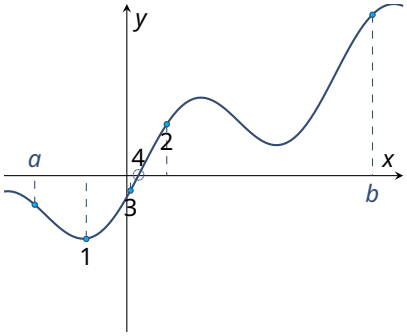
# Secant and False Position Method

**Secant/False Position (Regula Falsi) Method**

- Provides faster convergence given sufficiently smooth behavior.
- Differs from the bisection method in the choice of the next point:
  - **Bisection**: selects the mid-point of the interval.
  - **Secant/False position**: chooses the point where the approximating line intersects the axis.
- Adopts a new estimate by discarding one of the boundary points:
  - **Secant**: retains the most recent of the previous estimates.
  - **False position**: maintains the prior estimate with the opposite sign to ensure the points continue to bracket the root.

# Secant and False Position Method: Comparison

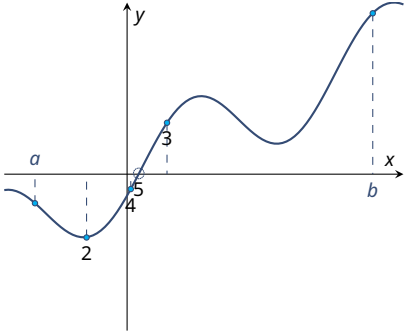
**Secant Method**




- Slightly faster convergence:

$$\lim_{n\to\infty} \left|\varepsilon_{n+1}\right| = K\left|\varepsilon_n\right|^{1.618}$$

**False Position Method**




- Guaranteed convergence

# Secant and False Position Method

**Exercise 4:**

- Write a function in Excel and Python to find a root of a function using the Secant and False position methods.
- Assume that an initial bracketing interval $(x_1, x_2)$ is provided.
- Specify the required tolerance.
- Output the required number of iterations.
- Compare the bisection, false position, and secant methods.

# Secant and False Position Method

**Exercise 4:**

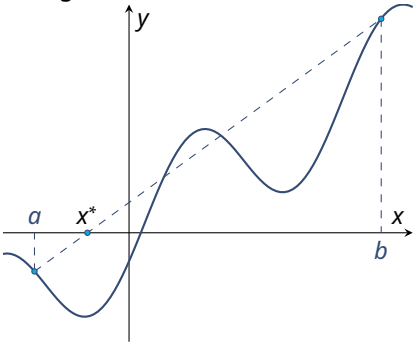
- Determination of the abscissa of the approximating line:

- Determine the approximating line using the expression:

$$f(x) \approx f(a) + \frac{f(b) - f(a)}{b - a}(x - a)$$

- Determine the abscissa where $f(x^*) = 0$:

$$x^* = a - \frac{f(a)(b - a)}{f(b) - f(a)}$$
$$= \frac{af(b) - bf(a)}{f(b) - f(a)}$$




Note: In the above equations, $a$ and $b$ are the initial guesses/boundaries where the root is suspected to be, and $f(x)$ is the function for which we are finding the root.

# Secant and False Position Method

**Exercise 4:**

- Write a function in Excel and Python to find a root of a function using the Secant and the False position methods.
- Assume that an initial bracketing interval $(x_1, x_2)$ is provided.
- Specify the required tolerance.
- Output the required number of iterations.
- Compare the bisection, false position, and secant methods.

# Secant and False Position Method

**Exercise 4: False Position Method in Excel**

| iteration | xa | xb | fa | fb | x absc | fabsc | interval |
|---|---|---|---|---|---|---|---|
| 0 | -1.5000 | 4.0000 | -0.3895 | 2.1628 | -0.6606 | -0.8455 | 5.5000 |
| 1 | -0.6606 | 4.0000 | -0.8455 | 2.1628 | 0.6493 | 0.6896 | 4.6606 |
| 2 | -0.6606 | 0.6493 | -0.8455 | 0.6896 | 0.0609 | -0.1972 | 1.3099 |
| 3 | 0.0609 | 0.6493 | -0.1972 | 0.6896 | 0.1917 | 0.0070 | 0.5884 |
| 4 | 0.0609 | 0.1917 | -0.1972 | 0.0070 | 0.1873 | -0.0001 | 0.1308 |
| 5 | 0.1873 | 0.1917 | -0.0001 | 0.0070 | 0.1874 | 0.0000 | 0.0045 |
| 6 | 0.1874 | 0.1917 | 0.0000 | 0.0070 | 0.1874 | 0.0000 | 0.0044 |
| 7 | 0.1874 | 0.1917 | 0.0000 | 0.0070 | 0.1874 | 0.0000 | 0.0044 |

Relevant expressions:

- `a=IF((a*fa)<0,a,xbsc)`
- `b=IF((b*fb)<0,b,xbsc)`
- `xbsc=a - fa*(xb-xa)/(fb-fa)`

# Secant and False Position Method

**Exercise 4:**

- Write a function in Excel and Python to find a root of a function using the Secant and the False position methods.
- Assume that an initial bracketing interval $(x_1, x_2)$ is provided.
- Also the required tolerance is specified.
- Also output the required number of iterations.
- Compare the bisection, false position, and secant methods.

# Secant and False Position Method

**Exercise 4: Secant method in excel**

| iteration | x | f |
|---|---|---|
| -1 | 2.0000 | 0.5895 |
| 0 | -1.0000 | -0.7591 |
| 1 | 0.6886 | 0.7368 |
| 2 | -0.1431 | -0.4819 |
| 3 | 0.1857 | -0.0026 |
| 4 | 0.1875 | 0.0002 |
| 5 | 0.1874 | 0.0000 |

**Relevant expressions:**

$$x_n = x_{n-1} - f(x_{n-1}) \frac{x_{n-1} - x_{n-2}}{f(x_{n-1}) - f(x_{n-2})}$$

$$f_n = f(x_n)$$

# Secant and False Position Method

### Exercise 4: False position method in Python

```python
def false_position(f, x0, x1, tol, max_iter):
    if f(x0) * f(x1) > 0:
        raise ValueError('f(x0) and f(x1) must have different signs.')

    history = []

    for i in range(max_iter):
        x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
        history.append(x2)

        if abs(f(x2)) < tol:
            break

        if f(x2) * f(x0) < 0:
            x1 = x2
        else:
            x0 = x2

    root = x2
    return root, history
```

Calling the function:

```python
secant_method(lambda x: x**2 - 4*x + 2, 0, 2, 1e-7, 100)
```

# Secant and False Position Method

### Exercise 4: Secant method in Python

```
def secant_method(f, x0, x1, tol, max_iter):
    history = [x0, x1]

    for i in range(1, max_iter):
        x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0))
        history.append(x2)

        if abs(x2 - x1) < tol:
            break

        x0 = x1
        x1 = x2

    root = x1
    return root, history
```

Calling the function:

```
false_position(lambda x: x**2 — 4*x + 2, 0, 2, 1e—7, 100)
```

# Comparison of Methods

**Exercise 4:**

- $\text{tol}_{\text{eps}}, \text{tol}_{\text{func2}} = 1e-15, \quad \text{and} \quad (x_1, x_2) = (0, 2)$
- $f(x) = x^2 - 4x + 2 = 0$

| Method | Nr. of iterations |
|---|---|
| Bisection | 52 |
| False position | 22 |
| Secant | 9 |

```
from scipy.optimize import root_scalar

root_scalar(lambda x: x**2 — 4*x + 2, method='brentq', bracket=[0, 2], xtol=1e—15)
```

Note the initial bracketing steps in root_scalar!

# Today's outline


- Introduction
  - General
- Direct Iteration Method
  - Passing functions
- Bracketing
- Bisection method
- Secant/False Position
- Brent's method

# Brent's Method

**Features of Brent's method:**

- Superlinear convergence with the sureness of bisection
- Keeps track of superlinear convergence, and if not achieved, alternates with bisection steps, ensuring at least linear convergence
- Implemented in MATLAB's `scipy.optimize.fzero` function:
  - Utilizes root-bracketing
  - Bisection/secant/inverse quadratic interpolation
- Inverse quadratic interpolation:
  - Uses three prior points to fit an inverse quadratic function ($x(y)$)
  - Involves contingency plans for roots falling outside the brackets

# Brent's method

**Formulas:**

$$x = b + \frac{P}{Q},$$

$$P = S\left[T(R-T)(c-b) - (1-R)(b-a)\right],$$

$$Q = (T-1)(R-1)(S-1),$$

$$R = \frac{f(b)}{f(c)}$$

$$S = \frac{f(b)}{f(a)}$$

$$T = \frac{f(a)}{f(c)}$$

- b = current best estimate
- $P/Q$ = a 'small' correction

Note: If P/Q does not land within the bounds or if bounds are not collapsing quickly enough, a bisection step is taken.

# Brent's method script

```python
def brent_method(f, a, b, tol=1e-6, max_iter=100):
    if f(a) * f(b) >= 0:
        raise ValueError("f(a) and f(b) must have different signs.")
    # Initialize variables
    c = a
    fa = f(a)
    fb = f(b)
    fc = fa
    history = [a, b]
    d = e = b - a
    for _ in range(max_iter):
        if fa * fc > 0:
            c = a
            fc = fa
            d = e = b - a
        if abs(fc) < abs(fb):
            a, b, c = b, c, a
            fa, fb, fc = fb, fc, fa
        tol1 = 2 * 1.0e-16 * abs(b) + 0.5 * tol
        xm = 0.5 * (c - b)
        if abs(xm) <= tol1 or fb == 0:
            return b, history
        if abs(e) >= tol1 and abs(fa) > abs(fb):
            s = fb / fa
            if a == c:
                # Linear interpolation (Secant method)
                p = 2 * xm * s
                q = 1 - s
            else:
                # Inverse quadratic interpolation
                q = fa / fc
                r = fb / fc
                p = s * (2 * xm * q * (q - r) - (b - a) * (r - 1))
                q = (q - 1) * (r - 1) * (s - 1)
            if p > 0:
                q = -q
            p = abs(p)

            if 2 * p < min(3 * xm * q - abs(tol1 * q), abs(e * q)):
                e = d
                d = p / q
            else:
                d = xm
                e = d
        else:
            d = xm
            e = d
        a = b
        fa = fb
        if abs(d) > tol1:
            b += d
        else:
            b += tol1 if xm > 0 else -tol1

        fb = f(b)
        history.append(b)
    raise ValueError("Maximum number of iterations reached.")
```

# Using Excel for Solving Non-linear Equations: Goal-Seek and Solver

**Setting up Goal-Seek and Solver in Excel:**

- Available in Excel with some prerequisites installation.
- For Excel 2010:
  - Install via `Excel` → `File` → `Options` → `Add-Ins` → `Go` (at the bottom) → `Select solver add-in`.
  - Accessible through the 'data' menu ('Oplosser' in Dutch).

**Procedure for solving:**

- Select the goal-cell.
- Specify whether you want to minimize, maximize, or set a certain value.
- Define the variable cells for Excel to adjust to find the solution.
- Set the boundary conditions (if any).
- Click 'solve', possibly after setting advanced options.

# Excel: Goal-Seek Example

**Using Goal-Seek to find a solution:**

- The Goal-Seek function can set the goal-cell to a desired value by adjusting another cell.
- Steps:
  1. Open Excel and input the following data:

| A | x | B |
|---|---|---|
| 1 | x | 3 |
| 2 | f(x) | f(x) = -3*B1^2 - 5*B1 + 2 |
| 3 | | |

  2. Navigate to `Data → What-if Analysis → Goal Seek` and input:
     - Set cell: B2
     - To value: 0
     - By changing cell: B1
  3. Press OK to find a solution of approximately 0.3333.

# Excel: Solver Example

**Using Solver to Find Solutions with Boundary Conditions:**

- Solver can adjust values in one or more cells to reach a desired goal-cell value, respecting specified boundary conditions.
- Example sheet setup:

| | A | B | C |
|---|---|---|---|
| 1 | | x | f(x) |
| 2 | x1 | 3 | =2*B2*B3-B3+2 |
| 3 | x2 | 4 | =2*B3-4*B2-4 |

- Procedure:
  1. Navigate to `Data → Solver`.
  2. Set the goal function to C2 with a target value of 0.
  3. Add a boundary condition: C3 = 0.
  4. Specify the cells to change as `$B$2:$B$3`.
  5. Click "Solve" to find B2 = 0 and B3 = 2 as solutions.

# Non-linear equations

## Towards the multi-dimensional case

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline

- Python solvers
- Newton-Raphson method
- Multi-dimensional Newton-Raphson

# Non-linear Equation Solving in Python (1 var)

**Single Variable Non-linear Zero Finding:**

- Use the `root_scalar` function from `scipy.optimize` for finding zeros of a single-variable non-linear function.
- Be aware of the initial bracketing steps in `root_scalar`.

```
1 from scipy.optimize import root_scalar
2
3 root_scalar(lambda x: -3*x**2 - 5*x + 2, method='brentq', bracket=[1, 4], xtol=1e-15)
```

```
    converged: True
         flag: converged
function_calls: 10
   iterations: 9
         root: 0.3333333333333333
```

# Non-linear equation solver in Python ($\geq$ 2 var)

**Solving Systems of Non-linear Equations (Multiple Variables):**

- Use `fsolve` from `scipy.optimize` for systems involving multiple variables.
- Suitable for non-linear equations with two or more variables.

```python
from scipy.optimize import fsolve

def equations(x):
    return [2*x[0]*x[1] - x[1] + 2, 2*x[1] - 4*x[0] - 4]

fsolve(equations, [1, 1], xtol=1e-15)
```

# Newton-Raphson Method

**Algorithm:**

- Requires evaluating both the function $f(x)$ and its derivative $f'(x)$ at arbitrary points.
- Extend the tangent line at the current point $x_i$ until it intersects with zero.
- Set the next guess $x_{i+1}$ as the abscissa of that zero crossing.
- For small enough $\delta x$ and well-behaved functions, non-linear terms in the Taylor series become unimportant.

$$f(x) \approx f(x_i) + f'(x_i)\delta x + \mathcal{O}(\delta x^2) + \dots$$

$$0 \approx f(x_i) + f'(x_i)\delta x$$

$$\delta x \approx -\frac{f(x_i)}{f'(x_i)}$$

$$\boxed{x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}}$$

- Can be extended to higher dimensions.
- Requires an initial guess close enough to the root to avoid failure.

# Newton-Raphson Method

**Example with the Formula:**

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

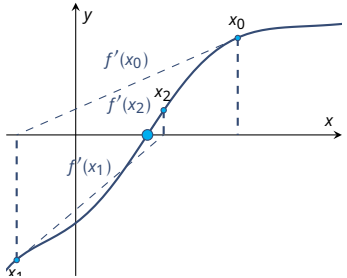
**When it works:**

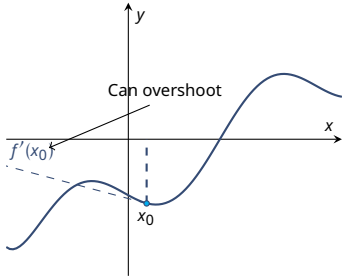
- Converges enormously fast when it functions correctly.

**When it does not work:**

- Underrelaxation can sometimes be helpful.
- Underrelaxation formula:

$$x_{n+1} = (1-\lambda)x_n + \lambda x_{n+1}$$
$$\lambda \in [0,1]$$



# Newton-Raphson Method

**Basic Algorithm:**
Given initial $x$ and a required tolerance $\varepsilon > 0$,

1. Compute $f(x)$ and $f'(x)$.
2. If $|f(x)| \leq \varepsilon$, return $x$.
3. Update $x$ using the formula:

$$x \leftarrow x - \frac{f(x)}{f'(x)}$$

Repeat the above steps until a solution is found within the tolerance or the maximum number of iterations is exceeded.

# Newton-Raphson Method

**Exercise 5: Newton-Raphson Method in Excel**

| iteration | x | f | f' |
|---|---|---|---|
| 0 | -2 | 14 | -8 |
| 1 | -0.25 | 3.0625 | -4.5 |
| 2 | 0.430556 | 0.463156 | -3.13889 |
| 3 | 0.57811 | 0.021772 | -2.84378 |
| 4 | 0.585766 | 5.86E-05 | -2.82847 |
| 5 | 0.585786 | 4.29E-10 | -2.82843 |
| 6 | 0.585786 | 0 | -2.82843 |

Used formulas:

$$f(x) = x^2 - 4x + 2$$

$$f' = 2x - 4$$

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

# Newton-Raphson Method

**Why is the Newton-Raphson so powerful?**

- High rate of convergence
- Can achieve quadratic convergence!

**Derivation of quadratic convergence:**

1. Subtract solution
2. Define error
3. Express in terms of error
4. Use taylor expansion around solution
5. Rewrite in terms of error
6. Ignore higher order terms

$$x_{n+1} - x^* = x_n - x^* - f(x_n)/f'(x_n)$$

$$\varepsilon_n = x_n - x^*$$

$$\varepsilon_{n+1} = \varepsilon_n - f(x_n)/f'(x_n)$$

$$\varepsilon_{n+1} \approx \varepsilon_n - \frac{f(x^*) + f'(x^*)\varepsilon_n + f''(x^*)\varepsilon_n^2}{f'(x^*) + \mathcal{O}(\varepsilon_n^2)}$$

$$\varepsilon_{n+1} \approx -\frac{f''(x^*)\varepsilon_n^2 + \mathcal{O}(\varepsilon_n^3)}{f'(x^*) + \mathcal{O}(\varepsilon_n^2)}$$

$$\boxed{\varepsilon_{n+1} \approx -K\varepsilon_n^2}$$

# Newton-Raphson Method

**Deriving the order of convergence**

- The main issue with determining the order of convergence is that the solution is not known a priori
- To get around this issue it is possible to rewrite the problem in terms of known quantities.
- In the coming derivation, the following steps are taken to derive the order of convergence:
  1. The formal definition of $K$ is given in terms of $\varepsilon$ and the order of convergence $m$
  2. This formal definition is used to rewrite the fraction of successive errors
  3. Logarithms are used to isolate $m$
- Since the $\varepsilon$ can't be computed without knowing the solution, the following approximation is made before plugging the final result:

$$\varepsilon_{n+1} \approx |x_{n+1} - x_n|$$

# Newton-Raphson Method

1. Formal definition of $K$ and $m$:

$$\lim_{n\to\infty} |\varepsilon_{n+1}| = K|\varepsilon_n|^m$$

2. Fraction of successive errors:

$$\frac{|\varepsilon_{n+1}|}{|\varepsilon_n|} = \frac{K|\varepsilon_n|^m}{K|\varepsilon_{n-1}|^m} \Rightarrow \left|\frac{\varepsilon_n}{\varepsilon_{n-1}}\right|^m$$

3. Extracting $m$:

$$\ln\left|\frac{\varepsilon_{n+1}}{\varepsilon_n}\right| = m\ln\left|\frac{\varepsilon_n}{\varepsilon_{n-1}}\right| \Rightarrow \boxed{m = \frac{\ln\left|\frac{\varepsilon_{n+1}}{\varepsilon_n}\right|}{\ln\left|\frac{\varepsilon_n}{\varepsilon_{n-1}}\right|}}$$

# Newton-Raphson Method

**Exercise 5: Newton-Raphson Method in Excel**

- In this exercise, you will be working with the Newton-Raphson method implemented in Excel.
- The order of convergence ($m$) can be estimated using the relation:

$$m = \frac{\ln\left(\frac{\varepsilon_{n+1}}{\varepsilon_n}\right)}{\ln\left(\frac{\varepsilon_n}{\varepsilon_{n-1}}\right)}$$

  Where it is assumed that $\varepsilon$ can be approximated by:

$$\varepsilon_{n+1} = |x_{n+1} - x_n|$$

- Solve a problem using the Newton-Raphson method in Excel and verify the order of convergence using the formulas above.

# Newton-Raphson Method

**Exercise 5: Newton-Raphson Method in Excel solution**

| iteration | x | f | f' | eps | m |
|---|---|---|---|---|---|
| 0 | -2.000 | 14.000 | -8.000 | 1.750 | |
| 1 | -0.250 | 3.063 | -4.500 | 0.681 | 1.619 |
| 2 | 0.431 | 0.463 | -3.139 | 0.148 | 1.935 |
| 3 | 0.578 | 0.022 | -2.844 | 0.008 | 1.998 |
| 4 | 0.586 | 0.000 | -2.828 | 0.000 | 2.000 |
| 5 | 0.586 | 0.000 | -2.828 | 0.000 | |
| 6 | 0.586 | 0.000 | -2.828 | | |

Used formulas:

$$x_{n+1} = x_n - f(x_n)/f'(x_n)$$

$$m = \frac{\ln\left(\frac{\varepsilon_{n+1}}{\varepsilon_n}\right)}{\ln\left(\frac{\varepsilon_n}{\varepsilon_{n-1}}\right)}$$

$$\varepsilon_{n+1} = |x_{n+1} - x_n|$$

# Newton-Raphson Method

**Exercise 6: Newton-Raphson Method in Python**

- Write a Python function to find the root of a function using the Newton-Raphson method.
- Assume that an initial guess $x_0$ is provided.
- The required tolerance for the solution should also be provided.
- Output the results of each iteration.
- Compute the order of convergence.

# Newton-Raphson Method

**Modifications to the Basic Algorithm**

- If $f'(x)$ is not known or is difficult to compute/program, a local numerical approximation can be used:

$$f'(x) \approx \frac{f(x+\delta x) - f(x)}{\delta x} \quad (\text{with } \delta x \sim 10^{-8})$$

- The chosen $\delta x$ should be small but not too small to avoid round-off errors.
- The method should be combined with:
  - A bracketing method to prevent the solution from wandering outside of the bounds.
  - A reduced Newton step method for more robustness; don't take the full step if the error doesn't decrease sufficiently.
  - Sophisticated step size controls like local line searches and backtracking using cubic interpolation for global convergence.

# Newton-Raphson Method

**How to Solve for Arbitrary Functions $f$: "Root Finding"**

- **One-dimensional case**:
  - Move all terms to the left to have $f(x) = 0$.
  - Bracket or 'trap' a root between bracketing values, then hunt it down "like a rabbit."
- **Multi-dimensional case**:
  - Involving $N$ equations in $N$ unknowns.
  - It is not guaranteed to find a solution; it might not have a real solution or might have more than one solution.
  - Much more challenging compared to the one-dimensional case.
  - It is unpredictable to know if a root is nearby unless it has been found.

# Newton-Raphson Method: Multi-dimensional Case (1)

- **Two-dimensional case:**

$$f(x,y) = 0,$$
$$g(x,y) = 0.$$

- **Multivariate Taylor series expansion:**

$$f(x+\delta x, y+\delta y) \approx f(x,y) + \frac{\partial f}{\partial x}\delta x + \frac{\partial f}{\partial y}\delta y + O(\delta x^2, \delta y^2) = 0$$

- **Neglecting higher order terms:**

$$g(x+\delta x, y+\delta y) \approx g(x,y) + \frac{\partial g}{\partial x}\delta x + \frac{\partial g}{\partial y}\delta y + O(\delta x^2, \delta y^2) = 0$$

- Leads to two linear equations in the unknowns $\delta x$ and $\delta y$:

$$\frac{\partial f}{\partial x}\delta x + \frac{\partial f}{\partial y}\delta y = -f(x,y),$$
$$\frac{\partial g}{\partial x}\delta x + \frac{\partial g}{\partial y}\delta y = -g(x,y).$$

# Newton-Raphson Method: Multi-dimensional Case (2)

**In matrix notation:**

$$\begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{bmatrix} \begin{bmatrix} \delta x \\ \delta y \end{bmatrix} = \begin{bmatrix} -f(x,y) \\ -g(x,y) \end{bmatrix}$$

**Elements of this equation:**

- Jacobian matrix:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{bmatrix}$$

- The small displacement vector and $\mathbf{f}$:

$$\delta\mathbf{x} = \begin{bmatrix} \delta x \\ \delta y \end{bmatrix} \qquad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} f(x,y) \\ g(x,y) \end{bmatrix}$$

**Solving equation by matrix inversion:**

- Expressing the stepping equation in matrix notation:

$$\mathbf{J}(\mathbf{x}) \cdot \delta\mathbf{x} = -\mathbf{f}(\mathbf{x})$$

- Multiplying both sides by the inverse of $\mathbf{J}$:

$$\delta\mathbf{x} = -\mathbf{J}^{-1}(\mathbf{x}) \cdot \mathbf{f}(\mathbf{x})$$

- Writing in terms of iteration number:

$$\boxed{\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{J}^{-1}(\mathbf{x}_n) \cdot \mathbf{f}(\mathbf{x}_n)}$$

# Newton-Raphson Method: Multi-dimensional Case (2)

**In matrix notation:**

$$\begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{bmatrix} \begin{bmatrix} \delta x \\ \delta y \end{bmatrix} = \begin{bmatrix} -f(x,y) \\ -g(x,y) \end{bmatrix}$$

**Elements of this equation:**

- Jacobian matrix:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \\ \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{bmatrix}$$

- The small displacement vector and **f**:

$$\delta\mathbf{x} = \begin{bmatrix} \delta x \\ \delta y \end{bmatrix} \qquad \mathbf{f}(\mathbf{x}) = \begin{bmatrix} f(x,y) \\ g(x,y) \end{bmatrix}$$

**Solution via Cramer's rule:**

- Determinant of the Jacobian $\mathtt{det}(\mathbf{J})$:

$$J = \mathtt{det}(\mathbf{J}) = \frac{\partial f}{\partial x}\frac{\partial g}{\partial y} - \frac{\partial f}{\partial y}\frac{\partial g}{\partial x}$$

- Solutions for $\delta x$ and $\delta y$:

$$\delta x = \frac{-f(x,y)\frac{\partial g}{\partial y} + g(x,y)\frac{\partial f}{\partial y}}{J}$$

$$\delta y = \frac{f(x,y)\frac{\partial g}{\partial x} - g(x,y)\frac{\partial f}{\partial x}}{J}$$

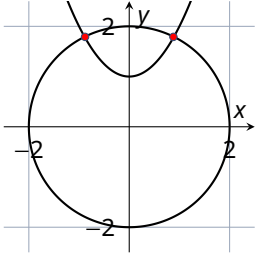
# Newton-Raphson Method: multi-dimensional case

**Example:** *intersection of circle with parabola in matrix form*

$$\begin{aligned} x^2+y^2&=4\\ y&=x^2+1 \end{aligned} \quad \text{can be represented as} \quad \begin{bmatrix}1 & 2\\ 2 & 1\end{bmatrix}\begin{bmatrix}x\\ f(x)\end{bmatrix}=\begin{bmatrix}x-f(x)\\ x^2+f(x^2)-4\end{bmatrix}$$

Iterations for solving:

| $i$ | $\mathbf{x}$ | $f$ | $\mathbf{J}$ | $\delta\mathbf{x}$ |
|---|---|---|---|---|
| 1 | $\begin{bmatrix}1.00\\ 2.00\end{bmatrix}$ | $\begin{bmatrix}1.00\\ 0.00\end{bmatrix}$ | $\begin{bmatrix}2.00 & 4.00\\ 2.00 & -1.00\end{bmatrix}$ | $\begin{bmatrix}-0.1\\ -0.2\end{bmatrix}$ |
| 2 | $\begin{bmatrix}0.90\\ 1.80\end{bmatrix}$ | $\begin{bmatrix}5.00\\ 1.00\end{bmatrix}\times 10^{-2}$ | $\begin{bmatrix}1.80 & 3.60\\ 1.80 & -1.00\end{bmatrix}$ | $\begin{bmatrix}-0.01\\ -8.7\times 10^{-3}\end{bmatrix}$ |
| 3 | $\begin{bmatrix}0.89\\ 1.79\end{bmatrix}$ | $\begin{bmatrix}1.83\\ 0.11\end{bmatrix}\times 10^{-4}$ | $\begin{bmatrix}1.78 & 3.58\\ 1.78 & -1.00\end{bmatrix}$ | $\begin{bmatrix}-6.99\times 10^{-5}\\ -1.65\times 10^{-5}\end{bmatrix}$ |
| 4 | $\begin{bmatrix}0.88\\ 1.79\end{bmatrix}$ | $\begin{bmatrix}5.16\\ 4.89\end{bmatrix}\times 10^{-9}$ | $\begin{bmatrix}1.78 & 3.58\\ 1.78 & -1.00\end{bmatrix}$ | $\begin{bmatrix}-2.78\times 10^{-9}\\ 5.94\times 10^{-11}\end{bmatrix}$ |



# Newton-Raphson Method: multi-dimensional case

**Extensions to multi-dimensional case:**
**Check order of convergence:**

| it | $x_1$ | $x_2$ | eps1 | eps2 | $m_1$ | $m_2$ |
|---|---|---|---|---|---|---|
| 1 | 1.0000 | 2.0000 | | | | |
| 2 | 0.9000 | 1.8000 | 0.1000 | 0.2000 | | |
| 3 | 0.8896 | 1.7913 | 0.0104 | 0.0087 | 1.9835 | 2.9482 |
| 4 | 0.8895 | 1.7913 | 0.0000699 | 0.0000165 | 2.0949 | 2.3208 |
| 5 | 0.8895 | 1.7913 | 0.0000000278 | 0.0000000059 | 2.0589 | 2.1382 |

**Quadratic convergence**

Doubling number of significant digits every iteration

$$m = \frac{\ln(\varepsilon_{n+1}) - \ln(\varepsilon_n)}{\ln(\varepsilon_n) - \ln(\varepsilon_{n-1})}$$

# Newton-Raphson Method

**Deriving the extension to more than two variables**:

1. Generalization to the N-dimensional case
2. Define variables
3. Multi-variate Taylor series expansion
4. Define Jacobian matrix
5. Neglect higher-order terms
6. Express in terms of iterations

$$\boxed{\mathbf{x}_{n+1} = \mathbf{x}_n - \mathbf{J}^{-1}(\mathbf{x}_n) \cdot \mathbf{f}(\mathbf{x}_n)}$$

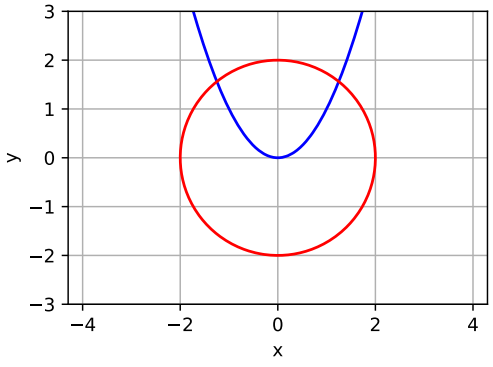
1. $f_i(x_1, x_2, \ldots, x_N) = 0$
2. $\mathbf{x} = [x_1, x_2, \ldots, x_N]\ \mathbf{f} = [f_1, f_2, \ldots, f_N]$
3. $f_i(\mathbf{x} + \delta\mathbf{x}) = f_i(\mathbf{x}) + \sum_{j=1}^{N} \frac{\partial f_i}{\partial x_j} \delta x_j + O(\delta\mathbf{x}^2)$
4. $J_{ij} = \frac{\partial f_i}{\partial x_j}\ f(\mathbf{x} + \delta\mathbf{x}) = f(\mathbf{x}) + \mathbf{J}\delta\mathbf{x} + O(\delta\mathbf{x}^2)$
5. $\mathbf{J} \cdot \delta\mathbf{x} = -\mathbf{f}(\mathbf{x})$

# Newton-Raphson Method

**Multi-variate Newton-Raphson in Python:**

Plotting the functions:

```
plot_implicit_function(lambda x,y: y-x**2, resolution=100, colors="blue")
plot_implicit_function(lambda x,y: y**2+x**2-4, resolution=100, colors="red")
```



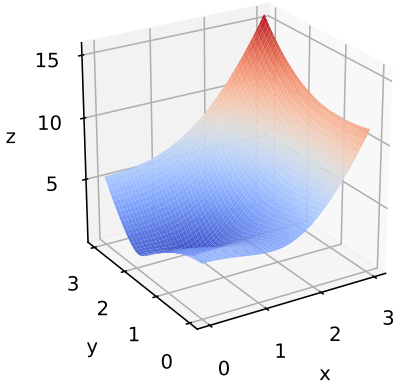

- Code can be found in `plot_implicit.py`
- Uses contour plot at $f(x,y) = 0$

# Newton-Raphson Method

**Multi-variate Newton-Raphson in Python:**

Plotting the norm of the function:

```
plot_surface_function(lambda x,y: np.sqrt((x**2 + y**2 -4)**2+(x**2-y+1)**2),
                      (0,3),(0,3))
```



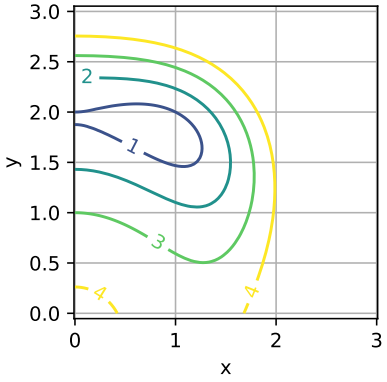

- Code can be found in `plot_implicit.py`
- Uses contour plot at $f(x,y) = 0$

# Newton-Raphson Method

**Multi-variate Newton-Raphson in Python:**

Plotting the norm of the function:

```
plot_contours(lambda x,y: np.sqrt((x**2 + y**2 -4)**2+(x**2-y+1)**2),
              (0, 3), (0, 3), resolution = 100, levels=[0, 1, 2, 3, 4])
```




- Code can be found in `plot_implicit.py`
- Uses contour plot at $f(x,y) = 0$

# Broyden's Method

**Multi-dimensional secant method ('quasi-Newton'):**

- Disadvantage of the Newton-Raphson method:
  - It requires the Jacobian matrix.
  - In many problems, no analytical Jacobian is available.
  - If the function evaluation is expensive, the numerical approximation using finite differences can be prohibitive.
- Solution: Use a cheap approximation of the Jacobian! (Secant or 'quasi-Newton' method)
- Comparison:

Newton-Raphson: $J_{ij}(\mathbf{x}) = \dfrac{\partial f_i}{\partial x_j}(\mathbf{x})$ (Analytical)

Secant method: $\mathbf{J}(\mathbf{x})$ approximated by $\mathbf{B}$ (Numerical)

# Broyden's Method

**Approximating $\mathbf{B}^{n+1}$:**

- Multi-dimensional secant method ('quasi-Newton'):
- Secant equation (generalization of 1D case):

$$\mathbf{B}^{n+1} \cdot \delta\mathbf{x}^n = \delta\mathbf{f}^n \quad \delta\mathbf{x}^n = \mathbf{x}^{n+1} - \mathbf{x}^n \quad \delta\mathbf{f}^n = \mathbf{f}^{n+1} - \mathbf{f}^n$$

- Underdetermined (not unique: -n equations with n unknowns), need another condition to pin down $B^{n+1}$.

**Broyden's method:**

- Determine $\mathbf{B}^{n+1}$ by making the least change to $\mathbf{B}^n$ that is consistent with the secant condition.
- Updating formula:

$$\mathbf{B}^{n+1} = \mathbf{B}^n + \frac{(\delta\mathbf{f}^n - \mathbf{B}^n \cdot \delta\mathbf{x}^n)}{\delta\mathbf{x}^n \cdot \delta\mathbf{x}^n} \otimes \delta\mathbf{x}^n$$

- Note: Sometimes $\delta B_{n-1}$ is updated directly.

# Broyden's Method

**Background of Broyden's method:**

- Secant equation:

$$\mathbf{B}^{n+1} \cdot \delta\mathbf{x}^n = \delta f_n$$

- Since there is no update on derivative info, why would $\mathbf{B}^n$ change in a direction orthogonal to $\delta\mathbf{x}^n$?

$$\Rightarrow (\delta\mathbf{x}^n)^T \delta\mathbf{w} = 0$$

$$\begin{aligned} \mathbf{B}^{n+1} \cdot \mathbf{w} &= \mathbf{B}^n \cdot \mathbf{w} \\ \mathbf{B}^{n+1} \cdot \delta\mathbf{x}^n &= \delta\mathbf{f}^n \end{aligned} \qquad \Rightarrow \qquad \mathbf{B}^{n+1} = \mathbf{B}^n + \frac{(\delta\mathbf{f}^n - \mathbf{B}^n \cdot \delta\mathbf{x}^n)}{\delta\mathbf{x}^n \cdot \delta\mathbf{x}^n} \otimes \delta\mathbf{x}^n$$

- Initialize $\delta\mathbf{x}^n$ and $\mathbf{B}_0$ with the identity matrix (or with finite difference approx.).

# Broyden's Method

- Same example as before but now with Broyden's method.
- Note how the approximate Jacobian (**B**) is updated over subsequent iterations:

$$
\begin{array}{cccc}
\begin{bmatrix} 1. & 0. \\ 0. & 1. \end{bmatrix} \rightarrow & \begin{bmatrix} 3. & -1. \\ 4. & -1. \end{bmatrix} \rightarrow & \begin{bmatrix} -1.0 & -9.0 \\ 3.4 & -2.2 \end{bmatrix} \rightarrow & \begin{bmatrix} -1.062 & -9.260 \\ 3.411 & -2.154 \end{bmatrix} \rightarrow \\
\begin{bmatrix} 5.290 & -3.864 \\ 2.493 & -2.934 \end{bmatrix} \rightarrow & \begin{bmatrix} 7.363 & -1.931 \\ 3.556 & -1.943 \end{bmatrix} \rightarrow & \begin{bmatrix} 2.349 & -0.773 \\ 3.547 & -1.941 \end{bmatrix} \rightarrow & \begin{bmatrix} -0.934 & -6.772 \\ 2.351 & -4.124 \end{bmatrix} \rightarrow \\
\begin{bmatrix} -0.384 & -5.879 \\ 2.500 & -3.884 \end{bmatrix} \rightarrow & \begin{bmatrix} 10.416 & 6.344 \\ 5.947 & 0.018 \end{bmatrix} \rightarrow & \begin{bmatrix} 9.781 & 5.515 \\ 5.641 & -0.382 \end{bmatrix} \rightarrow & \begin{bmatrix} 3.577 & 3.630 \\ 3.362 & -1.074 \end{bmatrix} \rightarrow \\
\begin{bmatrix} 3.116 & 3.238 \\ 2.912 & -1.458 \end{bmatrix} \rightarrow & \begin{bmatrix} 1.992 & 3.272 \\ 1.989 & -1.430 \end{bmatrix} \rightarrow & \dots \rightarrow & \dots \rightarrow
\end{array}
$$

- Compare with analytical jacobian: $\mathbf{B} = \begin{bmatrix} 1.748 & 3.261 \\ 1.736 & -1.439 \end{bmatrix}$ $\mathbf{J} = \begin{bmatrix} 1.779 & 3.583 \\ 1.779 & -1 \end{bmatrix}$
- Note that the approximate Jacobian (**B**) is not exact even when the solution has already been found!

# Conclusions

- Recommendations for root finding:
  - One-dimensional cases:
    - If it is not easy/cheap to compute the function's derivative $\Rightarrow$ use Brent's algorithm.
    - If derivative information is available $\Rightarrow$ use Newton-Raphson's method + bookkeeping on bounds provided you can supply a good enough initial guess!!
    - There are specialized routines for (multiple) root finding of polynomials (but not covered in this course).
  - Multi-dimensional cases:
    - Use Newton-Raphson method, but make sure that you provide an initial guess close enough to achieve convergence.
    - In case derivative information is expensive $\Rightarrow$ use Broyden's method (but slower convergence!).

# Numerical interpolation

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Piecewise constant
- Linear
- Polynomial
- Splines
- Tutorials

# Interpolation problem

## Definition

Given a set of points $x_k$, $k = 0, \ldots, n$, $x_i \neq x_j$ with associated function values $f_k$, $k = 0, \ldots, n$, or simply: $\{x_k, f_k\}_{k=0}^n$. The interpolation problem is defined as: find a polynomial $p_n$ such that this interpolates the values of $f_k$ on the points $x_k$:

$$p_n(x_k) = f_k, \quad k = 0, \ldots, n$$

## Theorem

*The interpolation problem for $\{x_k, f_k\}_{k=0}^n$ has a unique solution when $x_i \neq x_j$ for $i \neq j$. Note that we cannot allow multiple function values $f_k$ for the same value of $x_k$.*

# What is interpolation?

Interpolation means constructing additional data points within the range of, and using, a discrete set of known data points.

It is typically performed on a uniformly spread data set, but this is not strictly necessary for all methods

# Is interpolation the same as curve fitting?

NO

- Curve-fitting requires additionally some way of computing the error between function (curve) and data
- Curve-fitting does not strictly enforce the function to match the data exactly
- Curve-fitting may be done on multiple datapoints at one position
- Curve-fitting is much more expensive to do, requires optimisation

# Why do chemical engineers need interpolation?

- Comparison of two data sets which are given at different positions
  - An experimental data set may have been recorded at a constant rate, but the numerical solution is computed at irregular intervals
- Reconstruction of field values distant of computing nodes
  - A CFD simulation on a regular grid containing structures that are not grid-conformant requires interpolation to the structures
- Calculation of a physical property at a condition between those of a lookup table
  - The viscosity of a substance may have been measured at 20°C and 30°C, but not at the desired 28.5°C

# General

Several important numerical interpolation methods are discussed today:

- Piecewise constant interpolation
- Linear interpolation
  - Bilinear interpolation
- Polynomial interpolation (Newton's method)
- Spline interpolation

# Today's outline

- Introduction
- Piecewise constant
- Linear
- Polynomial
- Splines
- Tutorials

# Today's data set

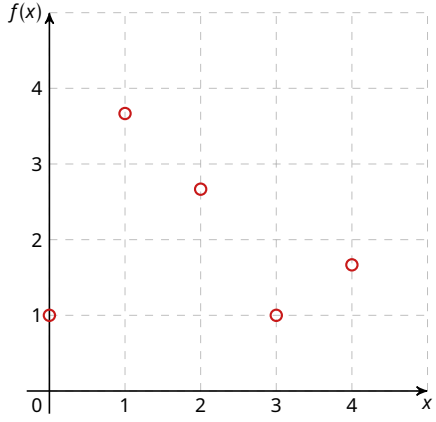
Generate the following data set:

```
import numpy as np
xdata = np.arange(0,6)
fun = lambda x: x**3/2 - (10*x**2)/3 + 11*x
     /2 + 1
ydata = fun(xdata)
```

This yields some sample points on which we base our examples:

| $x_k$ | $f_k$ |
|---|---|
| 0 | 1.00 |
| 1 | $\frac{11}{3} = 3.67$ |
| 2 | $\frac{8}{3} = 2.67$ |
| 3 | 1.00 |
| 4 | $\frac{5}{3} = 1.67$ |
| 5 | $\frac{23}{3} = 7.67$ |

Data set $f_n(x_n)$ represented by ○ at discrete intervals $x_n \in \{0, 5\}$



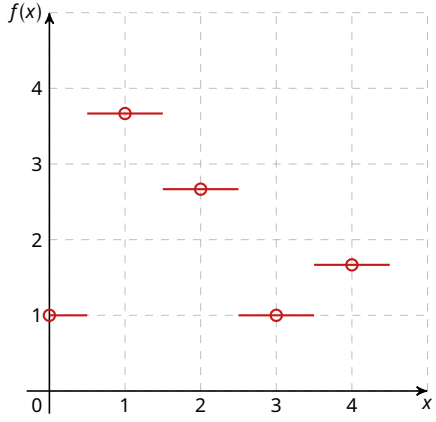
# Piecewise constant interpolation

- Nearest-neighbor interpolation in the continuous range $x \in [0, 5]$
- How to treat the point halfway (e.g. at $x = 2.5$)?

$$
\begin{aligned}
x \in [0, 0.5] &\rightarrow f(x) = f(0) \\
x \in [0.5, 1.5] &\rightarrow f(x) = f(1) \\
x \in [1.5, 2.5] &\rightarrow f(x) = f(2) \\
x \in [2.5, 3.5] &\rightarrow f(x) = f(3) \\
x \in [3.5, 4.5] &\rightarrow f(x) = f(4)
\end{aligned}
$$

- Not often used for simple problems, but e.g. for 2D (Voronoi)

Data set $f_n(x_n)$ represented by O at discrete intervals $x_n \in \{0, 5\}$



# Today's outline

- Introduction
- Piecewise constant
- Linear
- Polynomial
- Splines
- Tutorials

# Linear interpolation

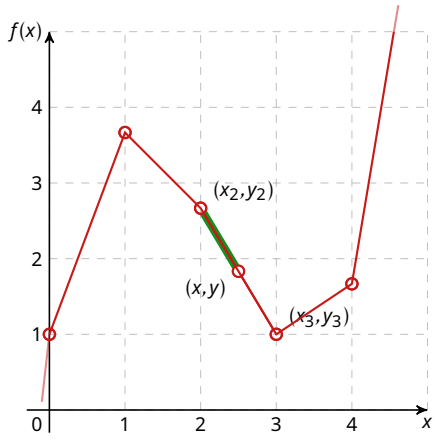
- Linear interpolation to $(x,y)$ between 2 data points $(x_2,y_2)$ and $(x_3,y_3)$:

$$\frac{y-y_2}{x-x_2} = \frac{y_3-y_2}{x_3-x_2}$$

- Reordered, and more formally:

$$y = y_n + (y_{n+1}-y_n)\frac{x-x_n}{x_{n+1}-x_n}$$

Data set $f_n(x_n)$ represented by ○ at discrete intervals $x_n \in \{0,5\}$



# Linear interpolation

- While linear interpolation is fast, and relatively easy to program, it is not very accurate
- At the nodes, the derivatives are discontinuous i.e. not differentiable
- Error is proportional to the square of the distance between nodes

# Interpolation in Python

Interpolation can be done using the SciPy interpolation submodule, e.g.:

```
from scipy.interpolate import interp1d
f = interp1d(xdata, ydata, kind='linear')
```

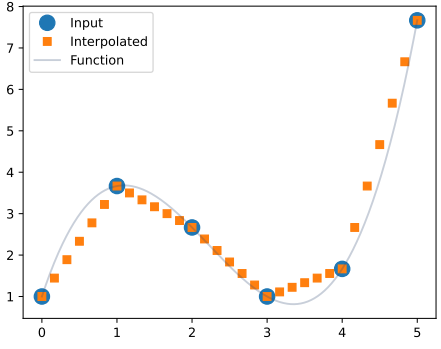
This creates a function object `f` based on the given data.

```
from scipy.interpolate import interp1d
import numpy as np

fun = lambda x: x**3/2 - (10*x**2)/3 + 11*x/2 + 1
xdata = np.arange(0,6)
ydata = fun(xdata)

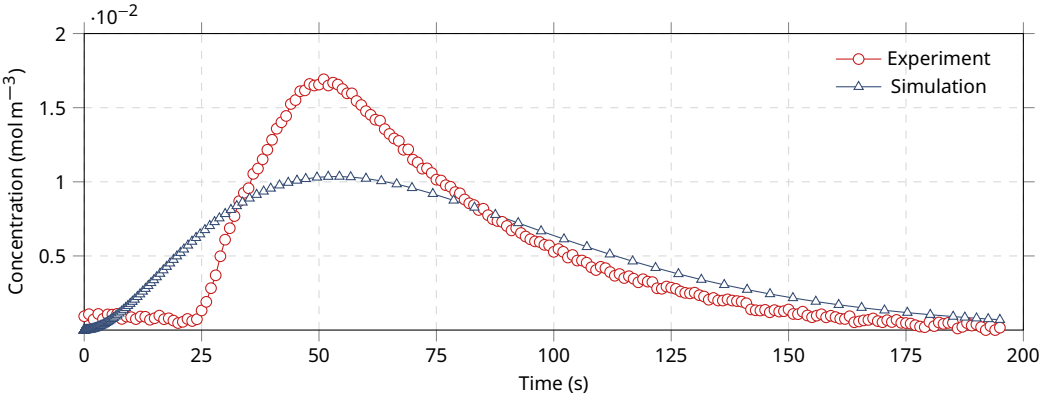
f = interp1d(xdata,ydata)
xint = np.linspace(0,5,31)
yint = f(xint)
```

```
import matplotlib.pyplot as plt
plt.plot(xdata,ydata,'o',markersize=12,label='
    Input')
plt.plot(xint,yint,'s',label='Interpolated')
```



# Example: Linear interpolation in Python

Consider the data sets in `exp_data.txt` and `sim_data.txt`, containing a normalized concentration and time vector for an experiment and a simulation. The simulation was performed with adaptive node distance to save computation time, thus the concentration is not known at the same times. We are not able to compare yet.



# Example: Linear interpolation in Python

Consider the data sets in `exp_data.txt` and `sim_data.txt`, containing a normalized concentration and time vector for an experiment and a simulation. The simulation was performed with adaptive node distance to save computation time, thus the concentration is not known at the same times. We are not able to compare yet.

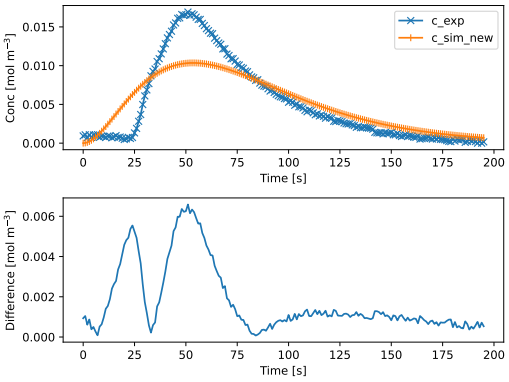
```python
import numpy as np
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

t_sim, c_sim = np.loadtxt("scripts/interpolation/sim_data.txt").T
t_exp, c_exp = np.loadtxt("scripts/interpolation/exp_data.txt").T

# Linear interpolation
f = interp1d(t_sim, c_sim)
diff = np.abs(c_exp - f(t_exp))

# Plot the solution
plt.subplot(2, 1, 1)
plt.plot(t_exp, c_exp, '-x', label='c_exp')
plt.plot(t_exp, f(t_exp), '-|', label='c_sim_new')
plt.xlabel('Time [s]'); plt.ylabel('Conc [mol m$^{-3}$]')
plt.legend()

plt.subplot(2, 1, 2)
plt.plot(t_exp, diff)
plt.xlabel('Time [s]'); plt.ylabel('Difference [mol m$^{-3}$]')
plt.tight_layout()
# plt.show()
plt.savefig('figures/sim_exp_data_interp.pdf')
```



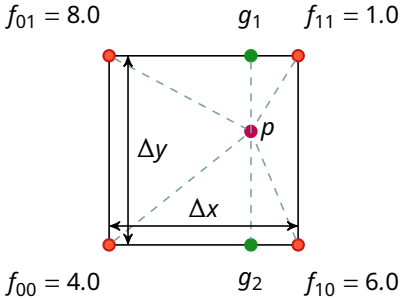
# Bi-linear interpolation

When a 2D field of some quantity is known, we can interpolate the solution to an arbitrary position in the 2D domain $p(x,y)$ using 4 field values $f_{00}, f_{10}, f_{01}$ and $f_{11}$.

$$
\begin{aligned}
g_1 &= f_{01}\frac{x_1 - x}{x_1 - x_0} + f_{11}\frac{x - x_0}{x_1 - x_0} \\
&= f_{01}\frac{x_1 - x}{\Delta x} + f_{11}\frac{x - x_0}{\Delta x}
\end{aligned}
$$

$$
g_2 = f_{00}\frac{x_1 - x}{\Delta x} + f_{10}\frac{x - x_0}{\Delta x}
$$

$$
p = g_2\frac{y_1 - y}{\Delta y} + g_1\frac{y - y_0}{\Delta y}
$$

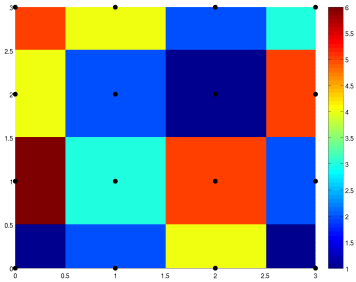



- The order of interpolation ($x$ or $y$ direction first) does not matter; the results are equal

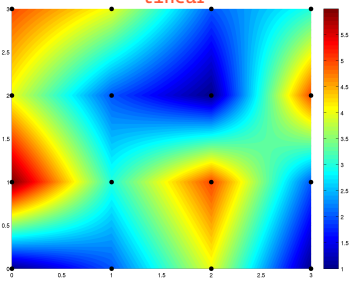
# Higher-dimensional field interpolation in Python

2D or higher-dimensional fields of data can be interpolated in Python using the `scipy.interpolate.interp2d`, `scipy.interpolate.interp3d`, or even `scipy.interpolate.RegularGridInterpolator` functions. The method can be adjusted:

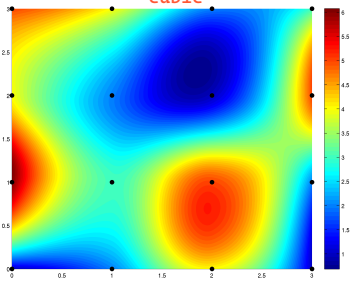
'nearest'



'linear'



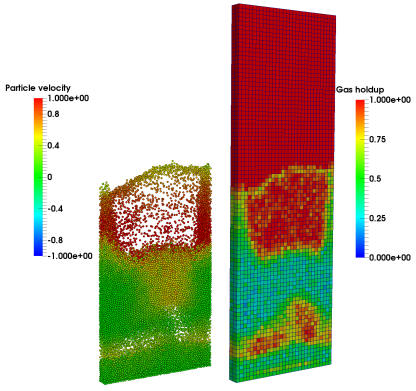
'cubic'



- Also consider tri-linear interpolation (for 3D fields) with `scipy.interpolate.LinearNDInterpolator`, or bicubic interpolation (2D, but third order) with `scipy.interpolate.interp2d`.

# A practical example

Field interpolation is used in e.g. CFD simulations, e.g. a fluidized bed simulation using a *discrete particle model*, where particles are found in between the grid nodes used for velocity computation.



# Today's outline

- Introduction
- Piecewise constant
- Linear
- Polynomial
- Splines
- Tutorials

# Polynomial interpolation

The examples that we have seen, are simplified forms of *Newton polynomials*. We can interpolate our data with a polynomial of degree $n$:

$$p_n(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_2 x^2 + a_1 x + a_0$$

# Polynomial interpolation via Vandermonde matrix

Consider the data points $(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$, the Vandermonde matrix $V$, coefficient vector $a$ and function value vector $y$:

$$V_{m,n} = \begin{pmatrix} x_1^0 & x_1^1 & x_1^2 & \cdots & x_1^{n-1} \\ x_2^0 & x_2^1 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_m^0 & x_m^1 & x_m^2 & \cdots & x_m^{n-1} \end{pmatrix} \quad a = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

The coefficients of a polynomial through the data are obtained by solving the linear system $Va = y$.

```
import numpy as np
x = np.array([0, 1, 2])
y = np.array([1.0000, 3.6667, 2.6667])
V = np.vander(x, increasing=True)
print(V)
```

```
[ 1. 4.50005 -1.83335]
```

So we found the equation:

$$p_2(x) = -1.8333x^2 + 4.5x + 1$$

These Vandermonde-systems are often *ill-conditioned*, so we need another, more stable, method!

# Construction of Newton polynomials

Formally, the polynomials $p_n(x)$ are described using prefactors $f[x_0,\ldots,x_k]$ and polynomial terms $w_m(x)$:

$$p_n(x) = \sum_{k=0}^{n} f[x_0,\ldots,x_k] w_k(x)$$

The polynomial terms are computed via:

$$w_0(x) = 1,\ w_1(x) = (x - x_0),\ w_2(x) = (x - x_0)\cdot(x - x_1),$$
$$w_m(x) = (x - x_0)\cdot(x - x_1)\cdots(x - x_{m-1}) = w_{m-1}\cdot(x - x_{m-1})$$
$$w_m(x) = \prod_{j=0}^{m-1}(x - x_j), \qquad m = 0,\ldots,n$$

The prefactors are *forward divided differences*, which can be computed as:

$$f[x_{x-k},\ldots,x_r] \equiv \frac{f[x_{r-k+1},\ldots,x_r] - f[x_{r-k},\ldots,x_{r-1}]}{x_r - x_{r-k}}$$

# Construction of Newton polynomials: example

Sample data

| $x_k$ | $f_k$ |
|---|---|
| 0 | 1.00 |
| 1 | $\frac{11}{3} = 3.67$ |
| 2 | $\frac{8}{3} = 2.67$ |

$$p_n(x) = \sum_{k=0}^{n} f[x_0,\ldots,x_k] w_k(x)$$

$$f[x_{x-k},\ldots,x_r] \equiv \frac{f[x_{r-k+1},\ldots,x_r] - f[x_{r-k},\ldots,x_{r-1}]}{x_r - x_{r-k}}$$

$$w_m(x) = \prod_{j=0}^{m-1} (x - x_j)$$

| $x_k$ | $f_k$ | | |
|---|---|---|---|
| $x_0$ | $f[x_0] = f_0$ | | |
| $x_1$ | $f[x_1] = f_1$ | $f[x_0,x_1] = \frac{f_1 - f_0}{x_1 - x_0}$ | |
| $x_2$ | $f[x_2] = f_2$ | $f[x_1,x_2] = \frac{f_2 - f_1}{x_2 - x_1}$ | $f[x_0,x_1,x_2] = \frac{f[x_1,x_2] - f[x_0,x_1]}{x_2 - x_0}$ |

| $x_k$ | $f_k$ | | |
|---|---|---|---|
| 0 | 1 | | |
| 1 | 3.67 | $\frac{\frac{11}{3} - 1}{1 - 0} = \frac{8}{3}$ | |
| 2 | 2.67 | $\frac{\frac{8}{3} - \frac{11}{3}}{2 - 1} = \frac{-1}{1} = -1$ | $\frac{(-1) - \frac{8}{3}}{2 - 0} = -\frac{11}{6}$ |

# Construction of Newton polynomials: example

Sample data

| $x_k$ | $f_k$ |
|---|---|
| 0 | 1.00 |
| 1 | $\frac{11}{3} = 3.67$ |
| 2 | $\frac{8}{3} = 2.67$ |

$p_n(x) = \sum_{k=0}^{n} f[x_0,\ldots,x_k] w_k(x)$

$f[x_{x-k},\ldots,x_r] \equiv \frac{f[x_{r-k+1},\ldots,x_r] - f[x_{r-k},\ldots,x_{r-1}]}{x_r - x_{r-k}}$

$w_m(x) = \prod_{j=0}^{m-1} (x - x_j)$

| $x_k$ | $f_k$ | | |
|---|---|---|---|
| 0 | 1 | | |
| 1 | 3.67 | $\frac{\frac{11}{3}-1}{1-0} = \frac{8}{3}$ | |
| 2 | 2.67 | $\frac{\frac{8}{3}-\frac{11}{3}}{2-1} = \frac{-1}{1} = -1$ | $\frac{(-1)-\frac{8}{3}}{2-0} = -\frac{11}{6}$ |

$$p_2(x) = 1 \cdot w_m(0) + \frac{8}{3} \cdot w_m(1) + \left(-\frac{11}{6}\right) \cdot w_m(2)$$

$$= 1 \cdot 1 + \frac{8}{3} \cdot (x-0) + \left(-\frac{11}{6}\right) \cdot (x-0)(x-1) = -\frac{11}{6}x^2 + 4\frac{1}{2}x + 1$$

# Construction of Newton polynomials: example

For each three points, a new polynomial interpolant can be derived:

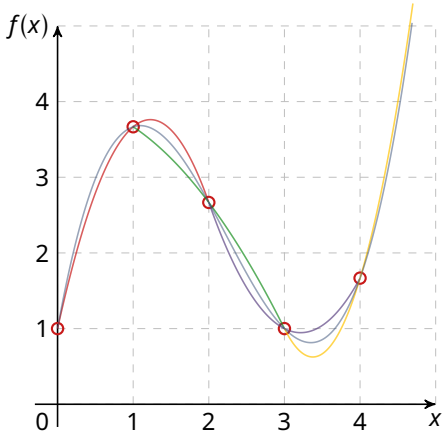
$$p_2(x) = -\frac{11}{6}x^2 + 4\frac{1}{2}x + 1$$

$$p_2(x) = 4 - \frac{x^2}{3}$$

$$p_2(x) = \frac{7x^2}{6} - 7\frac{1}{2}x + 13$$

$$p_2(x) = \frac{8}{3}x^2 - 18x + 31$$

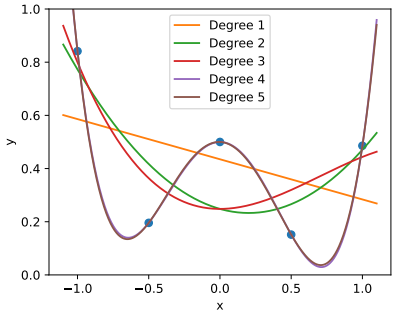
$$f(x) = \frac{x^3}{2} - \frac{10x^2}{3} + \frac{11x}{2} + 1$$



# Polynomial fitting in Python: example

Develop the polynomials $p_1(x)$ through $p_5(x)$ using the following data set:

```
import numpy as np
import matplotlib.pyplot as plt
xdata = np.arange(-1,1.5,0.5)
ydata = [x * np.sin(x)/np.sqrt(x+2) if x != 0 else 0.5 for x in xdata]
plt.plot(xdata,ydata,'o')
```




```
xc = np.linspace(-1.1,1.1,1001,endpoint=True)
for deg in range(1,6):
    # Fit coefficients
    p_coeffs = np.polyfit(xdata,ydata,deg)
    # Compute function values
    y = np.polyval(p_coeffs,xc)
    # Plot
    plt.plot(xc,y,label=f'Degree {deg}')
```

```
RankWarning: Polyfit may be poorly conditioned
```

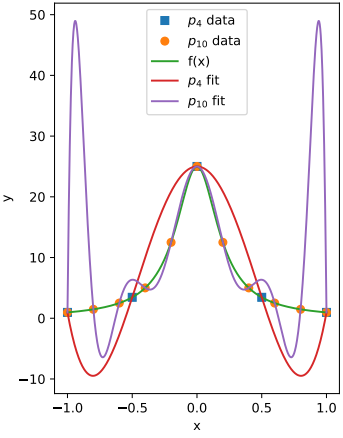
# Exercise

Develop the $p_4(x)$ and $p_{10}(x)$ interpolants from the following data sets:

$$f(x) = \frac{1}{x^2 + \frac{1}{25}} \qquad x \in [-1, 1]$$

```
import numpy as np
import matplotlib.pyplot as plt
f = lambda x: 1/(x**2 + 1/25)
x4,x10,xinf = [np.linspace(-1, 1, n) for n in [5,11,1001]]
y4,y10,yinf = f(x4), f(x10), f(xinf)
```

```
# Get coefficients for 4th and 10th order polynomial
p4 = np.polyfit(x4, y4, 4)
p10 = np.polyfit(x10, y10, 10)
# Compute function values using fitted coeffs
yinf4 = np.polyval(p4, xinf)
yinf10 = np.polyval(p10, xinf)
```



# Final thoughts on polynomial interpolation

- An polynomial interpolant of order $n$ requires $n + 1$ data points
  - More data points: interpolant does *not always* cross the points
  - Fewer data points: interpolant is not unique
- Higher-degree polynomials at equidistant points may cause strong oscillatory behaviour (Runge's phenomenon)
  - Mitigation of the problem on Chebyshev (i.e. non uniform grid)...
  - ... or by performing piecewise interpolation (next topic)
- Python functions `np.polyfit(x,y,n)` and `np.polyval(p,x_new)` were demonstrated.

# Today's outline


- Introduction
- Piecewise constant
- Linear
- Polynomial
- Splines
- Tutorials

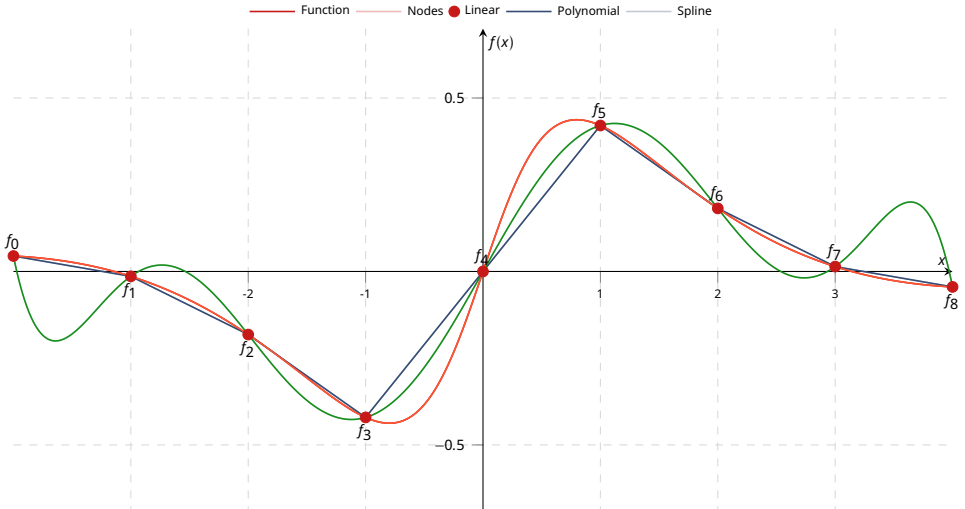
# Spline interpolation

A spline is a numerical function that represents a smooth, higher order, piecewise polynomial interpolants of a data set.

- Smooth: the interpolant is continuous in the first and second derivatives
- Higher order: The most common type of splines uses third-order polynomials (cubic splines)
- Piecewise polynomial: The interpolant is constructed between each two consecutive tabulated points

# Splines: comparison to other interpolation techniques

Interpolation of $f(x) = \dfrac{\sin x}{1+x^2}$



# Spline interpolation in Python

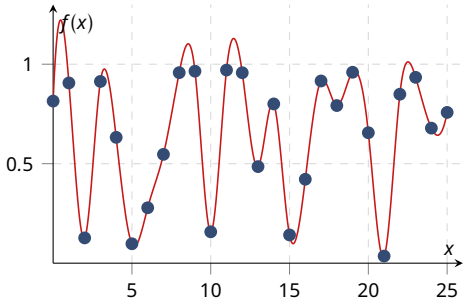
We can generate a random data set, and interpolate using `scipy.interpolate.interp1d`:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import make_interp_spline

# Generate random data set
xdata = np.arange(0, 26)
ydata = np.random.rand(len(xdata))

# Interpolant on a fine mesh
xc = np.linspace(0, 25, 1001)
ifun = make_interp_spline(xdata, ydata)
yc = ifun(xc)

# Plot the data
plt.plot(xdata, ydata, 'o')
plt.plot(xc, yc, '-r')
plt.show()
```




Note: The SciPy Optimize module contains various interpolation methods with a similar interface.

# Summary

- Interpolation is used to obtain data between existing data points
  - (Bi-)Linear, polynomial and spline interpolation methods
  - Construction of Newton polynomials
  - Oscillations of high-order polynomials
- Legendre polynomials: alternative way of performing the polynomial interpolation (not discussed here)

# Interpolation tutorials

1. In Python, generate the data:

```
x = np.arange(-4, 6, 1)
y = [0, 0, 0, 1, 1, 1, 0, 0, 0, 0]
```

   Interpolate the data using polynomial interpolation (which order do you use?) and a spline. Plot the results together with the original data in a graph.

2. Do the same exercise for the following data. Can you explain your observations?

```
t = [0, 0.1, 0.499, 0.5, 0.6, 1.0, 1.4, 1.5, 1.899, 1.9, 2.0]
y = [0, 0.06, 0.17, 0.19, 0.21, 0.26, 0.29, 0.29, 0.30, 0.31, 0.31]
```

**Hint:** Use `scipy.interpolate.interp1d(...,kind="...")` to use different splines.

# Numerical integration

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Riemann integrals
- Trapezoid rule
- Simpson's rule
- Conclusion
- Tutorials

# What is numerical integration?

To determine the integral $I(x)$ of an integrand $f(x)$, which can be used to compute the area underneath the integrand between $x = a$ and $x = b$.

$$I(x) = \int_a^b f(x)dx$$

Today we will outline different numerical integration methods.

- Riemann integrals
- Trapezoidal rule
- Simpson's rule

# Why do chemical engineers need integration?

- Obtaining the cumulative particle size distribution from a particle size distribution
- The concentration outflow over time may be integrated to yield the residence time distribution
- Integration of a varying product outflow yields the total product outflow
- Quantitative analysis of mixture components via e.g. GC/MS
- Not all function have an explicit antiderivative, e.g. $\int e^{x^2} dx$ or $\int \frac{1}{\ln x} dx$

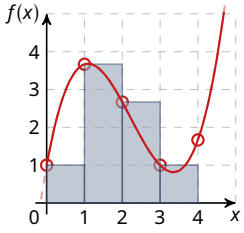
# Today's outline


- Introduction
- Riemann integrals
- Trapezoid rule
- Simpson's rule
- Conclusion
- Tutorials

# Riemann integrals

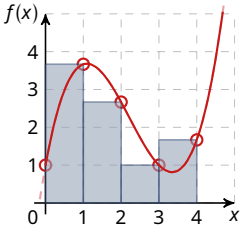
Basic idea: Subdivide the interval $[a, b]$ into $n$ subintervals of equal length $\Delta x = \frac{b-a}{n}$ and use the sum of area to approximate the integral.

Left endpoint rule



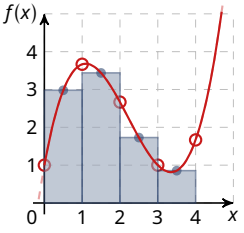

$$L_n = \sum_{i=0}^{n-1} f(x_i)\Delta x_i$$

Right endpoint rule




$$R_n = \sum_{i=0}^{n-1} f(x_{i+1})\Delta x_i$$

Midpoint rule




$$M_n = \sum_{i=0}^{n-1} f(\bar{x}_i)\Delta x_i$$

with $\bar{x}_i = \frac{x_i + x_{i+1}}{2}$

# Errors in Riemann integrals

We define the exact integral as $I = \int_a^b f(x)dx$, and $L_n$, $R_n$ and $M_n$ represent the left, right and midpoint rule approximations of $I$ based on $n$ intervals.

Writing $f^{(k)}_{\max}$ for the maximum value of the $k$-th derivative, the upper-bounds of the errors by Riemann integrals are:

- $|I - L_n| \leq \frac{f^{(1)}_{\max}(b-a)^2}{2n}$
- $|I - R_n| \leq \frac{f^{(1)}_{\max}(b-a)^2}{2n}$
- $|I - M_n| \leq \frac{f^{(2)}_{\max}(b-a)^3}{24n^2}$

Note that while $|I - L_n|$ and $|I - R_n|$ give the same *upper-bounds* of the error, this does not mean the same error. Rather, the error is of opposite sign!

# Today's outline


- Introduction
- Riemann integrals
- Trapezoid rule
- Simpson's rule
- Conclusion
- Tutorials

# Trapezoid rule

Since the sign of the approximation error of the left and right endpoint rules is opposite, we can take the average of these approximations:

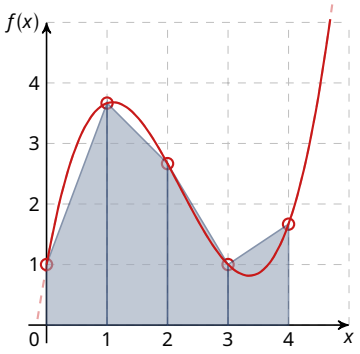
$$T_n = \frac{L_n + R_n}{2}$$

The total area is obtained by geometric reconstruction of trapezoids:

$$T_n = \sum_{i=0}^{n-1} \frac{f(x_{i+1}) + f(x_i)}{2} \Delta x_i$$

Note that this can be rewritten for equidistant intervals:

$$T_n = \frac{b-a}{2n} \left( f(x_0) + 2f(x_1) + \ldots + 2f(x_{n-1}) + f(x_n) \right)$$



# Error in trapezoid integration

The trapezoid rule result over $n$ intervals $T_n$ approximates the exact integral $I = \int_a^b f(x)dx$. The upper-bounds of the error is given as:

$$|I - T_n| \leq \frac{f^{(2)}_{\max}(b-a)^3}{12n^2}$$

Recall that the midpoint rule approximates with an upper-bound error of

$$|I - M_n| \leq \frac{f^{(2)}_{\max}(b-a)^3}{24n^2}$$

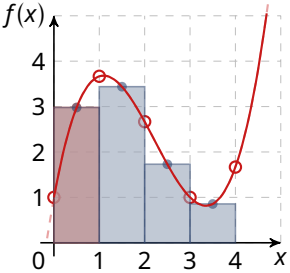
The midpoint rule approximation has lower error bounds than the trapezoid rule. A linear function is, however, better approximated by the trapezoid rule.

# Today's outline

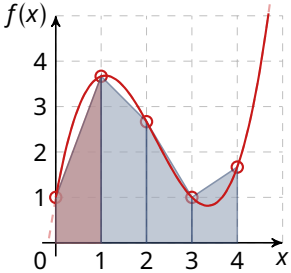

- Introduction
- Riemann integrals
- Trapezoid rule
- Simpson's rule
- Conclusion
- Tutorials

# Towards higher-order integration

Compare how the midpoint and trapezoid functions behave on convex and concave parts of a graph.




Midpoint rule




Trapezoid rule

In convex parts (bending down), the midpoint rule tends to overestimate the integral (trapezoid underestimates). In concave parts (bending up), the midpoint rule tends to underestimate the integral (trapezoid overestimates).

# Towards higher-order integration

The errors of the midpoint rule and trapezoid rule behave in a similar way, but have opposite signs.

- Midpoint: $|I - M_n| \leq \dfrac{f^{(2)}_{\max}(b-a)^3}{24n^2}$
- Trapezoid: $|I - T_n| \leq \dfrac{f^{(2)}_{\max}(b-a)^3}{12n^2}$

For a quadratic function, the errors relate as:

$$|I - M_n| = \frac{1}{2}|I - T_n|$$

Taking the weighted average of these two yields the Simpson's rule:

$$S_{2n} = \frac{2}{3}M_n + \frac{1}{3}T_n$$

The $2n$ means we have $2n$ subintervals: the $n$ trapezoid intervals are subdivided by the midpoint rule.

# Simpson's rule

Consider the interval $i \in [x_0, x_2]$, subdivided in three equidistant interpolation points: $x_0, x_1, x_2$.

- Midpoint: $M_i = f(\frac{x_0 + x_2}{2})2\Delta x = f(x_1)2\Delta x$
- Trapezoid: $T_i = \frac{f(x_0) + f(x_2)}{2}2\Delta x$
- Simpson: $S_i = \frac{2}{3}M_i + \frac{1}{3}T_i$

Note that $M_i$ and $T_i$ were computed on interval $x_2 - x_0 = 2\Delta x$.

Now we have:

$$S_i = \frac{2}{3}\left[f(x_1)2\Delta x\right] + \frac{1}{3}\left[\frac{f(x_0) + f(x_2)}{2}2\Delta x\right]$$
$$= \frac{4\Delta x}{3}f(x_1) + \frac{\Delta x}{3}f(x_0) + f(x_2) = \frac{\Delta x}{3}\left(f(x_0) + 4f(x_1) + f(x_2)\right)$$

# Simpson's rule

We write $f(x_k) = f_k$. The integral of an interval $i \in [x_0, x_2]$ is approximated as:

$$S_i = \frac{\Delta x}{3}(f_0 + 4f_1 + f_2)$$

The next interval, $S_j$ with $j \in [x_2, x_4]$ with midpoint $x_3 = \frac{x_2+x_4}{2}$ is approximated as:

$$S_j = \frac{\Delta x}{3}(f_2 + 4f_3 + f_4)$$

If we sum these two intervals we obtain:

$$\begin{aligned} I \approx S_i + S_j &= \left[\frac{\Delta x}{3}(f_0 + 4f_1 + f_2)\right] + \left[\frac{\Delta x}{3}(f_2 + 4f_3 + f_4)\right] \\ &= \frac{\Delta x}{3}(f_0 + 4f_1 + 2f_2 + 4f_3 + f4) \end{aligned}$$

# Simpson's rule

In general, Simpson's rule can be written as:

$$\int_a^b f(x)dx \approx \sum_{\substack{k=2 \\ k \text{ even}}}^{n} \frac{\Delta x}{3}(f_{k-2} + 4f_{k-1} + f_k)$$

$$= \frac{\Delta x}{3}(f_0 + 4f_1 + 2f_2 + 4f_3 + 2f_4 + \ldots + 2f_{n-2} + 4f_{n-1} + f_n)$$

The error is given by:

$$|I - S_n| \leq \frac{f^{(4)}_{\max}(b-a)^5}{180n^4}$$

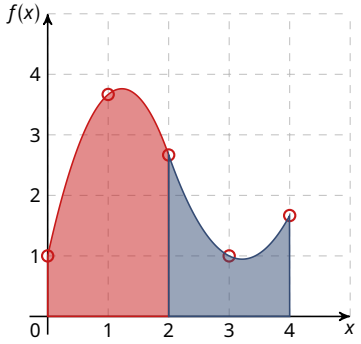
if integrand $f$ is differentiable on $[a, b]$.

# Simpson's rule: example

Recall our example data, described by $f(x) = \frac{x^3}{2} - \frac{10x^2}{3} + \frac{11x}{2} + 1$

$I = \int_0^4 \frac{x^3}{2} - \frac{10x^2}{3} + \frac{11x}{2} + 1 = \frac{80}{9} \approx 8.888\ldots$

- Interpolating $x_0$, $x_1$ and $x_2$: $p_{2a}(x) = -\frac{11}{6}x^2 + 4\frac{1}{2}x + 1$
  $\int_0^2 p_{2a} = \frac{55}{9} \approx 6.1111$
- Interpolating $x_2$, $x_3$ and $x_4$: $p_{2b}(x) = \frac{7x^2}{6} - 7\frac{1}{2}x + 13$
  $\int_2^4 p_{2b} = \frac{25}{9} \approx 2.777\ldots$
- Adding the separate integrals:
  $\int_0^2 p_{2a} + \int_2^4 p_{2b} = \frac{80}{9}$




Using Simpson's rule:

$I \approx \frac{\Delta x}{3}(f_0 + 4f_1 + 2f_2 + 4f_3 + f_4) = \frac{1}{3}(1 + 4 \cdot 3.6667 + 2 \cdot 2.6667 + 4 \cdot 1.0000 + 1.6667) = 8.88888 = \frac{80}{9}$

Simpson's method is of fourth order, and it gives exact approximations of third order polynomials!

# Integration in Python

Integration can be done numerically in Python.

- `np.trapz(y, x)` uses the trapezoid rule to integrate the data. Make sure you use the `x` variable if your data is not spaced with $\Delta x = 1$. Can handle non-equidistant data.

```
import numpy as np
x = np.linspace(-2, 2, 2001)
y = 1 / (x**2 + 1)
I = np.trapz(y, x) # Or: scipy.integrate.trapezoid
print(I)
```

```
2.214297328921525
```

- Integration of functions can be done using the `quad(func, a, b)` function:

```
import numpy as np
from scipy.integrate import quad
f = lambda x: np.exp(-x**2)
I, err = quad(f, 0, 10)
print(I, err)
```

```
0.886226925452758 1.8483380528941764e-13
```

# Today's outline


- Introduction
- Riemann integrals
- Trapezoid rule
- Simpson's rule
- Conclusion
- Tutorials

# What hasn't been discussed?

This course is by no means complete, and further reading is possible.

- Gaussian quadrature: A third-order integration method that requires only two base points (in contrast to the third order Simpson's method, which requires three points)
- Adaptive techniques: Parts of a function that are relatively steady (no wild oscillations) and differentiable can be integrated with much larger step sizes than other parts of the function.
- Simpson's 3/8-rule: Yet another integration technique, requiring an additional data point

# Summary

- Several techniques for numerical integration were discussed:
  - Riemann sums, trapezoid rule, Simpson's rule
  - Upper-bound errors were given for each technique
  - Built-in Python functions were illustrated
- Continue with characterization of convergence of the integration methods in the tutorials!

# Integration tutorials

1. Implement a function to integrate a mathematical function for a specific number of integration intervals. Implement it as a function, which can be called with arguments:
   - Function (handle) to integrate
   - Integration boundaries (as separate arguments or as a $2 \times 1$ numpy array)
   - Number of integration intervals

   For instance: `def leftrule(func, x0, x1, N):.`
2. Set up a function to integrate:

```
1 def myfunction(x):
2     return x**2 - 4*x + 6 + np.sin(5*x)
```

3. Integrate the function, e.g. `int_left = leftrule(myfunction, 0, 10, 25)`
4. Assess how the number of intervals affects the deviation from the true integral value.
5. Create a log-log plot of the deviation vs. number of intervals used.
6. Do this for all methods discussed[2] and compare their performance in a graph

[2] Riemann left, right, midpoint, trapezoid, and Simpson

# Ordinary differential equations 1

## Explicit techniques for ODEs

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline

- Introduction
- Euler's method
  - Forward Euler
- Rates of convergence
- Runge-Kutta methods
  - RK2 methods
  - RK4 method
- Step size control
- Solving ODEs in Python

# Overview

## Ordinary differential equations

An equation containing a function of one independent variable and its derivatives, in contrast to a *partial differential equation*, which contains derivatives with respect to more independent variables.

## Main question

How to solve

$$\frac{d\boldsymbol{y}}{dx} = f(\boldsymbol{y}(x), x) \quad \text{with} \quad \boldsymbol{y}(x = 0) = \boldsymbol{y}_0$$

accurately and efficiently?

# What is an ODE?

- Algebraic equation:

$$f(y(x), x) = 0 \qquad \text{e.g. } -\ln(K_{eq}) = (1 - \zeta)$$

- First order ODE:

$$f\left(\frac{dy}{dx}(x), y(x), x\right) = 0 \quad \text{e.g. } \frac{dc}{dt} = -kc^n$$

- Second order ODE:

$$f\left(\frac{d^2y}{dx^2}(x), \frac{dy}{dx}(x), y(x), x\right) = 0 \quad \text{e.g. } \mathcal{D}\frac{d^2c}{dx^2} = -\frac{kc}{1 + Kc}$$

# About second order ODEs

Very often a second order ODE can be rewritten into a system of first order ODEs (whether it is handy depends on the boundary conditions!)

## More general

Consider the second order ODE:

$$\frac{d^2y}{dx^2} + q(x)\frac{dy}{dx} = r(x)$$

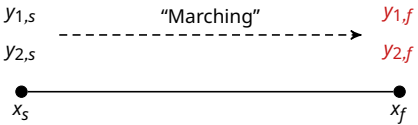
Now define and solve using $z$ as a new variable:

$$\frac{dy}{dx} = z(x)$$
$$\frac{dz}{dx} = r(x) - q(x)z(x)$$

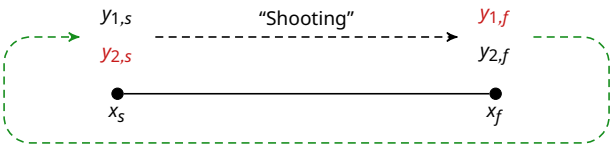
# Importance of boundary conditions

The nature of boundary conditions determines the appropriate numerical method. Classification into 2 main categories:

- *Initial value problems (IVP)*
  We know the values of all $y_i$ at some starting position $x_s$, and it is desired to find the values of $y_i$ at some final point $x_f$.




- *Boundary value problems (BVP)*
  Boundary conditions are specified at more than one $x$. Typically, some of the BC are specified at $x_s$ and the remainder at $x_f$.



# Overview

Initial value problems:

- Explicit methods
  - First order: forward Euler
  - Second order: improved Euler (RK2)
  - Fourth order: Runge-Kutta 4 (RK4)
  - Step size control
- Implicit methods
  - First order: backward Euler
  - Second order: midpoint rule

Boundary value problems

- Shooting method

# Today's outline


- Introduction
- Euler's method
  - Forward Euler
- Rates of convergence
- Runge-Kutta methods
  - RK2 methods
  - RK4 method
- Step size control
- Solving ODEs in Python

# Euler's method

Consider the following single initial value problem:

$$\frac{dc}{dt} = f(c(t), t) \quad \text{with} \quad c(t=0) = c_0 \quad \text{(initial value problem)}$$

Easiest solution algorithm: Euler's method, derived here via Taylor series expansion:

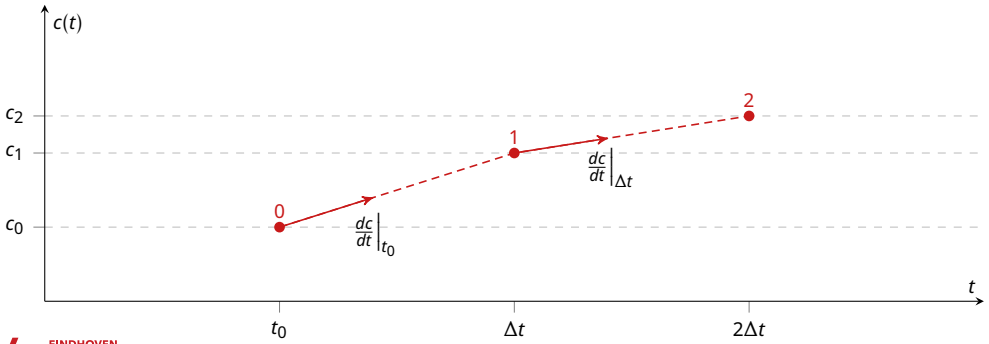
$$c(t_0 + \Delta t) \approx c(t_0) + \left.\frac{dc}{dt}\right|_{t_0} \Delta t + \frac{1}{2} \left.\frac{d^2c}{dt^2}\right|_{t_0} (\Delta t)^2 + \mathcal{O}(\Delta t^3)$$

Neglect terms with higher order than two: $\left.\frac{dc}{dt}\right|_{t_0} = \frac{c(t_0+\Delta t)-c(t_0)}{\Delta t}$ Substitution:

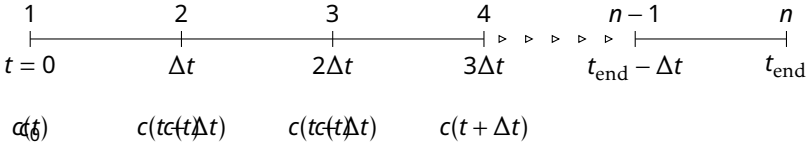
$$\frac{c(t_0 + \Delta t) - c(t_0)}{\Delta t} = f(c_0, t_0) \Rightarrow c(t_0 + \Delta t) = c(t_0) + \Delta t f(c_0, t_0)$$

# Euler's method: graphical example

$$\frac{c(t_0 + \Delta t) - c(t_0)}{\Delta t} = f(c_0, t_0) \Rightarrow c(t_0 + \Delta t) = c(t_0) + \Delta t f(c_0, t_0)$$



# Euler's method - solution method




Start with $t = t_0$, $c = c_0$, then calculate at discrete points in time:
$c(t_1 = t_0 + \Delta t) = c(t_0) + \Delta t f(c_0, t_0)$.

Pseudo-code Euler's method: $\frac{dy}{dx} = f(x,y)$ and $y(x_0) = y_0$.

1. Initialize variables, functions; set $h = \frac{x_1 - x_0}{N}$
2. Set $x = x_0$, $y = y_0$
3. While $x < x_{\text{end}}$ do
   $x_{i+1} = x_i + h$; $y_{i+1} = y_i + h f(x_i, y_i)$

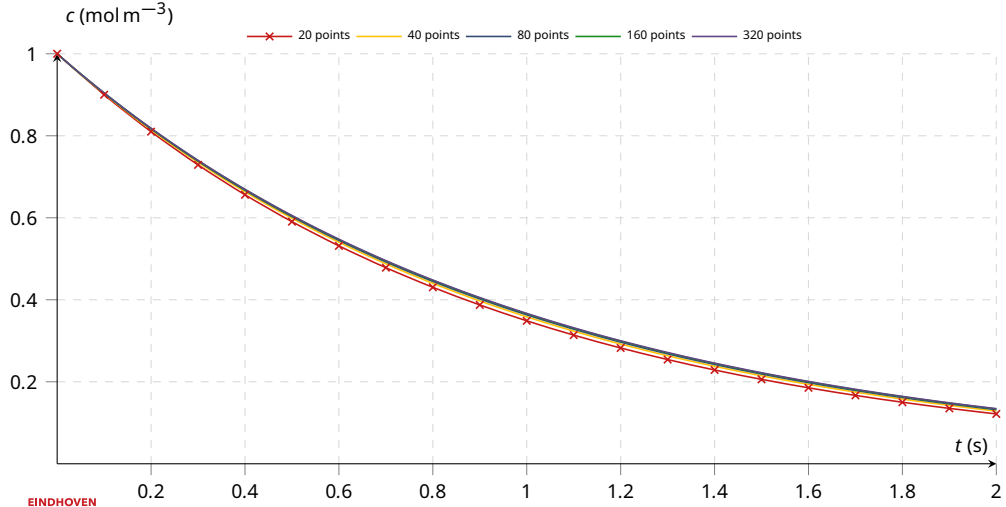
# Euler's method - example

First order reaction in a batch reactor:

$$\frac{dc}{dt} = -kc \quad \text{with} \quad c(t=0) = 1\,\text{mol}\,\text{m}^{-3}, \quad k = 1\,\text{s}^{-1}, \quad t_{\text{end}} = 2\,\text{s}$$

| Time [s] | Concentration [mol m$^{-3}$] |
|---|---|
| $t_0 = 0$ | $c_0 = 1.00$ |
| $t_1 = t_0 + \Delta t$ | $c_1 = c_0 + \Delta t \cdot (-kc_0)$ |
| $= 0 + 0.1 = 0.1$ | $= 1 + 0.1 \cdot (-1 \cdot 1) = 0.9$ |
| $t_2 = t_1 + \Delta t$ | $c_2 = c_1 + \Delta t \cdot (-kc_1)$ |
| $= 0.1 + 0.1 = 0.2$ | $= 0.9 + 0.1 \cdot (-1 \cdot 0.9) = 0.81$ |
| $t_3 = t_2 + \Delta t$ | $c_3 = c_2 + \Delta t \cdot (-kc_2)$ |
| $= 0.2 + 0.1 = 0.3$ | $= 0.81 + 0.1 \cdot (-1 \cdot 0.81) = 0.729$ |
| $\dots$ | $\dots$ |
| $t_{i+1} = t_i + \Delta t$ | $c_{i+1} = c_i + \Delta t \cdot (-kc_i)$ |
| $\dots$ | $\dots$ |
| $t_{20} = 2.0$ | $c_{20} = c_{19} + \Delta t \cdot (-kc_{19}) = 0.121577$ |

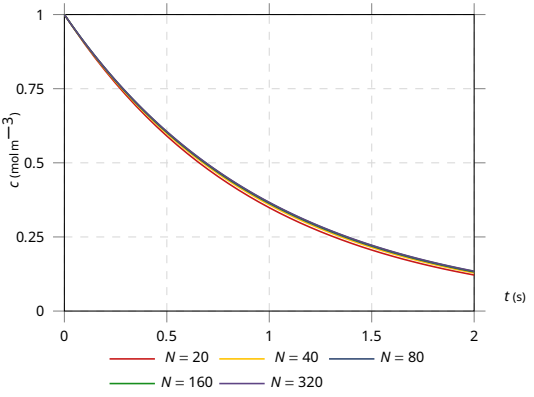
# Euler's method - example



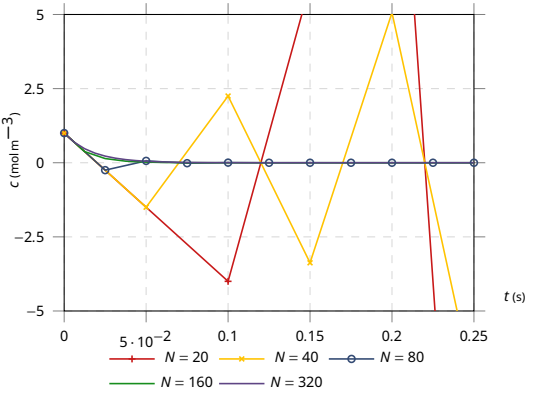
# Problems with Euler's method

The question is: What step size, or how many steps to use?

1. *Accuracy* $\Rightarrow$ need information on numerical error!
2. *Stability* $\Rightarrow$ need information on stability limits!




Reaction rate: $k = 1\ \mathrm{s}^{-1}$



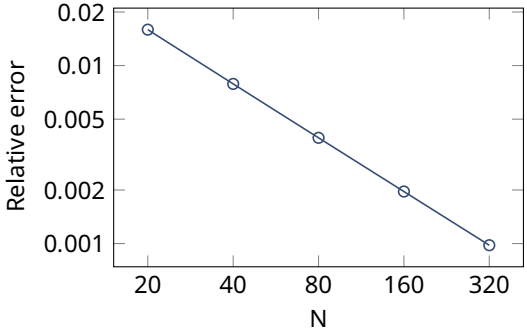

Reaction rate: $k = 50\ \mathrm{s}^{-1}$

# Accuracy

Comparison with analytical solution for $k = 1\ \mathrm{s}^{-1}$:

$$c(t) = c_0 \exp(-kt) \Rightarrow \zeta = 1 - \exp(-kt) \Rightarrow \zeta_{\text{analytical}} = 0.864665$$

| $N$ | $\zeta$ | $\frac{\zeta_{\text{numerical}} - \zeta_{\text{analytical}}}{\zeta_{\text{analytical}}}$ |
|---|---|---|
| 20 | 0.878423 | 0.015912 |
| 40 | 0.871488 | 0.007891 |
| 80 | 0.868062 | 0.003929 |
| 160 | 0.866360 | 0.001961 |
| 320 | 0.865511 | 0.000979 |



# Accuracy

For Euler's method: Error halves when the number of grid points is doubled, i.e. error is proportional to $\Delta t$: first order method.

Error estimate:

$$\left.\frac{dx}{dt}\right|_{t_0} = \frac{x(t_0+\Delta t)-x(t_0)}{\Delta t} + \frac{1}{2}\left.\frac{d^2x}{dt^2}\right|_{t_0}(\Delta t) + \mathcal{O}(\Delta t)^2$$

$$\frac{x(t_0+\Delta t)-x(t_0)}{\Delta t} = f(x_0,t_0) - \frac{1}{2}\left.\frac{d^2x}{dt^2}\right|_{t_0}(\Delta t) + \mathcal{O}(\Delta t)^2$$

# Errors and convergence rate

**Convergence rate (or: order of convergence) $r$**

$$\epsilon = \lim_{\Delta x \to 0} c(\Delta x)^r$$

- A first order method reduces the error by a factor 2 when increasing the number of steps by a factor 2
- A second order method reduces the error by a factor 4 when increasing the number of steps by a factor 2

What to do when there is no analytical solution available? Compare to calculations with different number of steps: $\epsilon_1 = c(\Delta x_1)^r$ and $\epsilon_2 = c(\Delta x_2)^r$ and solve for $r$:

$$\frac{\epsilon_2}{\epsilon_1} = \frac{c(\Delta x_2)^r}{c(\Delta x_1)^r} = \left(\frac{\Delta x_2}{\Delta x_1}\right)^r \Rightarrow \log\left(\frac{\epsilon_2}{\epsilon_1}\right) = \log\left(\frac{\Delta x_2}{\Delta x_1}\right)^r$$

$$\Rightarrow \quad r \quad = \quad \frac{\log\left(\frac{\epsilon_2}{\epsilon_1}\right)}{\log\left(\frac{\Delta x_2}{\Delta x_1}\right)} \quad = \quad \frac{\log\left(\frac{\epsilon_2}{\epsilon_1}\right)}{\log\left(\frac{N_1}{N_2}\right)} \quad \text{in the limit of} \quad \Delta x \to 0 \quad \text{or} \quad N \to \infty$$

# Today's outline


- Introduction
- Euler's method
  - Forward Euler
- Rates of convergence
- Runge-Kutta methods
  - RK2 methods
  - RK4 method
- Step size control
- Solving ODEs in Python

# Errors and convergence rate

## $L_2$ norm (Euclidean norm)

$$\|\boldsymbol{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \ldots + v_n^2} = \sqrt{\sum_{i=1}^{n} v_i^2}$$

## $L_\infty$ norm (maximum norm)

$$\|\boldsymbol{v}\|_\infty = \max(|v_1|, \ldots, |v_n|)$$

## Absolute difference

$$\epsilon_{\text{abs}} = \left\| \boldsymbol{y}_{\text{numerical}} - \boldsymbol{y}_{\text{analytical}} \right\|_{2,\infty}$$

## Relative difference

$$\epsilon_{\text{rel}} = \left\| \frac{\boldsymbol{y}_{\text{numerical}} - \boldsymbol{y}_{\text{analytical}}}{\boldsymbol{y}_{\text{analytical}}} \right\|_{2,\infty}$$

# Errors and convergence rate

**Convergence rate (or: order of convergence) $r$**

$$\epsilon = \lim_{\Delta x \to 0} c(\Delta x)^r$$

- A first order method reduces the error by a factor 2 when increasing the number of steps by a factor 2
- A second order method reduces the error by a factor 4 when increasing the number of steps by a factor 2

# Computing the rate of convergence

When the analytical solution is available, choose 1 or 2 for a particular number of grid points $N$:

1. Compute the relative or absolute error vector $\bar{\epsilon}$. Take the norm to compute a single error value $\epsilon$ following:
   - Based on $L_1$-norm: $\epsilon = \frac{\|\bar{\epsilon}\|_1}{N}$
   - Based on $L_2$-norm: $\epsilon = \frac{\|\bar{\epsilon}\|_2}{\sqrt{N}}$
   - Based on $L_\infty$-norm: $\epsilon = \|\bar{\epsilon}\|_\infty$
2. Compute the relative or absolute error at a single indicative points (e.g. middle of domain, outlet).

Compare to calculations with different number of steps: $\epsilon_1 = c(\Delta x_1)^r$ and $\epsilon_2 = c(\Delta x_2)^r$ and solve for $r$:

$$\frac{\epsilon_2}{\epsilon_1} = \frac{c(\Delta x_2)^r}{c(\Delta x_1)^r} = \left(\frac{\Delta x_2}{\Delta x_1}\right)^r \Rightarrow \log\left(\frac{\epsilon_2}{\epsilon_1}\right) = \log\left(\frac{\Delta x_2}{\Delta x_1}\right)^r$$

$$\Rightarrow \quad r \quad = \quad \frac{\log\left(\frac{\epsilon_2}{\epsilon_1}\right)}{\log\left(\frac{\Delta x_2}{\Delta x_1}\right)} \quad = \quad \frac{\log\left(\frac{\epsilon_2}{\epsilon_1}\right)}{\log\left(\frac{N_1}{N_2}\right)} \quad \text{in the limit of} \quad \Delta x \to 0 \quad \text{or} \quad N \to \infty$$

# Computing the rate of convergence

When the analytical solution is **not** available:

1. Compute the solution with $N+1$, $N$, $N-1$ and $N-2$ grid points
2. Select a single indicative grid point (e.g. middle of domain, outlet) that lies at exactly the same position in each computation
3. Use the solution $c$ at this grid point for various grid sizes to compute:

$$r = \frac{\log \frac{c_{N+1} - c_N}{c_N - c_{N-1}}}{\log \frac{c_N - c_{N-1}}{c_{N-1} - c_{N-2}}}$$

4. Alternative for simulations with $2N$, $N$ and $\frac{N}{2}$ grid points:

$$r = \frac{\log \left| \frac{c_{2N} - c_N}{c_N - c_{\frac{N}{2}}} \right|}{\log \left| \frac{N}{2N} \right|}$$

# Example: Euler's method — order of convergence

| $N$ | $\zeta$ | $\frac{\zeta_{\text{numerical}}-\zeta_{\text{analytical}}}{\zeta_{\text{analytical}}}$ | $r=\frac{\log\left(\frac{\epsilon_i}{\epsilon_{i-1}}\right)}{\log\left(\frac{N_{i-1}}{N_i}\right)}$ |
|---|---|---|---|
| 20 | 0.878423 | 0.015912 | — |
| 40 | 0.871488 | 0.007891 | 1.011832 |
| 80 | 0.868062 | 0.003929 | 1.005969 |
| 160 | 0.866360 | 0.001961 | 1.002996 |
| 320 | 0.865511 | 0.000979 | 1.001500 |

$\Rightarrow$ Euler's method is a first order method (as we already knew from the truncation error analysis)

Wouldn't it be great to have a method that can give the answer using much less steps? $\Rightarrow$ Higher order methods

# Today's outline

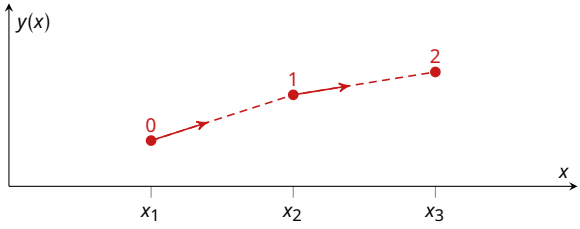
- Introduction
- Euler's method
  - Forward Euler
- Rates of convergence
- Runge-Kutta methods
  - RK2 methods
  - RK4 method
- Step size control
- Solving ODEs in Python

# Runge-Kutta methods

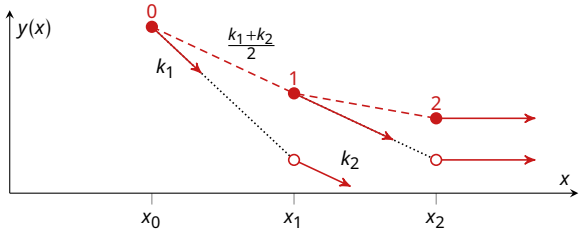
Propagate a solution by combining the information of several Euler-style steps (each involving one function evaluation) to match a Taylor series expansion up to some higher order.

Euler: $y_{i+1} = y_i + hf(x_i, y_i)$ with $h = \Delta x$, i.e. slope $= k_1 = f(x_i, y_i)$.

Euler's method




RK2 method



# Classical second order Runge-Kutta (RK2) method

This method is also called Heun's method, or improved Euler method:

1. Approximate the slope at $x_i$: $k_1 = f(x_i, y_i)$
2. Approximate the slope at $x_{i+1}$: $k_2 = f(x_{i+1}, y_{i+1})$ where we use Euler's method to approximate $y_{i+1} = y_i + hf(x_i, y_i) = y_i + hk_1$
3. Perform an Euler step with the average of the slopes: $y_{i+1} = y_i + h\frac{1}{2}(k_1 + k_2)$

In pseudocode:

$x = x_0$, $y = y_0$
**while** $x < x_{\text{end}}$ **do**
  $x_{i+1} = x_i + h$
  $k_1 = f(x_i, y_i)$
  $k_2 = f(x_i + h, y_i + hk_1)$
  $y_{i+1} = y_i + h\frac{1}{2}(k_1 + k_2)$
**end while**

# Runge-Kutta methods — derivation

$$\frac{dy}{dx} = f(x, y(x))$$

Using Taylor series expansion: $y_{i+1} = y_i + h \left.\frac{dy}{dx}\right|_i + \frac{h^2}{2} \left.\frac{d^2y}{dx^2}\right|_i + \mathcal{O}(h^3)$

$$\left.\frac{dy}{dx}\right|_i = f(x_i, y_i) \equiv f_i$$

$$\left.\frac{d^2y}{dx^2}\right|_i = \left.\frac{d}{dx} f(x, y(x))\right|_i = \left.\frac{\partial f}{\partial x}\right|_i + \left.\frac{\partial f}{\partial y}\right|_i \left.\frac{\partial y}{\partial x}\right|_i = \left.\frac{\partial f}{\partial x}\right|_i + \left.\frac{\partial f}{\partial y}\right|_i f_i \quad \text{(chain rule)}$$

Substitution gives:

$$y_{i+1} = y_i + h f_i + \frac{h^2}{2} \left( \left.\frac{\partial f}{\partial x}\right|_i + \left.\frac{\partial f}{\partial y}\right|_i f_i \right) + \mathcal{O}(h^3)$$

$$y_{i+1} = y_i + \frac{h}{2} f_i + \frac{h}{2} \left( f_i + h \left.\frac{\partial f}{\partial x}\right|_i + h f_i \left.\frac{\partial f}{\partial y}\right|_i \right) + \mathcal{O}(h^3)$$

# Runge-Kutta methods — derivation

Note multivariate Taylor expansion:

$$f(x_i + h, y_i + k) = f_i + h \left.\frac{\partial f}{\partial x}\right|_i + k \left.\frac{\partial f}{\partial y}\right|_i + \mathcal{O}(h^2)$$

$$\Rightarrow \frac{h}{2}\left(f_i + h \left.\frac{\partial f}{\partial x}\right|_i + h f_i \left.\frac{\partial f}{\partial y}\right|_i\right) = \frac{h}{2} f\left(x_i + h, y_i + h f_i\right) + \mathcal{O}(h^3)$$

Concluding:

$$y_{i+1} = y_i + \frac{h}{2} f_i + \frac{h}{2} f\left(x_i + h, y_i + h f_i\right) + \mathcal{O}(h^3)$$

Rewriting:

$$\begin{aligned} k_1 &= f(x_i, y_i) \\ k_2 &= f(x_i + h, y_i + h k_1) \\ \Rightarrow y_{i+1} &= y_i + \frac{h}{2}(k_1 + k_2) \end{aligned}$$

# Runge-Kutta methods — derivation

Generalization: $y_{i+1} = y_i + h(b_1 k_1 + b_2 k_2) + \mathcal{O}(h^3)$
with $k_1 = f_i$, $k_2 = f(x_i + c_2 h, y_1 + a_{2,1} h k_1)$
(Note that classical RK2: $b_1 = b_2 = \frac{1}{2}$ and $c_2 = a_{2,1} = 1$.)

Bivariate Taylor expansion:

$$f(x_i + c_2 h, y_i + a_{2,1} h k_1) = f_i + c_2 h \left.\frac{\partial f}{\partial x}\right|_i + a_{2,1} h k_1 \left.\frac{\partial f}{\partial y}\right|_i + \mathcal{O}(h^2)$$

$$\begin{aligned}
y_{i+1} &= y_i + h(b_1 k_1 + b_2 k_2) + \mathcal{O}(h^3) \\
&= y_i + h\left[b_1 f_i + b_2 f(x_i + c_2 h, y_1 + a_{2,1} h k_1)\right] + \mathcal{O}(h^3) \\
&= y_i + h\left[b_1 f_i + b_2 \left\{f_i + c_2 h \left.\frac{\partial f}{\partial x}\right|_i + a_{2,1} h k_1 \left.\frac{\partial f}{\partial y}\right|_i + \mathcal{O}(h^2)\right\}\right] + \mathcal{O}(h^3) \\
&= y_i + h(b_1 + b_2) f_i + h^2 b_2 \left(c_2 \left.\frac{\partial f}{\partial x}\right|_i + a_{2,1} f_i \left.\frac{\partial f}{\partial y}\right|_i\right) + \mathcal{O}(h^3)
\end{aligned}$$

Comparison with Taylor:

$$y_{i+1} = y_i + h f_i + \frac{h^2}{2}\left(\left.\frac{\partial f}{\partial x}\right|_i + \left.\frac{\partial f}{\partial y}\right|_i f_i\right) + \mathcal{O}(h^3)$$

Using $b_1 + b_2 = 1$, $c_2 b_2 = \frac{1}{2}$, $a_{2,1} b_2 = \frac{1}{2}$ $\Rightarrow$ 3 eqns and 4 unknowns $\Rightarrow$ multiple possibilities!

# Runge-Kutta methods — derivation

$$y_{i+1} = y_i + h(b_1 + b_2)f_i + h^2 b_2 \left( c_2 \left.\frac{\partial f}{\partial x}\right|_i + a_{2,1} f_i \left.\frac{\partial f}{\partial y}\right|_i \right) + \mathcal{O}(h^3)$$

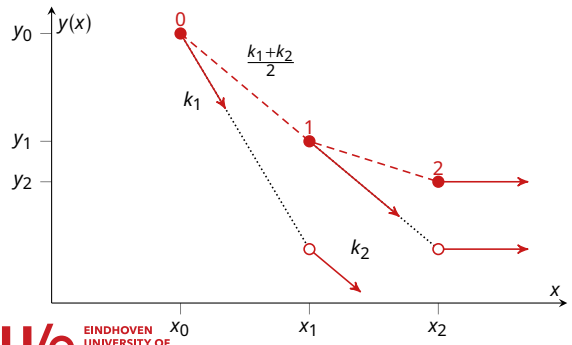
$$y_{i+1} = y_i + hf_i + \frac{h^2}{2} \left( \left.\frac{\partial f}{\partial x}\right|_i + \left.\frac{\partial f}{\partial y}\right|_i f_i \right) + \mathcal{O}(h^3)$$

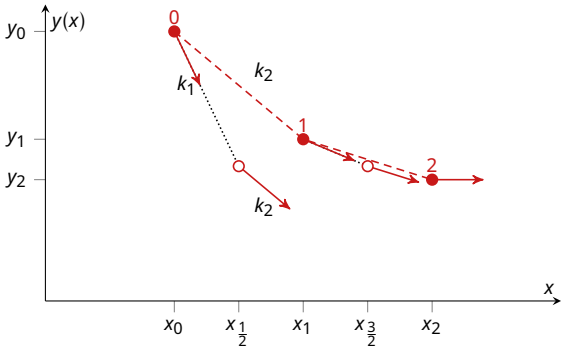
$\Rightarrow$ 3 eqns and 4 unknowns $\Rightarrow$ multiple possibilities!

1. Classical RK2:
   $b_1 = b_2 = \frac{1}{2}$ and $c_2 = a_{2,1} = 1$
2. Midpoint rule (modified Euler):
   $b_1 = 0$, $b_2 = 1$, $c_2 = a_{2,1} = \frac{1}{2}$

# Second order Runge-Kutta methods

| Classical RK2 method (= Heun's method, improved Euler method) | Explicit midpoint rule (modified Euler method) |
|---|---|
| $k_1 = f_i$ | $k_1 = f_i$ |
| $k_2 = f(x_i + h, y_i + hk_1)$ | $k_2 = f(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_1)$ |
| $y_{i+1} = y_i + \frac{1}{2}h(k_1 + k_2)$ | $y_{i+1} = y_i + hk_2$ |

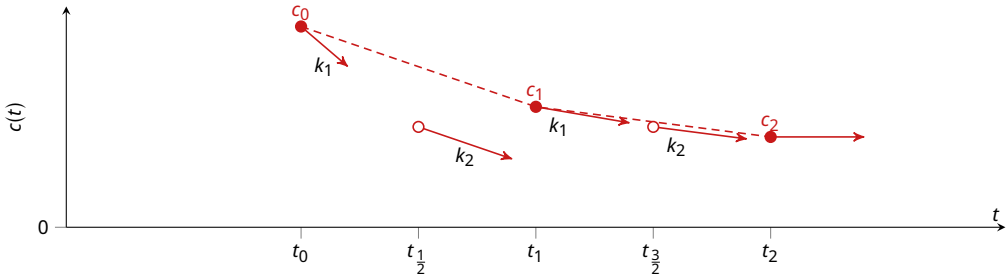


# Second order Runge-Kutta method — Example

First order reaction in a batch reactor: $\frac{dc}{dt} = -kc$ with $c(t=0) = 1\ \text{mol}\,\text{m}^{-3}$, $k = 1\ \text{s}^{-1}$, $t_{\text{end}} = 2$ s.

| Time [s] | C [mol m$^{-3}$] | $k_1 = hf(x_i, y_i)$ | $k_2 = hf(x_i + \frac{1}{2}h, y_n + \frac{1}{2}k_1)$ |
|---|---|---|---|
| 0 | 1.00 | $0.1 \cdot (-1 \cdot 1) = -0.1$ | $0.1 \cdot (-1 \cdot (1 - 0.5 \cdot 0.1)) = -0.095$ |
| 0.1 | $1 - 0.095 = 0.905$ | $0.1 \cdot (-1 \cdot 0.0905) = -0.0905$ | $0.1 \cdot (-1 \cdot (0.905 - 0.5 \cdot 0.0905)) = -0.085975$ |
| … | … | … | … |
| 2 | 0.1358225 | −0.0135822 | −0.0129031 |



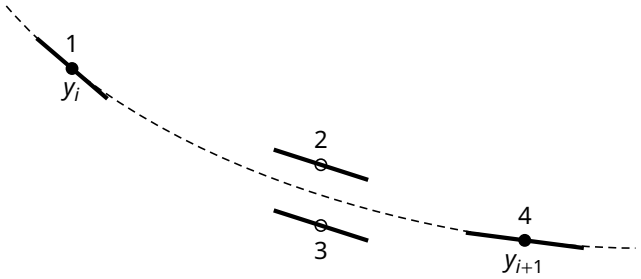
# RK2 method — order of convergence

| $N$ | $\zeta$ | $\frac{\zeta_{\text{numerical}}-\zeta_{\text{analytical}}}{\zeta_{\text{analytical}}}$ | $r=\frac{\log\left(\frac{\epsilon_i}{\epsilon_{i-1}}\right)}{\log\left(\frac{N_{i-1}}{N_i}\right)}$ |
|---|---|---|---|
| 20 | 0.864178 | $5.634\times10^{-4}$ | — |
| 40 | 0.864548 | $1.355\times10^{-4}$ | 2.056 |
| 80 | 0.864636 | $3.323\times10^{-5}$ | 2.028 |
| 160 | 0.864658 | $8.229\times10^{-6}$ | 2.014 |
| 320 | 0.864663 | $2.048\times10^{-6}$ | 2.007 |

$\Rightarrow$ RK2 is a second order method. Doubling the number of cells reduces the error by a factor 4!

Can we do even better?

# RK4 method (classical fourth order Runge-Kutta method)




$$k_1 = f(x_i, y_i)$$

$$k_2 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_1\right)$$

$$k_3 = f\left(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_2\right)$$

$$k_4 = f(x_i + h, y_i + hk_3)$$

$$y_{i+1} = y_i + h\left(\frac{1}{6}k_1 + \frac{1}{3}(k_2 + k_3) + \frac{1}{6}k_4\right)$$

# RK4 method — order of convergence

| $N$ | $\zeta$ | $\frac{\zeta_{\text{numerical}}-\zeta_{\text{analytical}}}{\zeta_{\text{analytical}}}$ | $r = \frac{\log\left(\frac{\epsilon_i}{\epsilon_{i-1}}\right)}{\log\left(\frac{N_{i-1}}{N_i}\right)}$ |
|---|---|---|---|
| 20 | 0.864664472 | $2.836 \times 10^{-7}$ | — |
| 40 | 0.864664702 | $1.700 \times 10^{-8}$ | 4.060 |
| 80 | 0.864664716 | $1.040 \times 10^{-9}$ | 4.030 |
| 160 | 0.864664717 | $6.435 \times 10^{-11}$ | 4.015 |
| 320 | 0.864664717 | $4.001 \times 10^{-12}$ | 4.007 |

$\Rightarrow$ RK4 is a fourth order method: Doubling the number of cells reduces the error by a factor 16!

Can we do even better?

# Today's outline

- Introduction
- Euler's method
  - Forward Euler
- Rates of convergence
- Runge-Kutta methods
  - RK2 methods
  - RK4 method
- Step size control
- Solving ODEs in Python

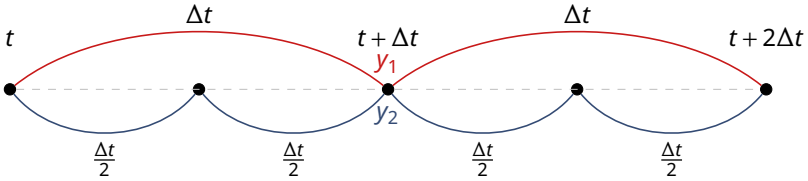
# Adaptive step size control

The step size (be it either position, time or both (PDEs)) cannot be decreased indefinitely to favour a higher accuracy, since each additional grid point causes additional computation time. It may be wise to adapt the step size according to the computation requirements.

Globally two different approaches can be used:

1. Step doubling: compare solutions when taking one full step or two consecutive halve steps
2. Embedded methods: Compare solutions when using two approximations of different order

# Adaptive step size control: step doubling




- RK4 with one large step of $h$: $y_{i+1} = y_1 + ch^5 + \mathcal{O}(h^6)$
- RK4 with two steps of $\frac{1}{2}h$: $y_{i+1} = y_2 + 2c(\frac{1}{2}h)^5 + \mathcal{O}(h^6)$

# Adaptive step size control: step doubling

- Estimation of truncation error by comparing $y_1$ and $y_2$:
  $\Delta = y_2 - y_1$
- If $\Delta$ too large, reduce step size for accuracy
- If $\Delta$ too small, increase step size for efficiency.
- Ignoring higher order terms and solving for $c$:
  $\Delta = \frac{15}{16} ch^5 \Rightarrow ch^5 = \frac{16}{15}\Delta \Rightarrow y_{i+1} = y_2 + \frac{\Delta}{15} + \mathcal{O}(h^6)$
  (local Richardson extrapolation)

Note that when we specify a tolerance *tol*, we can estimate the maximum allowable step size as: $h_{\text{new}} = \alpha h_{\text{old}} \left| \frac{\text{tol}}{\Delta} \right|^{\frac{1}{5}}$ with $\alpha$ a safety factor (typically $\alpha = 0.9$).

# Adaptive step size control: embedded methods

Use a special fourth and a fifth order Runge Kutta method to approximate $y_{i+1}$

- The fourth order method is special because we want to use the same positions for the evaluation for computational efficiency.
- RK45 is the preferred method (minimum number of function evaluations) (this is the default method in `scipy.integrate.solve_ivp`).

# Today's outline

- Introduction
- Euler's method
  - Forward Euler
- Rates of convergence
- Runge-Kutta methods
  - RK2 methods
  - RK4 method
- Step size control
- Solving ODEs in Python

# Solving ODEs in Python

SciPy provides convenient procedures to solve (systems of) ODEs automatically.

The procedure is as follows:

1. Create a function that specifies the ODE(s). Specifically, this function returns the $\frac{dy}{dx}$ value (vector).
2. Initialise solver variables and settings (e.g. step size, initial conditions, tolerance)
3. Call the ODE solver function, passing the ODE function as argument
   - The ODE solver will return a solution oject (e.g. `sol`), with attribute `sol.t` as the independent variable vector, and a `sol.y` the solution vector (or matrix for systems of ODEs).

# Solving ODEs in Python: example 1

We solve the system: $\frac{dx}{dt} = -k_1 x + k_2, k_1 = 0.2, k_2 = 2.5$

- Create a lambda function:

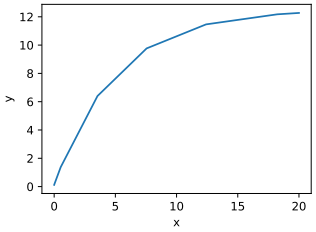
```
1 dydx = lambda x,y: (-0.2*y + 2.5)
```

- Solve with a call to `solve_ivp(function, timespan, initial_condition)`:

```
1 from scipy.integrate import solve_ivp
2 sol = solve_ivp(dydx, tspan, y0)
```

- Draw the results by calling the relevant Matplotlib commands:

```
3 import matplotlib.pyplot as plt
4 plt.plot(sol.t, sol.y[0,:])
5 plt.show()
```



# Solving ODEs in Python: example 2

We solve the system: $\frac{dx}{dt} = \begin{cases} -\frac{k_1}{x^2} & t \leq 10 \\ \frac{k_2}{x} - \frac{k_1}{x^2} & t > 10 \end{cases}$ with $k_1 = 0.5, k_2 = 1, x(0) = 2$

## Create an ODE function

```
def myEqnFunction(t,x):
    k1 = 0.5;
    k2 = 1;
    dxdt = int(t>10)*k2/x - k1/x**2;
    return dxdt
```

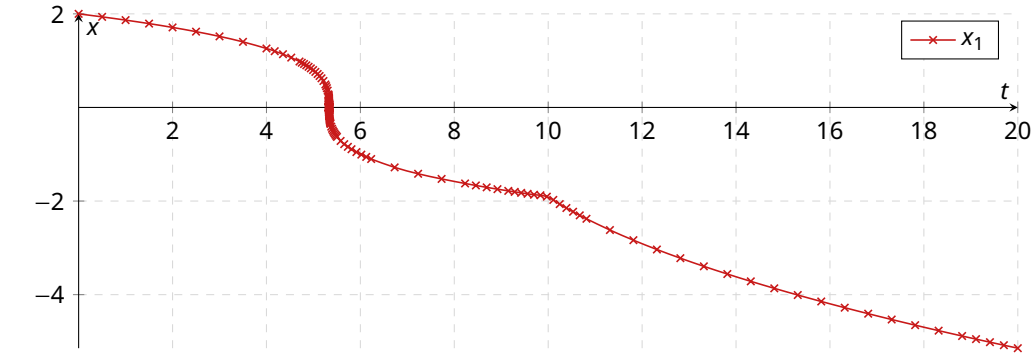
## Create a solution script

```
tspan = [0, 20]
x_init = [2]
sol = solve_ivp(myEqnFunction, tspan, x_init, rtol=1e-8, atol=1e-6)
```

# Solving ODEs in Python: example 2

Plot the solution:

```
plt.plot(sol.t, sol.y[0,:],'r-x')
plt.grid()
plt.show()
```




Note the refinement in regions where large changes occur.

# Solving ODEs in Python: example

A few notes on working with `scipy.integrate.solve_ivp` and other ODE solvers. If we want to give additional arguments (e.g. `k1` and `k2`) to our ODE function, we can list them in the function line:

```
func = lambda t,x,k1,k2: k1*x+k2
# or
def func(t,x,k1,k2):
    return k1*x+k2
```

The additional arguments can now be set in the solver script by *adding them as `args` list*:

```
sol = solve_ivp(func,[0,5],[1],args=(k1, k2))
```

Of course, in the solver script, the variables do not have to be called `k1` and `k2`:

```
sol = solve_ivp(func,[0,5],[1],args=(q, u))
```

These variables may be of any type (scalar, vector, dictionary, list). For carrying over many variables, a dictionary is useful and descriptive.

# Solving systems of ODEs in Python: example

You have noticed that the step size in $t$ varies. This is because we have given just the begin and end times of our time span:

```
tspan = [0, 5];
```

You can also obtain the solution at specific points, by supplying a list `t_eval`:

```
sol = solve_ivp(func,tspan,[1],args=(some_k1, some_k2),t_eval=np.linspace(tspan[0],tspan[1],31))
```

This example provides 31 explicit time steps between 0 and 5 seconds. Note that the results are interpolated to these data points afterwards; you do not influence the efficiency and accuracy of the solver algorithm this way!

# Ordinary differential equations 2

## Implicit methods, systems of ODEs and boundary value problems

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
  - Backward Euler
  - Implicit midpoint method
- Systems of ODEs
  - Solution methods for systems of ODEs
  - Solving systems of ODEs in Python
  - Stiff systems of ODEs
- Boundary value problems
  - Shooting method
- Conclusion

# Problems with Euler's method: instability

Consider the ODE:

$$\frac{dy}{dx} = f(x, y(x)) \qquad \text{with} \qquad y(x = 0) = y_0$$

First order approximation of derivative: $\frac{dy}{dx} = \frac{y_{i+1} - y_i}{\Delta x}$.

Where to evaluate the function $f$?

1. Evaluation at $x_i$: Explicit Euler method (forward Euler)
2. Evaluation at $x_{i+1}$: Implicit Euler method (backward Euler)

# Problems with Euler's method: instability – forward Euler

Explicit Euler method (forward Euler):

- Use values at $x_i$:
  $\frac{y_{i+1}-y_i}{\Delta x} = f(x_i, y_i) \Rightarrow y_{i+1} = y_i + hf(x_i, y_i)$.
- This is an explicit equation for $y_{i+1}$ in terms of $y_i$.
- It can give instabilities with large function values.

Consider the first order batch reactor:

$$\frac{dc}{dt} = -kc \Rightarrow c_{i+1} = c_i - kc_i\Delta t \Rightarrow \frac{c_{i+1}}{c_i} = 1 - k\Delta t$$

It follows that unphysical results are obtained for $k\Delta t \geq 1$!!

**Stability requirement**

$$k\Delta t < 1$$

(but probably accuracy requirements are more stringent here!)

# Problems with Euler's method: instability – backward Euler

Implicit Euler method (backward Euler):

- Use values at $x_{i+1}$: $\frac{y_{i+1}-y_i}{\Delta x} = f(x_{i+1}, y_{i+1}) \Rightarrow y_{i+1} = y_i + hf(x_{i+1}, y_{i+1})$.
- This is an implicit equation for $y_{i+1}$, because it also depends on terms of $y_{i+1}$.

Consider the first order batch reactor:

$$\frac{dc}{dt} = -kc \Rightarrow c_{i+1} = c_i - kc_{i+1}\Delta t \Rightarrow \frac{c_{i+1}}{c_i} = \frac{1}{1+k\Delta t}$$

This equation does never give unphysical results!
The implicit Euler method is *unconditionally stable*
(but maybe not very accurate or efficient).

# Semi-implicit Euler method

Usually $f$ is a non-linear function of $y$, so that linearization is required (recall Newton's method).

$$\frac{dy}{dx} = f(y) \Rightarrow y_{i+1} = y_i + hf(y_{i+1}) \quad \text{using} \quad f(y_{i+1}) = f(y_i) + \left.\frac{df}{dy}\right|_i (y_{i+1} - y_i) + \dots$$

$$\Rightarrow y_{i+1} = y_i + h\left[f(y_i) + \left.\frac{df}{dy}\right|_i (y_{i+1} - y_i)\right]$$

$$\Rightarrow \left(1 - h\left.\frac{df}{dy}\right|_i\right) y_{i+1} = \left(1 - h\left.\frac{df}{dy}\right|_i\right) y_i + hf(y_i)$$

$$\Rightarrow \quad y_{i+1} \quad = \quad y_i \quad + \quad h\left(1 - h\left.\frac{df}{dy}\right|_i\right)^{-1} f(y_i)$$

For the case that $f(x, y(x))$ we could add the variable $x$ as an additional variable $y_{n+1} = x$. Or add one fully implicit Euler step (which avoids the computation of $\frac{\partial f}{\partial x}$):

$$y_{i+1} = y_i + hf(x_{i+1}, y_{i+1}) \Rightarrow y_{i+1} = y_i + h\left(1 - h\left.\frac{df}{dy}\right|_i\right)^{-1} f(x_{i+1}, y_i)$$

# Semi-implicit Euler method - example

Second order reaction in a batch reactor:
$\frac{dc}{dt} = -kc^2$ with $c_0 = 1\ \text{mol}\,\text{m}^{-3}$, $k = 1\ \text{m}^3\,\text{mol}^{-1}\,\text{s}^{-1}$, $t_{\text{end}} = 2\ \text{s}$
Analytical solution: $c(t) = \frac{c_0}{1+kc_0t}$

Define $f = -kc^2$, then $\frac{df}{dc} = -2kc \Rightarrow c_{i+1} = c_i - \frac{hkc_i^2}{1+2hkc_i}$.

| $N$ | $\zeta$ | $\frac{\zeta_{\text{numerical}} - \zeta_{\text{analytical}}}{\zeta_{\text{analytical}}}$ | $r = \frac{\log\left(\frac{\epsilon_i}{\epsilon_{i-1}}\right)}{\log\left(\frac{N_{i-1}}{N_i}\right)}$ |
|---|---|---|---|
| 20 | 0.654066262 | $1.89 \times 10^{-2}$ | — |
| 40 | 0.660462687 | $9.31 \times 10^{-3}$ | 1.02220 |
| 80 | 0.663589561 | $4.62 \times 10^{-3}$ | 1.01162 |
| 160 | 0.665134433 | $2.30 \times 10^{-3}$ | 1.00594 |
| 320 | 0.665902142 | $1.15 \times 10^{-3}$ | 1.00300 |

# Second order implicit method: Implicit midpoint method

| Implicit midpoint rule (second order) | Explicit midpoint rule (modified Euler method) |
|---|---|
| $y_{i+1} = y_i + hf\left(x_i + \frac{1}{2}h, \frac{1}{2}(y_i + y_{i+1})\right)$ | $y_{i+1} = y_i + hf(x_i + \frac{1}{2}h, y_i + \frac{1}{2}hk_1)$ |

in case $f(y)$ then:

$$f\left(\frac{1}{2}(y_i + y_{i+1})\right) = f_i + \left.\frac{df}{dy}\right|_i \left(\frac{1}{2}(y_i + y_{i+1}) - y_i\right) = f_i + \frac{1}{2}\left.\frac{df}{dy}\right|_i (y_{i+1} - y_i)$$

Implicit midpoint rule reduces to:

$$y_{i+1} = y_i + hf_i + \frac{h}{2}\left.\frac{df}{dy}\right|_i (y_{i+1} - y_i)$$

$$\Rightarrow \left(1 - \frac{h}{2}\left.\frac{df}{dy}\right|_i\right) y_{i+1} = \left(1 - \frac{h}{2}\left.\frac{df}{dy}\right|_i\right) y_i + hf_i$$

$$\Rightarrow \quad y_{i+1} \quad = \quad y_i \quad + \quad h\left(1 - \frac{h}{2}\left.\frac{df}{dy}\right|_i\right)^{-1} f_i$$

# Implicit midpoint method — example

Second order reaction in a batch reactor:
$\frac{dc}{dt} = -kc^2$ with $c_0 = 1\ \text{mol}\,\text{m}^{-3}$, $k = 1\ \text{m}^3\,\text{mol}^{-1}\,\text{s}^{-1}$, $t_{\text{end}} = 2$ s (Analytical solution: $c(t) = \frac{c_0}{1+kc_0t}$).

Define $f = -kc^2$, then $\frac{df}{dc} = -2kc$.

Substitution:

$$\begin{aligned} c_{i+1} &= c_i + h\left(1 - \frac{h}{2}\cdot(-2kc_i)\right)^{-1}\cdot(-kc_i^2) \\ &= c_i - \frac{hkc_i^2}{1+hkc_i} = \frac{c_i + hkc_i^2 - hkc_i^2}{1+hkc_i} \Rightarrow c_{i+1} = \frac{c_i}{1+hkc_i} \end{aligned}$$

You will find that this method is exact for all step sizes $h$ because of the quadratic source term!

# Implicit midpoint method — example

Second order reaction in a batch reactor:
$\frac{dc}{dt} = -kc^2$ with $c_0 = 1\ \mathrm{mol\,m^{-3}}$, $k = 1\ \mathrm{m^3\,mol^{-1}\,s^{-1}}$, $t_{\mathrm{end}} = 2\ \mathrm{s}$
Analytical solution: $c(t) = \frac{c_0}{1+kc_0t}$

$$c_{i+1} = \frac{c_i}{1 + hkc_i}$$

| $N$ | $\zeta$ | $\frac{\zeta_{\mathrm{numerical}} - \zeta_{\mathrm{analytical}}}{\zeta_{\mathrm{analytical}}}$ | $r = \frac{\log\left(\frac{\epsilon_i}{\epsilon_{i-1}}\right)}{\log\left(\frac{N_{i-1}}{N_i}\right)}$ |
|---|---|---|---|
| 20 | 0.6666666667 | $1.665 \times 10^{-16}$ | — |
| 40 | 0.6666666667 | 0 | — |
| 80 | 0.6666666667 | 0 | — |
| 160 | 0.6666666667 | 0 | — |
| 320 | 0.6666666667 | 0 | — |

# Implicit midpoint method — example

Third order reaction in a batch reactor: $\frac{dc}{dt} = -kc^3$

Analytical solution: $c(t) = \frac{c_0}{\sqrt{1+2kc_0^2 t}}$

$$c_{i+1} = c_i - \frac{hkc_i^3}{1 + \frac{3}{2}hkc_i^2}$$

| $N$ | $\zeta$ | $\frac{\zeta_{\text{numerical}} - \zeta_{\text{analytical}}}{\zeta_{\text{analytical}}}$ | $r = \frac{\log\left(\frac{\epsilon_i}{\epsilon_{i-1}}\right)}{\log\left(\frac{N_{i-1}}{N_i}\right)}$ |
|---|---|---|---|
| 20 | 0.5526916174 | $1.71 \times 10^{-4}$ | — |
| 40 | 0.5527633731 | $4.17 \times 10^{-5}$ | 2.041 |
| 80 | 0.5527807304 | $1.03 \times 10^{-5}$ | 2.021 |
| 160 | 0.5527849965 | $2.55 \times 10^{-6}$ | 2.011 |
| 320 | 0.5527860538 | $6.34 \times 10^{-7}$ | 2.005 |

# Today's outline


- Introduction
  - Backward Euler
  - Implicit midpoint method
- Systems of ODEs
  - Solution methods for systems of ODEs
  - Solving systems of ODEs in Python
  - Stiff systems of ODEs
- Boundary value problems
  - Shooting method
- Conclusion

# Systems of ODEs

A system of ODEs is specified using vector notation:

$$\frac{d\boldsymbol{y}}{dx} = \boldsymbol{f}(x, \boldsymbol{y}(x))$$

for

$$\frac{dy_1}{dx} = f_1(x, y_1(x), y_2(x)) \quad \text{or} \quad f_1(x, y_1, y_2)$$
$$\frac{dy_2}{dx} = f_2(x, y_1(x), y_2(x)) \quad \text{or} \quad f_2(x, y_1, y_2)$$

The solution techniques discussed before can also be used to solve systems of equations.

# Systems of ODEs: Explicit methods

## Forward Euler method

$$\boldsymbol{y}_{i+1} = \boldsymbol{y}_i + h\boldsymbol{f}(x_i, \boldsymbol{y}_i)$$

## Improved Euler method (classical RK2)

$$\boldsymbol{y}_{i+1} = \boldsymbol{y}_i + \frac{h}{2}(\boldsymbol{k}_1 + \boldsymbol{k}_2) \quad \text{using} \quad \begin{aligned} \boldsymbol{k}_1 &= \boldsymbol{f}(x_i, \boldsymbol{y}_i) \\ \boldsymbol{k}_2 &= \boldsymbol{f}(x_i + h, \boldsymbol{y}_i + h\boldsymbol{k}_1) \end{aligned}$$

## Modified Euler method (midpoint rule)

$$\boldsymbol{y}_{i+1} = \boldsymbol{y}_i + h\boldsymbol{k}_2 \quad \text{using} \quad \begin{aligned} \boldsymbol{k}_1 &= \boldsymbol{f}(x_i, \boldsymbol{y}_i) \\ \boldsymbol{k}_2 &= \boldsymbol{f}(x_i + \frac{h}{2}, \boldsymbol{y}_i + \frac{h}{2}\boldsymbol{k}_1) \end{aligned}$$

# Systems of ODEs: Explicit methods

## Classical fourth order Runge-Kutta method (RK4)

$$\boldsymbol{y}_{i+1} = \boldsymbol{y}_i + h\left(\frac{\boldsymbol{k}_1}{6} + \frac{1}{3}(\boldsymbol{k}_2 + \boldsymbol{k}_3) + \frac{\boldsymbol{k}_4}{6}\right)$$

using

$$\boldsymbol{k}_1 = \boldsymbol{f}(x_i, \boldsymbol{y}_i)$$

$$\boldsymbol{k}_2 = \boldsymbol{f}(x_i + \frac{h}{2}, \boldsymbol{y}_i + \frac{h}{2}\boldsymbol{k}_1)$$

$$\boldsymbol{k}_3 = \boldsymbol{f}(x_i + \frac{h}{2}, \boldsymbol{y}_i + \frac{h}{2}\boldsymbol{k}_2)$$

$$\boldsymbol{k}_4 = \boldsymbol{f}(x_i + h, \boldsymbol{y}_i + h\boldsymbol{k}_3)$$

# Solving systems of ODEs in Python

Solving systems of ODEs in Python is completely analogous to solving a single ODE:

1. Create a function that specifies the ODEs. This function returns the $\frac{d\boldsymbol{y}}{dx}$ vector.
2. Initialise solver variables and settings (e.g. step size, initial conditions, tolerance), in a separate script. Initial conditions and tolerances should be given per-equation, i.e. as a vector.
3. Call the ODE solver function, using a function argument to the ODE function described in point 1.
   - The ODE solver will return the vector for the independent variable (e.g. time), and a solution matrix, with a column as the solution for each equation in the system.

# Solving systems of ODEs in Python: example

We solve the system $\frac{dx_0}{dt} = ax_0 - x_1, \quad \frac{dx_1}{dt} = bx_1 + x_0$, with $a = -1$ and $b = -2$:

- Create an ODE function:

```
# Example scipy solve_ivp/Example scipy solve_ivp vector.py
def func(t, x, a, b):
    #output can be of list or np.array type:
    dxdt = np.zeros(2)

    dxdt[0] = a*x[0] - x[1]
    dxdt[1] = b*x[1] + x[0]
    return dxdt
```

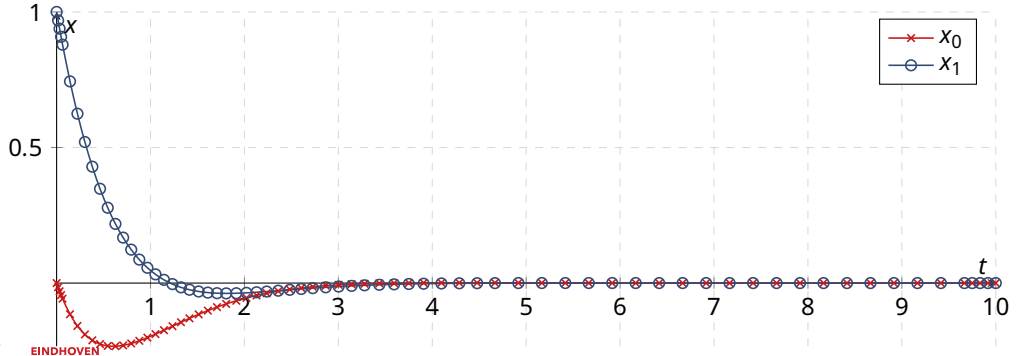
- Solve by calling `solve_ivp`

```
from scipy.integrate import solve_ivp
x_init = [0,1]; % Initial conditions
tspan = [0,10]; % Time span
sol = solve_ivp(func, tspan, x_init, args=(-1,-2), rtol=1e-12)
```

# Solving systems of ODEs in Python: example

Plot the solution (note: the solution is attribute `sol.y`):

```
import matplotlib.pyplot as plt
plt.plot(sol.t, sol.y[0], 'r-x', linewidth=2)
plt.plot(sol.t, sol.y[1], 'b-o', linewidth=2)
```



# Solving ODEs in Python: example

A few notes on working with `scipy.integrate.solve_ivp` and other ODE solvers. If we want to give additional arguments (e.g. `k1` and `k2`) to our ODE function, we can list them in the function line:

```
func = lambda t,x,k1,k2: k1*x+k2
# or
def func(t,x,k1,k2):
    return k1*x+k2
```

The additional arguments can now be set in the solver script by *adding them as `args` list*:

```
sol = solve_ivp(func,[0,5],[1],args=(k1, k2))
```

Of course, in the solver script, the variables do not have to be called `k1` and `k2`:

```
sol = solve_ivp(func,[0,5],[1],args=(q, u))
```

These variables may be of any type (scalar, vector, dictionary, list). For carrying over many variables, a dictionary is useful and descriptive.

# Solving systems of ODEs in Python: example

You may have noticed that the step size in $t$ varies. This happens when only the begin and end times of the time span are defined, and `scipy.integrate.solve_ivp` uses adaptive step size for efficiency:

```
tspan = [0, 10]
```

You can also retrieve the solution at specific steps, by supplying all steps explicitly as an additional argument to `solve_ivp`, e.g.:

```
sol = solve_ivp(func, tspan, x_init, args=(-1,-2), t_eval=np.linspace(0, 10, 101), rtol=1e-12)
```

This example provides 101 explicit time steps between 0 and 10 seconds. It can be useful if you need a direct comparison with e.g. measurements at specific times.

Note that this is an interpolated result. The solver uses, in the background, still the adaptive step size functionality!

# Systems of ODEs: Implicit methods

## Backward Euler method

$$\boldsymbol{y}_{i+1} = \boldsymbol{y}_i + h\left(\boldsymbol{I} - h\left.\frac{d\boldsymbol{f}}{d\boldsymbol{y}}\right|_i\right)^{-1} \boldsymbol{f}(\boldsymbol{y}_i)$$

## Implicit midpoint method

$$\boldsymbol{y}_{i+1} = \boldsymbol{y}_i + h\left(\boldsymbol{I} - \frac{h}{2}\left.\frac{d\boldsymbol{f}}{d\boldsymbol{y}}\right|_i\right)^{-1} \boldsymbol{f}(\boldsymbol{y}_i)$$

# Stiff systems of ODEs

A system of ODEs can be stiff and require a different solution method. For example:

$$\frac{dc_1}{dt} = 998c_1 + 1998c_2 \qquad \frac{dc_2}{dt} = -999c_1 - 1999c_2$$

with boundary conditions $c_1(t = 0) = 1$ and $c_2(t = 0) = 0$.
The analytical solution is:

$$c_1 = 2e^{-t} - e^{-1000t} \qquad c_2 = -e^{-t} + e^{-1000t}$$

For the explicit method we require $\Delta t < 10^{-3}$ despite the fact that the term is completely negligible, but essential to keep stability.

The "disease" of stiff equations: we need to follow the solution on the shortest length scale to maintain stability of the integration, although accuracy requirements would allow a much larger time step.

# Demonstration with example

Forward Euler (explicit)

$$\frac{c_{1,i+1} - c_{1,i}}{dt} = 998c_{1,i} + 1998c_{2,i}$$
$$\frac{c_{2,i+1} - c_{2,i}}{dt} = -999c_{1,i} - 1999c_{2,i}$$

$$\Rightarrow \begin{aligned} c_{1,i+1} &= (1 + 998\Delta t)c_{1,i} + 1998\Delta t c_{2,i} \\ c_{2,i+1} &= -999\Delta t c_{1,i} + (1 - 1999\Delta t)c_{2,i} \end{aligned}$$

# Demonstration with example

Backward Euler (implicit)

$$\frac{c_{1,i+1} - c_{1,i}}{\Delta t} = 998c_{1,i+1} + 1998c_{2,i+1}$$
$$\frac{c_{2,i+1} - c_{2,i}}{\Delta t} = -999c_{1,i+1} - 1999c_{2,i+1}$$

$$\Rightarrow \begin{aligned} (1 - 998\Delta t)c_{1,i+1} - 1998\Delta t c_{2,i} &= c_{1,i} \\ 999\Delta t c_{1,i+1} + (1 + 999\Delta t)c_{2,i+1} &= c_{2,i} \end{aligned}$$

$$A\boldsymbol{c}_{i+1} = \boldsymbol{c}_i \text{ with } A = \begin{pmatrix} 1 - 998\Delta t & -1998\Delta t \\ 999\Delta t & 1 + 1999\Delta t \end{pmatrix} \text{ and } \boldsymbol{b} = \begin{pmatrix} c_{1,i} \\ c_{2,i} \end{pmatrix}$$

# Demonstration with example

Backward Euler (implicit) $A\boldsymbol{c}_{i+1} = \boldsymbol{c}_i$ with $A = \begin{pmatrix} 1-998\Delta t & -1998\Delta t \\ 999\Delta t & 1+1999\Delta t \end{pmatrix}$ and $\boldsymbol{b} = \begin{pmatrix} c_{1,i} \\ c_{2,i} \end{pmatrix}$

Cramers rule:

$$c_{1,i+1} = \frac{\begin{vmatrix} c_{1,i} & -1998\Delta t \\ c_{2,i} & 1+1999\Delta t \end{vmatrix}}{\det|A|} = \frac{(1+1999\Delta t)c_{1,i}+1998\Delta t c_{2,i}}{(1-998\Delta t)(1+1999\Delta t)+1998\cdot 999\Delta t^2}$$

$$c_{2,i+1} = \frac{\begin{vmatrix} 1-998\Delta t & c_{1,i} \\ 999\Delta t & c_{2,i} \end{vmatrix}}{\det|A|} = \frac{-999\Delta t c_{1,i}+(1-998\Delta t)c_{2,i}}{(1-998\Delta t)(1+1999\Delta t)+1998\cdot 999\Delta t^2}$$

Forward Euler: $\Delta t \leq 0.001$ for stability

Backward Euler: always stable, even for $\Delta t > 100$ (but then not very accurate!)

# Demonstration with example

Cure for stiff problems: use implicit methods! To find out whether your system is stiff: check whether one of the eigenvalues have an imaginary part

# Implicit methods in Python

SciPy offers a solver that detects stiff systems, using `method='LSODA'`.

$$\frac{dc_1}{dt} = 998c_1 + 1998c_2 \quad \frac{dc_2}{dt} = -999c_1 - 1999c_2, c_1(0) = 1, c_2(0) = 0$$

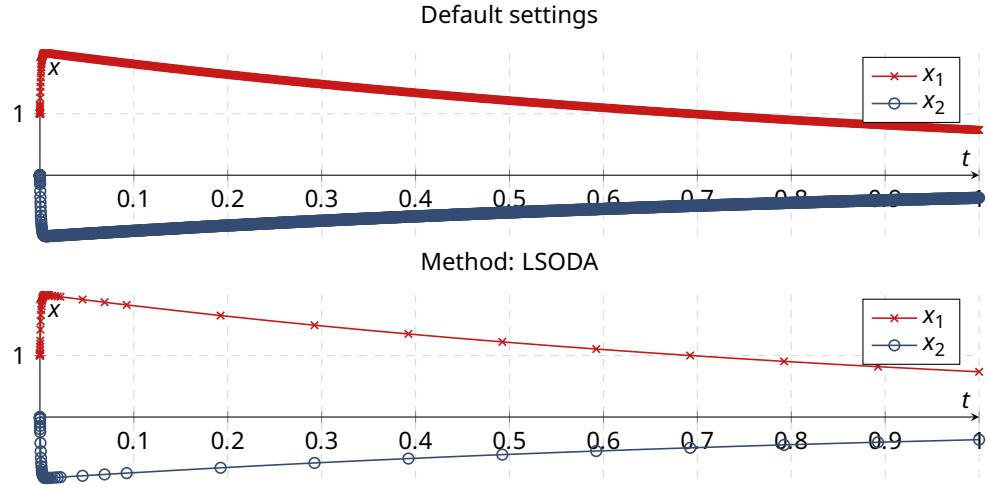
- Create the ode function (see `slide_example_solve_ivp_implicit.py`)

```
function [dcdt] = stiff_ode(t,c)
dcdt = zeros(2,1); % Pre-allocation
dcdt(1) = 998 * c(1) + 1998*c(2);
dcdt(2) = -999 * c(1) - 1999*c(2);
return
```

- Compare the resolution of the solutions (see next slide)

```
sol1 = solve_ivp(stiff_ode, [0, 1], [1, 0])
# plot sol1
sol2 = solve_ivp(stiff_ode, [0, 1], [1, 0],method = 'LSODA')
# plot sol2
```

# Implicit methods in Python




The explicit solver requires 1245 data points (default settings), the implicit solver just 48!

# Implicit methods in Python: Generic backward Euler

How to make a generic Backward Euler implementation? Recall the update formula:

$$\boldsymbol{y}_{i+1} = \boldsymbol{y}_i + h\left(\boldsymbol{I} - h\left.\frac{d\boldsymbol{f}}{d\boldsymbol{y}}\right|_i\right)^{-1} \boldsymbol{f}(\boldsymbol{y}_i)$$

- Set up input: Number of steps, end time, initial conditions
- Preallocate and calculate: create a full time vector, calculate the step size $h$, preallocate $y$ with zeros and store the initial condition as the first $y$.
- Loop over the number of iterations:
  - Compute the Jacobian: calculate the function both for $y_i$ as well as for $y_i + s$, where $s$ is a very small number. Recall:

$$\frac{df}{dy} = \frac{f(y+s) - f(y)}{s}$$

  - Compute the update formula for $y_{i+1}$. Use `eye`, `inv`.

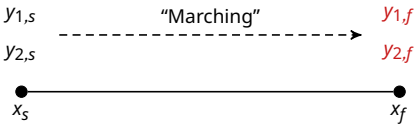
# Today's outline

- Introduction
  - Backward Euler
  - Implicit midpoint method
- Systems of ODEs
  - Solution methods for systems of ODEs
  - Solving systems of ODEs in Python
  - Stiff systems of ODEs
- **Boundary value problems**
  - Shooting method
- Conclusion

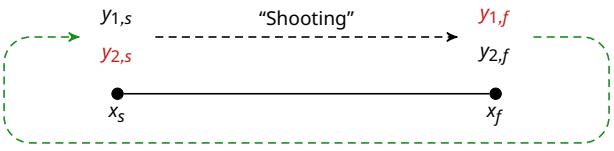
# Importance of boundary conditions

The nature of boundary conditions determines the appropriate numerical method. Classification into 2 main categories:

- *Initial value problems (IVP)*
  We know the values of all $y_i$ at some starting position $x_s$, and it is desired to find the values of $y_i$ at some final point $x_f$.

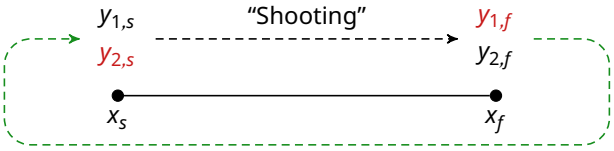



- *Boundary value problems (BVP)*
  Boundary conditions are specified at more than one $x$. Typically, some of the BC are specified at $x_s$ and the remainder at $x_f$.



# Shooting method

How to solve a BVP using the shooting method:




- Define the system of ODEs
- Provide an initial guess for the unknown boundary condition
- Solve the system and compare the resulting boundary condition to the expected value
- Adjust the guessed boundary value, and solve again. Repeat until convergence.
  - Of course, you can subtract the expected value from the computed value at the boundary, and use a non-linear root finding method

# BVP: example in Excel

Consider a chemical reaction in a liquid film layer of thickness $\delta$:

$$\mathcal{D}\frac{d^2c}{dx^2} = k_R c \text{ with } \begin{array}{ll} c(x=0) = C_{A,i,L} = 1 & \text{(interface concentration)} \\ c(x=\delta) = 0 & \text{(bulk concentration)} \end{array}$$

Question: compute the concentration profile in the film layer.

## Step 1: Define the system of ODEs

This second-order ODE can be rewritten as a system of first-order ODEs, if we define the flux $q$ as:

$$q = -\mathcal{D}\frac{dc}{dx}$$

Now, we find:

$$\frac{dc}{dx} = -\frac{1}{\mathcal{D}}q$$
$$\frac{dq}{dx} = -k_R c$$

# BVP: example in Excel

Consider a chemical reaction in a liquid film layer of thickness $\delta$:

$$\mathcal{D}\frac{d^2c}{dx^2} = k_R c \text{ with } \begin{array}{ll} c(x=0) = C_{A,i,L} = 1 & \text{(interface concentration)} \\ c(x=\delta) = 0 & \text{(bulk concentration)} \end{array}$$

Question: compute the concentration profile in the film layer.

**Step 2: Set the boundary conditions**

The boundary conditions for the concentrations at $x = 0$ and $x = \delta$ are known.

The flux at the interface, however, is not known, and should be solved for.

$$\frac{dc}{dx} = -\frac{1}{\mathcal{D}}q$$

$$\frac{dq}{dx} = -k_R c$$

# BVP: example in Excel

Solving the two first-order ODEs in Excel. First, the cells with constants:

| | A | B | C |
|---|---|---|---|
| 1 | CAiL | 1 | mol/m3 |
| 2 | D | 1e–8 | m2/s |
| 3 | kR | 10 | 1/s |
| 4 | delta | 1e–4 | m |
| 5 | N | 100 | |
| 6 | dx | =B4/B5 | |

$$\frac{dc}{dx} = -\frac{1}{\mathcal{D}} q$$

$$\frac{dq}{dx} = -k_R c$$

Now, we program the forward Euler (explicit) schemes for $c$ and $q$ below:

| | A | B | C |
|---|---|---|---|
| 10 | x | c | q |
| 11 | 0 | =B1 | 10 |
| 12 | =A11+$B$6 | =B11+$B$6*(–1/$B$2*C11) | =C11+$B$6*(-$B$3*B11) |
| 13 | =A12+$B$6 | =B12+$B$6*(–1/$B$2*C12) | =C12+$B$6*(-$B$3*B12) |
| … | … | … | … |
| 111 | =A110+$B$6 | =B110+$B$6*(–1/$B$2*C110) | =C110+$B$6*(-$B$3*B110) |

# BVP: example in Excel

- We now have profiles for $c$ and $q$ as a function of position $x$.
- The concentration $c(x = \delta)$ depends (eventually) on the boundary condition at the interface $q(x = 0)$
- We can use the solver to change $q(x = 0)$ such that the concentration at the bulk meets our requirement: $c(x = \delta) = 0$

# BVP: example in Python

We first program the system of ODEs in a separate function:

$$\frac{dc}{dx} = -\frac{1}{\mathcal{D}}q$$
$$\frac{dq}{dx} = -k_R c$$

```
# slides_example_bvp_1.py
def diffReactSystem(x, y, param):
    c, q = y
    f = np.zeros_like(y)
    f[0] = -q/param['Diff']
    f[1] = -param['kR']*c
    return f
```

Note that we pass a variable (type: dictionary) that contains required parameters: `param`.

# BVP: example in Python

Let's first try to solve the ODE system using `scipy.integrate.solve_ivp`:

```python
# slides_example_bvp_1.py
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

### Definition of diffReactSystem here (see slide 498 )

# Set up parameters
q0 = 1e-3 # Initial guess flux@t=0
param = {'cAiL': 1.0,'Diff':1e-8,'kR': 10,'delta': 1e-4,'N': 100}

# Solve ODE system
sol = solve_ivp(lambda x, y: diffReactSystem(x, y, param), # ODE func with params
                              [0, param['delta']], # Time span
                              [param['cAiL'], q0]) # Initial conditions

fig,ax1 = plt.subplots()
ax1.plot(sol.t,sol.y[0,:],'-b',label='Concentration $mol/m^3$')
ax2 = ax1.twinx() # Create y-y axis
ax2.plot(sol.t,sol.y[1,:],'-r',label='Flux $mol/m^2/s$')
fig.legend(bbox_to_anchor=(0.5, 0.5))
plt.show()
```

# BVP: example in Python

That seems to work! Now we want to fit the value for $q$ at $x = 0$ (defined below as `bcq`), such that the concentration at $x = \delta$ equals zero. We create a function with the output defined as the deviation from the target value:

```
# slides_example_bvp_2.py
def diffReactFitCriterium(bcq, param):
    # Solve the ODE system using changeable parameter bcq
    # (boundary condition for q), other parameters are defined in param
    sol = solve_ivp(lambda x, y: diffReactSystem(x, y, param), [0, param['delta']], [param['cAiL'
        ], bcq])
    # Return the last value of the concentration (column 0 in y) at x=delta (hence [-1])
    return sol.y[0,-1] - 0
```

Note the following:

- We use the interval $0 \leq x \leq \delta$
- Boundary conditions are given as: $c(x = 0) = 1$ and $q(x = 0) = \texttt{bcq}$, which is given as a separate argument to the function (i.e. changable from 'outside'!)
- The function returns the concentration at $x = \delta$

# BVP: example in Python

Finally, we should solve the system so that we obtain the right boundary condition $q = \texttt{bcq}$ such that $c(x = \delta) = 0$. We can use the `scipy.optimize.root_scalar` function to do this. Extend the script from slide 499 by:

```
# slides_example_bvp_2.py
from scipy.optimize import root_scalar

### Define diffReactSystem, diffReactFitCriterium, parameters

# Solve such that c(delta)=0:
sol = root_scalar(lambda x: diffReactFitCriterium(x, param),
                  method='brentq',bracket=[0,1], xtol=1e-15, rtol=1e-15)
q0 = sol.root
print(f"{q0 = }")

# Solve ODE once more such that we can plot the final data
sol = solve_ivp(lambda x, y: diffReactSystem(x, y,param),
                [0, param['delta']], [param['cAiL'], q0],
                t_eval = np.linspace(0, param['delta'], 101))
```

Postprocessing of the data can be done similar to the example in slide 499.

# BVP example: analytical solution

Compare with the analytical solution:

$$q = k_L E_A C_{A,i,L} \quad \text{with}$$

$$E_A = \frac{\mathrm{Ha}}{\tanh \mathrm{Ha}} \qquad \text{(Enhancement factor)}$$

$$\mathrm{Ha} = \frac{\sqrt{k_R \mathcal{D}}}{k_L} \qquad \text{(Hatta number)}$$

$$k_L = \frac{\mathcal{D}}{\delta} \qquad \text{(mass transfer coefficient)}$$

# Today's outline

- Introduction
  - Backward Euler
  - Implicit midpoint method
- Systems of ODEs
  - Solution methods for systems of ODEs
  - Solving systems of ODEs in Python
  - Stiff systems of ODEs
- Boundary value problems
  - Shooting method
- Conclusion

# Other methods

Other explicit methods:

- Bulirsch-Stoer method (Richardson extrapolation + modified midpoint method)

Other implicit methods:

- Rosenbrock methods (higher order implicit Runge-Kutta methods)
- Predictor-corrector methods

# Summary

- Several solution methods and their derivation were discussed:
  - Explicit solution methods: Euler, Improved Euler, Midpoint method, RK45
  - Implicit methods: Implicit Euler and Implicit midpoint method
  - A few examples of their spreadsheet implementation were shown
- We have paid attention to accuracy and instability, rate of convergence and step size
- Systems of ODEs can be solved by the same algorithms. Stiff problems should be treated with care.
- An example of solving ODEs with Python was demonstrated.

# Partial differential equations

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Instationary diffusion equation
  - Discretization
  - Solving the diffusion equation
  - Non-linear source terms
- Convection
  - Discretization
  - Central difference scheme
  - Upwind scheme
- Conclusions
  - Other methods
  - Summary

# Overview

## Main question

How to solve parabolic PDEs like:

$$\frac{\partial c}{\partial t} = \mathcal{D}\frac{\partial^2 c}{\partial x^2} - u\frac{\partial c}{\partial x} + R$$

with

$$\begin{aligned} t = 0; 0 \leq x \leq \ell \quad &\Rightarrow c = c_0 \\ t > 0; x = 0 \quad &\Rightarrow -\mathcal{D}\frac{\partial c}{\partial x} + uc = u_{\text{in}} c_{\text{in}} \\ t > 0; x = \ell \quad &\Rightarrow \frac{\partial c}{\partial x} = 0 \end{aligned}$$

accurately and efficiently?

# What is a PDE?

**Partial differential equation**

An equation containing a function and their derivatives to multiple independent variables.

**Order of PDE**

The highest derivative appearing in the PDE

General second order PDE:

$$A\frac{\partial^2 f}{\partial x^2} + B\frac{\partial^2 f}{\partial x \partial y} + C\frac{\partial^2 f}{\partial y^2} + D\frac{\partial f}{\partial x} + E\frac{\partial f}{\partial y} + Ff = G$$

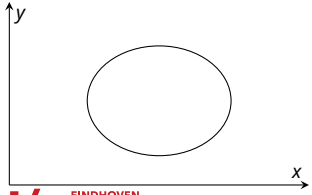
- Linear equation: Coefficients $A, B, \ldots, G$ do not depend on $x$ and $y$.
- Non-linear equation: Coefficients $A, B, \ldots, G$ are a function of $x$ and $y$.

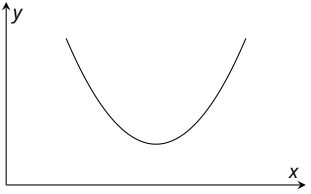
# Classification of PDE's

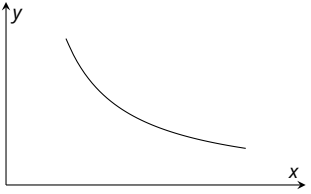
$$A\frac{\partial^2 f}{\partial x^2} + B\frac{\partial^2 f}{\partial x \partial y} + C\frac{\partial^2 f}{\partial y^2} + D\frac{\partial f}{\partial x} + E\frac{\partial f}{\partial y} + Ff = G$$

The discriminant $\Delta$ of a quadratic polynomial is computed as (note: only the higher order coefficients are important):
$\Delta = B^2 - 4AC$

- $\Delta < 0 \Rightarrow$ Elliptic equation
  (e.g. Laplace equation for stationary diffusion in 2D)
- $\Delta = 0 \Rightarrow$ Parabolic equation
  (e.g. instationary heat penetration in 1D)
- $\Delta > 0 \Rightarrow$ Hyperbolic equation
  (e.g. wave equation)



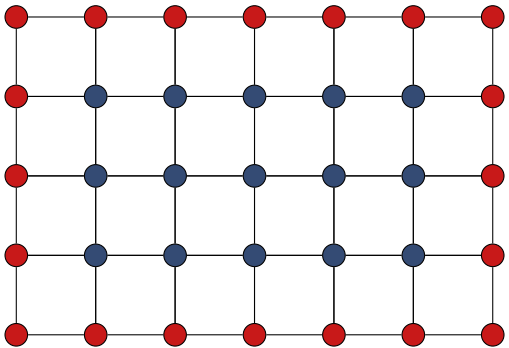



# Importance of classification

Different PDE types require different solution techniques because of the difference in range of influence:

- *Characteristics*
  Curves in $xy$-domain along with signal propagation takes place
- *Domain of dependence of point $P$*
  points in $xy$-domain which influence the value of $f$ in point $P$
- *Range of influence of point $P$*
  points in $xy$-domain which are influenced by the value of $f$ in point $P$

# Example elliptic PDE (boundary value problems: BVP)



Grid point at which dependent variable has to be computed

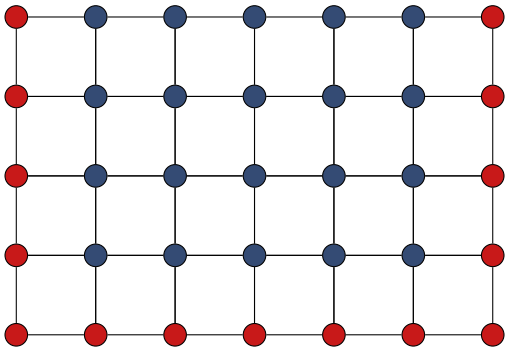
Grid point at which boundary condition is specified

Typical example: Poisson equation

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = f(x,y)$$

Efficiency (memory requirements, CPU time) of the numerical method is of crucial importance.

# Example parabolic PDE (initial value problem: IVP)



Grid point at which dependent variable has to be computed

Grid point at which boundary condition is specified

Typical example: Poisson equation

$$\frac{\partial c}{\partial t} + u\frac{\partial c}{\partial x} = \mathcal{D}\frac{\partial^2 c}{\partial x^2} + R$$

Stability (in numerical sense) of the numerical method is of crucial importance.

# Boundary conditions

- Dirichlet or fixed condition: prescribed value of $f$ at boundary

  $f = f_0$ $f_0$ is a known function

- Neumann condition: prescribed value of derivative of $f$ at boundary

  $\frac{\partial f}{\partial n} = q$ $q$ is a known function

- Mixed or Robin condition: relation between $f$ and $\frac{\partial f}{\partial n}$ at boundary

  $a\frac{\partial f}{\partial n} + bf = c$ $a$, $b$ and $c$ are known functions

# Numerical solution method

Finite differences (method of lines, MOL):

1. Discretize spatial domain in discrete grid points
2. Find suitable approximation for the spatial derivatives
3. Substitute approximations in PDE, which gives a system of ODE's, one for every grid points
4. Advance in time with a suitable ODE solver

Alternative methods: collocation, Galerkin or Finite Element methods

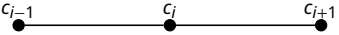
# Today's outline


- Introduction
- Instationary diffusion equation
  - Discretization
  - Solving the diffusion equation
  - Non-linear source terms
- Convection
  - Discretization
  - Central difference scheme
  - Upwind scheme
- Conclusions
  - Other methods
  - Summary

# Instationary diffusion equation (Fick's second law)

$$\frac{\partial c}{\partial t} = \mathcal{D}\frac{\partial^2 c}{\partial x^2}, \quad \text{with} \quad \begin{aligned} &t = 0; 0 \le x \le \ell \Rightarrow c = c_0 \\ &t > 0; x = 0 \Rightarrow c = c_L \\ &t > 0; x = \ell \Rightarrow c = c_R \end{aligned}$$

Second derivative $\dfrac{\partial^2 c}{\partial x^2}$




$$c_{i+1} = c_i + \left.\frac{\partial c}{\partial x}\right|_i \Delta x + \frac{1}{2}\left.\frac{\partial^2 c}{\partial x^2}\right|_i \Delta x^2 + \frac{1}{6}\left.\frac{\partial^3 c}{\partial x^3}\right|_i \Delta x^3 + \ldots$$

$$c_{i-1} = c_i - \left.\frac{\partial c}{\partial x}\right|_i \Delta x + \frac{1}{2}\left.\frac{\partial^2 c}{\partial x^2}\right|_i \Delta x^2 - \frac{1}{6}\left.\frac{\partial^3 c}{\partial x^3}\right|_i \Delta x^3 + \ldots$$

---

$$c_{i+1} + c_{i-1} = 2c_i + \left.\frac{\partial^2 c}{\partial x^2}\right|_i \Delta x^2 + \mathcal{O}(\Delta x^4)$$

$$\Rightarrow \left.\frac{\partial^2 c}{\partial x^2}\right|_i = \frac{c_{i+1} - 2c_i + c_{i-1}}{\Delta x^2} + \mathcal{O}(\Delta x^2)$$

Due to symmetric discretization: second order (central discretization).

# Instationary diffusion equation (Fick's second law)

An alternative discretization:

$$\left.\frac{\partial^2 c}{\partial x^2}\right|_i = \frac{\left.\frac{\partial c}{\partial x}\right|_{i+\frac{1}{2}} - \left.\frac{\partial c}{\partial x}\right|_{i-\frac{1}{2}}}{\Delta x} + \mathcal{O}(\Delta x^2)$$




$$= \frac{\frac{c_{i+1}-c_i}{\Delta x} - \frac{c_i - c_{i-1}}{\Delta x}}{\Delta x} = \frac{c_{i+1} - 2c_i + c_{i-1}}{\Delta x^2}$$

This is convenient for the derivation of $\frac{\partial}{\partial x}\left(\mathcal{D}\frac{\partial c}{\partial x}\right)$:

$$\frac{\partial}{\partial x}\left(\mathcal{D}\frac{\partial c}{\partial x}\right) = \frac{\mathcal{D}_{i+\frac{1}{2}}\left.\frac{\partial c}{\partial x}\right|_{i+\frac{1}{2}} - \mathcal{D}_{i-\frac{1}{2}}\left.\frac{\partial c}{\partial x}\right|_{i-\frac{1}{2}}}{\Delta x} = \frac{\mathcal{D}_{i+\frac{1}{2}}\frac{c_{i+1}-c_i}{\Delta x} - \mathcal{D}_{i-\frac{1}{2}}\frac{c_i - c_{i-1}}{\Delta x}}{\Delta x}$$

$$= \frac{\mathcal{D}_{i+\frac{1}{2}}c_{i+1} - \left(\mathcal{D}_{i+\frac{1}{2}} + \mathcal{D}_{i-\frac{1}{2}}\right)c_i + \mathcal{D}_{i-\frac{1}{2}}c_{i-1}}{(\Delta x)^2}$$

# Instationary diffusion equation (Fick's second law)

$\frac{\partial^2 f}{\partial x^2}$




$$f_{i+\frac{1}{2}} = f_i + \frac{1}{2}\Delta x \left.\frac{\partial f}{\partial x}\right|_i \Delta x + \frac{1}{2}\left(\frac{1}{2}\Delta x\right)^2 \left.\frac{\partial^2 f}{\partial x^2}\right|_i + \mathcal{O}(\Delta x^3)$$

$$f_{i-\frac{1}{2}} = f_i - \frac{1}{2}\Delta x \left.\frac{\partial f}{\partial x}\right|_i \Delta x + \frac{1}{2}\left(\frac{1}{2}\Delta x\right)^2 \left.\frac{\partial^2 f}{\partial x^2}\right|_i + \mathcal{O}(\Delta x^3)$$

---

$$f_{i+\frac{1}{2}} - f_{i-\frac{1}{2}} = \Delta x \frac{\partial f}{\partial x} + \mathcal{O}(\Delta x^3)$$

$$\Rightarrow \left.\frac{\partial f}{\partial x}\right|_i = \frac{f_{i+\frac{1}{2}} - f_{i-\frac{1}{2}}}{\Delta x} + \mathcal{O}(\Delta x^2)$$

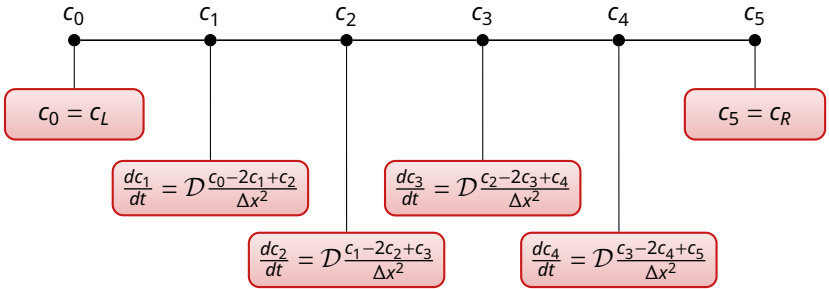
Symmetric discretization yields second order!

# Instationary diffusion equation: spatial discretization

Substitution of spatial derivatives yields:

$$\frac{dc_i}{dt} = \mathcal{D}\frac{c_{i-1} - 2c_i + c_{i+1}}{\Delta x^2} \quad \text{for } i = 0, \ldots, N$$

For example, using 6 (ridiculously low number!) grid points:



# Instationary diffusion equation: boundary conditions

Two options:

1. Keep boundary conditions as additional equations:

$$c_0 = c_L, \frac{dc_1}{dt} = \mathcal{D}\frac{c_0 - 2c_1 + c_2}{\Delta x^2}, \frac{dc_2}{dt} = \mathcal{D}\frac{c_1 - 2c_2 + c_3}{\Delta x^2},$$
$$\frac{dc_3}{dt} = \mathcal{D}\frac{c_2 - 2c_3 + c_4}{\Delta x^2}, \frac{dc_4}{dt} = \mathcal{D}\frac{c_3 - 2c_4 + c_5}{\Delta x^2}, c_5 = c_R$$

2. Substitute boundary conditions to reduce number of equations:

$$\frac{dc_1}{dt} = \mathcal{D}\frac{c_L - 2c_1 + c_2}{\Delta x^2}, \frac{dc_2}{dt} = \mathcal{D}\frac{c_1 - 2c_2 + c_3}{\Delta x^2},$$
$$\frac{dc_3}{dt} = \mathcal{D}\frac{c_2 - 2c_3 + c_4}{\Delta x^2}, \frac{dc_4}{dt} = \mathcal{D}\frac{c_3 - 2c_4 + c_R}{\Delta x^2}$$

# Instationary diffusion equation: temporal discretization

$$\frac{dc_i}{dt} = \mathcal{D}\frac{c_{i-1} - 2c_i + c_{i+1}}{\Delta x^2}$$

## Time discretization: forward Euler (explicit)

$$\frac{c_i^{n+1} - c_i^n}{\Delta t} = \mathcal{D}\frac{c_{i-1}^n - 2c_i^n + c_{i+1}^n}{\Delta x^2}$$

$$\Rightarrow c_i^{n+1} = \mathrm{Fo}c_{i-1}^n + (1 - 2\mathrm{Fo})c_i^n + \mathrm{Fo}c_{i+1}^n \quad \text{with Fo} = \frac{\mathcal{D}\Delta t}{\Delta x^2}$$

Straightforward updating (explicit equation), simple to implement in a program but stability constraint Fo $= \frac{\mathcal{D}\Delta t}{\Delta x^2} < \frac{1}{2}$!

Small $\Delta x \Rightarrow$ small $\Delta t \Rightarrow$ patience required ☹

# Instationary diffusion equation: temporal discretization

$$\frac{dc_i}{dt} = \mathcal{D}\frac{c_{i-1} - 2c_i + c_{i+1}}{\Delta x^2}$$

## Time discretization: backward Euler (implicit)

$$\frac{c_i^{n+1} - c_i^n}{\Delta t} = \mathcal{D}\frac{c_{i-1}^{n+1} - 2c_i^{n+1} + c_{i+1}^{n+1}}{\Delta x^2}$$

$$\Rightarrow -\mathrm{Fo}c_{i-1}^{n+1} + (1 + 2\mathrm{Fo})c_i^{n+1} - \mathrm{Fo}c_{i+1}^{n+1} = c_i^n \quad \text{with Fo} = \frac{\mathcal{D}\Delta t}{\Delta x^2}$$

Requires the solution of a system of linear equations, but no stability constraints!

Note: extension to higher order schemes (with time step adaptation) straightforward. Often second or third order optimal, because for each Euler-like step in the additional order an often large system needs to be solved (not treated in this course).

# Solving the instationary diffusion equation: example

Solve the diffusion problem using explicit discretization:

$$\frac{\partial c_i}{\partial t} = \mathcal{D}\frac{\partial^2 c}{\partial x^2} \quad \text{with}$$

$0 \leq x \leq \delta$, $\delta = 5 \cdot 10^{-3}$ m
$\delta/\Delta x = 100$ grid cells
$\mathcal{D} = 1 \cdot 10^{-8}$ m$^2$ s$^{-1}$
$t_{\text{end}} = 5000$ s
$c_{\text{L}} = 1$ mol m$^{-3}$ $c_{\text{R}} = 0$ mol m$^{-3}$

$$c_i^{n+1} = \text{Fo}c_{i-1}^n + (1 - 2\text{Fo})c_i^n + \text{Fo}c_{i+1}^n \quad \text{with Fo} = \frac{\mathcal{D}\Delta t}{\Delta x^2}$$

1. Initialise variables
2. Compute time step so that $\text{Fo} \leq \frac{1}{2} \Rightarrow \Delta t = 0.125$s!
3. Compute 40000 time steps times 100 grid nodes!
4. Store solution

# Solving the instationary diffusion equation: example

Initialise the variables and matrices:

```
import numpy as np

Nx = 100 # Nx grid points
Nt = 40000 # Nt time steps
D = 1e-8 # m/s
c_L = 1.0; c_R = 0 # mol/m3
t_end = 5000.0 # s
x_end = 5e-3 # m

# Time step and grid size
dt = t_end / Nt
dx = x_end / Nx

# Fourier number
Fo = D * dt / dx / dx

# Initial matrices for solutions (Nx times Nt)
c = np.zeros((Nt + 1, Nx + 1)) # All concentrations are zero
c[:, 0] = c_L # Concentration at the left side
c[:, Nx] = c_R # Concentration at the right side

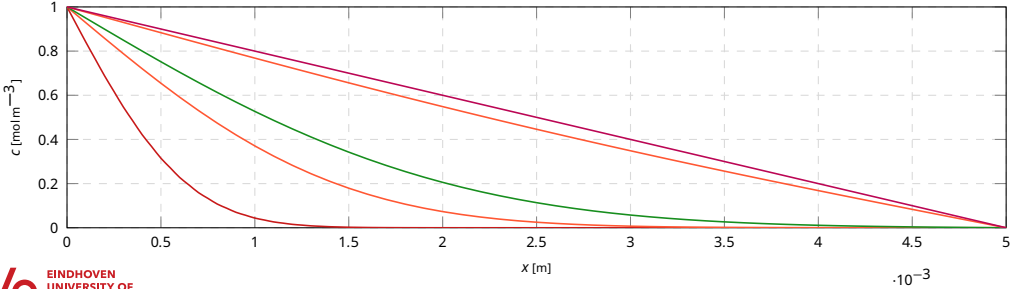
# Grid node and time step positions
x = np.linspace(0, x_end, Nx + 1)
```

# Solving the instationary diffusion equation: example

Compute the solution (nested time-and-grid loop):

- Create a time-loop
- Create a loop over *internal* grid points
- Update each node using $c_i^{n+1} = \mathrm{Fo} c_{i-1}^n + (1 - 2\mathrm{Fo}) c_i^n + \mathrm{Fo} c_{i+1}^n$
- Plot the solution for selected time steps

Plotting the solution at $t = \{12.5, 62.5, 125, 625, 5000\}$ s.



# Solving the instationary diffusion equation: example

A double-loop can impose serious computation times if the number of grid points increases:

```
for n in range(Nt - 1): # time loop
   for i in range(1, Nx): # Nested loop for grid nodes
      c[n+1, i] = Fo * c[n, i-1] + (1 - 2*Fo) * c[n, i] + Fo * c[n, i+1]
```

Remedy: vectorization. Construct a 3-point stencil Laplacian matrix first, then use the matrix product to evolve the simulation:

```
from scipy.sparse import diags

# Construct sparse matrix
e = np.ones(Nx-1)
md = np.concatenate(([1], (1 - 2 * Fo) * e, [1]))
ld = np.concatenate((Fo * e, [0]))
ud = np.concatenate(([0], Fo * e))
A = diags([ld, md, ud], offsets=[-1, 0, 1])

# Time evolution
for n in range(Nt - 1): # time loop
   c[n+1, :] = A.dot(c[n,:])
```

# Solving the diffusion equation implicitly

Linear system $A\boldsymbol{x} = \boldsymbol{b}$ from $-\mathrm{Fo}c_{i-1}^{n+1} + (1+2\mathrm{Fo})c_i^{n+1} - \mathrm{Fo}c_{i+1}^{n+1} = c_i^n$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & \cdots & 0 \\ -\mathrm{Fo} & (1+2\mathrm{Fo}) & -\mathrm{Fo} & 0 & \cdots & 0 \\ 0 & -\mathrm{Fo} & (1+2\mathrm{Fo}) & -\mathrm{Fo} & \cdots & 0 \\ 0 & 0 & -\mathrm{Fo} & (1+2\mathrm{Fo}) & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 1 \end{pmatrix} \cdot \begin{pmatrix} c_0^{n+1} \\ c_1^{n+1} \\ c_2^{n+1} \\ c_3^{n+1} \\ \vdots \\ c_m^{n+1} \end{pmatrix} = \begin{pmatrix} c_0^n \\ c_1^n \\ c_2^n \\ c_3^n \\ \vdots \\ c_m^n \end{pmatrix}$$

$1 \times c_0^{n+1} = c_0^n$ (boundary condition)

$-\mathrm{Fo}c_0^{n+1} + (1+2\mathrm{Fo})c_1^{n+1} - \mathrm{Fo}c_2^{n+1} = c_1^n$

$-\mathrm{Fo}c_1^{n+1} + (1+2\mathrm{Fo})c_2^{n+1} - \mathrm{Fo}c_3^{n+1} = c_2^n$

$-\mathrm{Fo}c_2^{n+1} + (1+2\mathrm{Fo})c_3^{n+1} - \mathrm{Fo}c_4^{n+1} = c_3^n$

$1 \times c_m^{n+1} = c_m^n$ (boundary condition)

# Solving the diffusion equation implicitly in Python

To solve the linear system, we need to define matrix $A$. It is clear that storing many zeros is not efficient in terms of memory. We use a *sparse matrix* format. Two alternative ways to set up the matrix:

Set individual elements of the matrix:

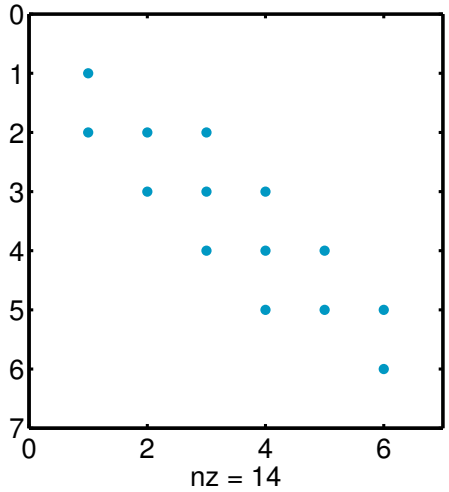
- Loop over the internal cells
- Set the coefficients in matrix `A` (main diagonal + elements left/right to it)
- Then set the coefficients for the boundary cells

Set matrix using bands:

- Consider the matrix structure (previous slide) and create vectors containing the values in each band
- Recall the `sp.sparse.diags` function to set entire bands to a sparse matrix

# Solving the diffusion equation implicitly in Python

The command `plt.spy(A)` shows a figure with the non-zero positions.



# Solving the diffusion equation implicitly in Python

The concentration matrix is initialised and the boundary conditions are set as follows:

```
# Initial matrices for solutions (Nx times Nt)
c = np.zeros((Nt+1, Nx+1)) # All concentrations are zero
c[:, 0] = c_L # Concentration at left side
c[:, Nx] = c_R # Concentration at right side
```

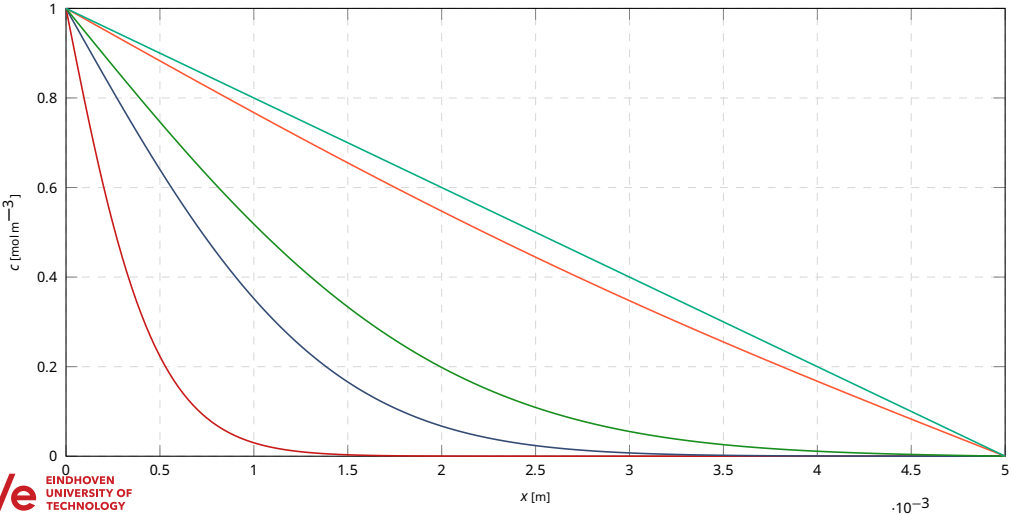
The right hand side vector (***b***) can now be set during the time-loop:

```
from scipy.sparse.linalg import spsolve

for n in range(Nt-1): # time loop
   b = c[n, :] # Set right hand side
   solX = spsolve(A, b) # Solve linear system
   c[n+1, :] = solX # Store solution each time step
```

# Solving the diffusion equation implicitly in Matlab

Plotting the solution at $t = \{12.5, 62.5, 125, 625, 5000\}$ s.



# About explicit vs. implicit solutions

- Explicit solution:
  - Easy to implement
  - Very small time steps required.
  - This problem took about 0.5 s.
- Implicit solution:
  - Harder to implement, needs sparse matrix solver
  - No stability constraint
  - This problem took about 0.05 s
- The difference will become much larger for systems with e.g. more grid nodes!

# Extension with non-linear source terms

$$\frac{\partial c}{\partial t} = \mathcal{D}\frac{\partial^2 c}{\partial x^2} + R(c) \quad \text{with} \quad \begin{aligned} t = 0; 0 \le x \le \ell &\Rightarrow c = c_0 \\ t > 0; x = 0 &\Rightarrow c = c_L \\ t > 0; x = \ell &\Rightarrow c = c_R \end{aligned}$$

- Forward Euler (explicit): simply add to right-hand side

$$\frac{c_i^{n+1} - c_i^n}{\Delta t} = \mathcal{D}\frac{c_{i-1}^n - 2c_i^n + c_{i+1}^n}{\Delta x^2} + R(c_i^n)$$

$$\Rightarrow c_i^{n+1} = \mathrm{Fo} c_{i-1}^n + (1 - 2\mathrm{Fo}) c_i^n + \mathrm{Fo} c_{i+1}^n + R_i^n \Delta t$$

- Backward Euler (implicit): linearization required

$$R(c_i^{n+1}) = R(c_i^n) + \left.\frac{dR}{dc}\right|_i^n (c_i^{n+1} - c_i^n)$$

$$\frac{c_i^{n+1} - c_i^n}{\Delta t} = \mathcal{D}\frac{c_{i-1}^{n+1} - 2c_i^{n+1} + c_{i+1}^{n+1}}{\Delta x^2} + R(c_i^{n+1})$$

$$\Rightarrow -\mathrm{Fo} c_{i-1}^{n+1} + (1 + 2\mathrm{Fo} - \left.\frac{dR}{dc}\right|_i^n \Delta t) c_i^{n+1} - \mathrm{Fo} c_{i+1}^{n+1} = c_i^n + \left(R_i^n - \left.\frac{dR}{dc}\right|_i^n c_i^n\right)\Delta t$$

# Today's outline

- Introduction
- Instationary diffusion equation
  - Discretization
  - Solving the diffusion equation
  - Non-linear source terms
- Convection
  - Discretization
  - Central difference scheme
  - Upwind scheme
- Conclusions
  - Other methods
  - Summary

# Extension with convection terms

$$\frac{\partial c}{\partial t} = \mathcal{D}\frac{\partial^2 c}{\partial x^2} - u\frac{\partial c}{\partial x} + R$$

Discretization of first derivative $\frac{dc}{dx}$,
looks simple but is numerical headache!

Central discretization:

$$\frac{dc}{dx} = \frac{c_{i+1} - c_{i-1}}{2\Delta x}$$

$\Rightarrow$ simple and easy, too bad it doesn't work: yields unstable solutions if convection dominated.

# Central difference scheme of 1st derivative

Unsteady convection:

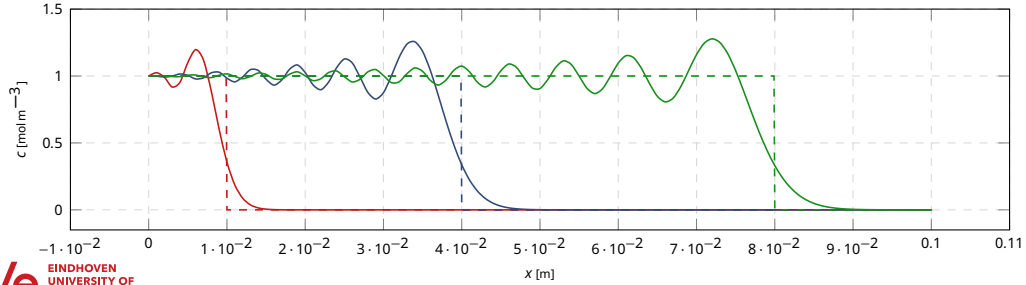
$$\frac{\partial c}{\partial t} = -u\frac{\partial c}{\partial x}$$

Central difference for first derivative:

$$\frac{dc}{dx} = \frac{c_{i+1} - c_{i-1}}{2\Delta x}$$

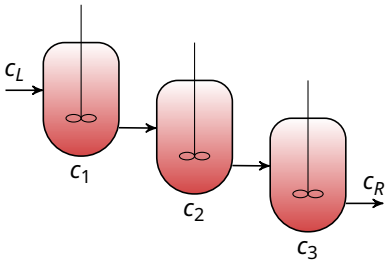
Forward Euler discretization of temporal and spatial domain:

$$\frac{c_i^{n+1} - c_i^n}{\Delta t} = -u\frac{c_{i+1} - c_{i-1}}{2\Delta x} \Rightarrow c_i^{n+1} = c_i^n - u\frac{c_{i+1}^n - c_{i-1}^n}{2\Delta x}\Delta t$$



# Extension with convection terms

Solution: upwind discretization, like CSTR's in series:




First order upwind: $-u\left.\frac{dc}{dx}\right|_i = \begin{cases} -u\frac{c_i - c_{i-1}}{\Delta x} & \text{if } u \geq 0 \\ -u\frac{c_{i+1} - c_i}{\Delta x} & \text{if } u < 0 \end{cases}$ Stable if $\text{Co} = \frac{u\Delta t}{\Delta x} < 1$ (with Co the Courant number). However, only 1$^{\text{st}}$ order accurate (large smearing of concentration fronts). Higher order upwind requires TVD schemes (trick of the trade)...

# First order upwind scheme of 1st derivative

Unsteady convection:

$$\frac{\partial c}{\partial t} = -u\frac{\partial c}{\partial x}$$

Upwind scheme for first derivative:

$$-u\left.\frac{dc}{dx}\right|_i = \begin{cases} -u\dfrac{c_i - c_{i-1}}{\Delta x} & \text{if } u \geq 0 \\ -u\dfrac{c_{i+1} - c_i}{\Delta x} & \text{if } u < 0 \end{cases}$$

Forward Euler discretization of temporal and spatial domain:

$$\frac{c_i^{n+1} - c_i^n}{\Delta t} = -u\frac{c_{i+1} - c_{i-1}}{2\Delta x}$$

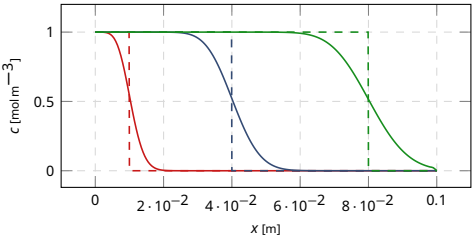
$$\Rightarrow c_i^{n+1} = \begin{cases} c_i^n - u\Delta t\dfrac{c_i - c_{i-1}}{\Delta x} & \text{if } u \geq 0 \\ c_i^n - u\Delta t\dfrac{c_{i+1} - c_i}{\Delta x} & \text{if } u < 0 \end{cases}$$

# Upwind scheme: example

Unsteady convection through a pipe:

$$\frac{\partial c}{\partial t} = -u\frac{\partial c}{\partial x} \quad \text{with} \quad u = 0.1\,\mathrm{m\,s^{-1}} \Rightarrow c_i^{n+1} = c_i^n - u\frac{c_i - c_{i-1}}{\Delta x}\Delta t$$

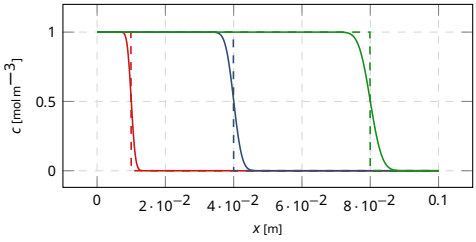
Using 100 grid cells

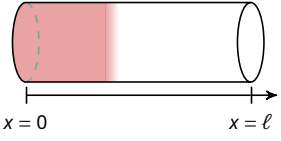






Using 1000 grid cells



# Today's outline

- Introduction
- Instationary diffusion equation
  - Discretization
  - Solving the diffusion equation
  - Non-linear source terms
- Convection
  - Discretization
  - Central difference scheme
  - Upwind scheme
- Conclusions
  - Other methods
  - Summary

# Extension to systems of PDE's

- Explicit methods: straightforward extension
- Implicit methods: yields block-tridiagonal matrix (note ordering of equations: all variables per grid cell)

# Extension to 2D or 3D systems

Spatial discretization in 2 directions — different methods available:

- Explicit
- Fully implicit
  - 1D gives tri-diagonal matrix
  - 2D gives penta-diagonal matrix
  - 3D gives hepta-diagonal matrix

  Use of dedicated matrix solvers (e.g. ICCG, multigrid, ...)
- Alternating direction implicit (ADI)
  - Per direction implicit, but still overall unconditionally stable

# Further extensions for parabolic PDEs

- Higher order temporal discretization (multi-step) with time step adaptation
- Non-uniform grids with automatic grid adaptation
- Higher-order discretization methods, especially higher order TVD (flux delimited) schemes for convective fluxes (e.g. WENO schemes)
- Higher-order finite volume schemes (Riemann solvers)

# Summary

- Several classes of PDEs were introduced
  - Elliptic, Parabolic, Hyperbolic PDEs
- Diffusion equation: discretization of temporal and spatial domain was discussed
  - Solutions of the diffusion equation using explicit and implicit methods
  - How to add non-linear source terms
- Convection: upwind vs. central difference schemes

# Curve fitting, regression and optimization

Dr.ir. Ivo Roghair, Prof.dr.ir. Martin van Sint Annaland

Complex Fluids and Interfaces group
Eindhoven University of Technology

Numerical Methods (6BER03), 2025-2026

# Today's outline


- Introduction
- Curve fitting
- Regression
- Fitting numerical models
- Optimization
- Linear programming
- Summary

# Overview

- We are going to fit measurements to models today
- You will also learn what $R^2$ actually means
- We get introduced to constrained and unconstrained optimization.
- We will use the simplex method to solve linear programming problems (LP)

# Today's outline


- Introduction
- Curve fitting
- Regression
- Fitting numerical models
- Optimization
- Linear programming
- Summary

# Let’s do an ’experiment’ to gather data

The following function can be found in the `poly_regression.py` script (see git repository).

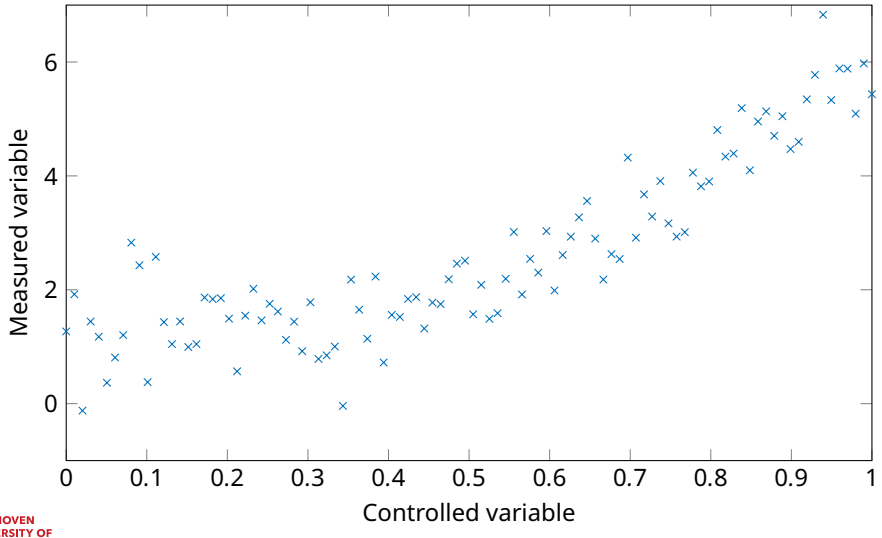
```
def generate_random_data(N=101,p=[1,1/3,1.5,3.5],draw=False):
   # Generate linear space of control points
   # N - Number of data points
   # p - Coefficients of polynomial
   x = np.linspace(0, 1, N) # Points (independent variable)

   # Generate 'measurement values' with errors following a normal distribution
   # Initialize randomizer
   pd = norm(loc=0, scale=0.1)
   # Add scatter data to the polynomial
   y = np.polyval(p,x) + pd.rvs(size=N)

   # Plot the generated data
   if draw:
      plt.plot(x, y, 'x')
      plt.show()
   return x,y
```

Gather some data by calling the function and storing `x` and `y`

# Fitting models to data



# How to fit a model to the data?

We would like to fit the following model to the data:

$$\hat{y} = a_0 x^3 + a_1 x^2 + a_2 x + a_3$$

First attempt - using the `polyfit` function we have seen with the interpolation lecture:

```
def fit_using_polyfit(x,y,n=2,draw=False):
    p = np.polyfit(x,y,n)
    if draw:
        plt.plot(x, y, 'x')
        plt.plot(x, np.polyval(p,x), '-')
        plt.show()
    return p
```

If we print `p`, we get the coefficients. But this is a black box, what does it do?

# How to fit a model to the data?

We would like to fit the following model to the data:

$$\hat{y} = a_0 x^3 + a_1 x^2 + a_2 x + a_3$$

Attempt to solve a linear system: If we have $N$ data points, we could write the model as the product of a matrix and a vector:

$$\begin{bmatrix} \hat{y}_0 \\ \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_{N-1} \end{bmatrix} = \begin{bmatrix} x_0^3 & x_0^2 & x_0 & 1 \\ x_1^3 & x_1^2 & x_1 & 1 \\ x_2^3 & x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ x_{N-1}^3 & x_{N-1}^2 & x_{N-1} & 1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

$$\hat{y} = Xa$$

$X$ is called the design matrix and $a$ are the fit parameters.

# Residuals

Second step: work out the residuals for each data point:

$$d_i = (y_i - \hat{y}_i)$$

Third step: work out the sum of squares of the residuals:

$$\text{SSE} = \sum_i d_i^2 = \sum_i (y_i - \hat{y}_i)^2$$

This can be written using the dot-product operation:

$$\text{SSE} = \sum_i d_i^2 = d \cdot d = d^T \cdot d = (y_i - \hat{y}_i)^T \cdot (y_i - \hat{y}_i)$$

# Minimizing the sum of squares

Choose the parameter vector such that the sum of squares of the residuals is minimized; the partial derivative with respect to each parameter should be set to zero:

$$\frac{\partial}{\partial a_i}\left[\left(y-(Xa)^T\right)(y-Xa)\right]$$

In Python, we can solve our linear system $\hat{Y} = Xa$ simply by running `a = np.linalg.solve(X,y)`.

- If there are more data points ($N > 4$), we can write an analogue, but maybe a consistent solution does not exist (the system is over specified).
- However, Python will find values for the vector a which minimize $\left\|y - aX\right\|^2$ , so i.e. a least squares fit.

# Example: fitting a linear model using least-squares

Assume the model is given as:

$$\hat{y} = ax + b$$

Then the error is computed as the sum of squares:

$$E(a,b) = \sum_{i=0}^{n-1} (y_i - \hat{y}_i)^2 = \sum_{i=0}^{n-1} (y_i - (ax_i + b))^2$$

The partial derivatives with respect to $a$ and $b$ are:

$$\frac{\partial E}{\partial a} = -2 \sum_{i=0}^{n-1} x_i (y_i - (ax_i + b))$$

$$\frac{\partial E}{\partial b} = -2 \sum_{i=0}^{n-1} (y_i - (ax_i + b))$$

# Example: fitting a linear model using least-squares

Setting the partial derivatives to zero gives the following system of equations:

$$\begin{bmatrix} \sum_i x_i^2 & \sum_i x_i \\ \sum_i x_i & N \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_i x_i y_i \\ \sum_i y_i \end{bmatrix}$$

Exercise: make a program that takes $x$ and $y$ as input and returns the coefficients $a$ and $b$.

- Either set up and solve the linear system
- Or substitute equations/use Cramer's rule to obtain:

$$a = \frac{n\sum_i x_i y_i - \sum_i x_i \sum_i y_i}{n\sum_i x_i^2 - (\sum_i x_i)^2}, \; b = \frac{\sum_i y_i - a\sum_i x_i}{n}$$

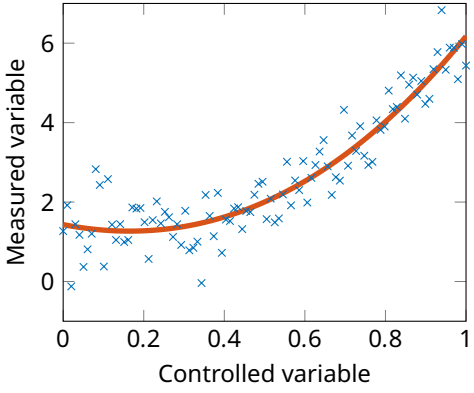
- Try for yourself to derive the equations for a second degree polynomial model
- Create a program that implements a generic polynomial model of degree $n$ using:

$$\begin{bmatrix} \sum_i x_i^{2n} & \sum_i x_i^{2n-1} & \cdots & \sum_i x_i^n \\ \sum_i x_i^{2n-1} & \sum_i x_i^{2n-2} & \cdots & \sum_i x_i^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_i x_i & \sum_i x_i & \cdots & N \end{bmatrix} \begin{bmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_0 \end{bmatrix} = \begin{bmatrix} \sum_i x_i^n y_i \\ \sum_i x_i^{n-1} y_i \\ \vdots \\ \sum_i y_i \end{bmatrix}$$

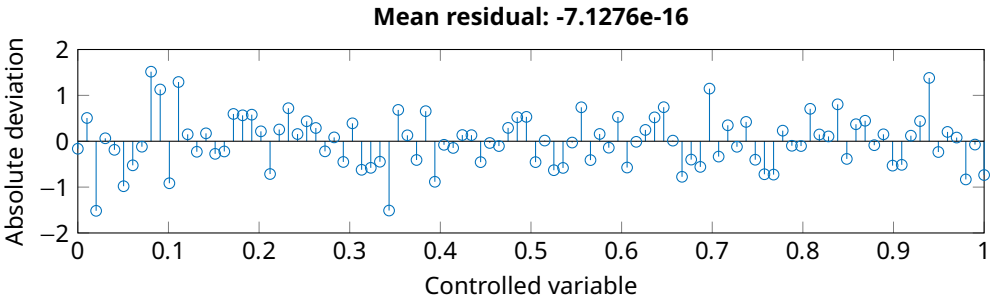
# Fitting our problem: Least squares solver

As a follow-up of the script provided in slide 554

```
def fit_using_lstsqr(x,y,n=2,draw=False):
    xmat = np.vander(x,n+1)
    sol = np.linalg.lstsq(xmat,y,rcond=None)
    A = sol[0]
    yhat = xmat@A
    if draw:
        plt.plot(x,y,'x')
        plt.plot(x,yhat,'-')
        plt.title('Fit using lstsq')
        plt.show()
    return A,yhat
```



# How good is the model?




- For a model to make sense the data points should be scattered randomly around the model predictions, the mean of the residuals $d$ should be zero: $d_i = (y_i - \hat{y}_i)$
- It's always good to check if the residuals are not correlated with the measured values, if that is the case, it can indicate that your model is wrong.

# Today's outline


- Introduction
- Curve fitting
- Regression
- Fitting numerical models
- Optimization
- Linear programming
- Summary

# Regression coefficients

- Variance measured in the data (y) is:

$$\sigma_y^2 = \frac{1}{N}\sum_i (y_i - \bar{y})^2$$

- Variance of the residuals is:

$$\sigma_{\text{error}}^2 = \frac{1}{N}\sum_i (d_i)^2$$

- Variance in the model is:

$$\sigma_{\text{model}}^2 = \frac{1}{N}\sum_i \left(\hat{y}_i - \bar{\hat{y}}\right)^2$$

# Regression coefficients

Given that the error is uncorrelated we can state that:

$$\sigma_y^2 = \sigma_{\text{error}}^2 + \sigma_{\text{model}}^2$$

$$R^2 = \frac{\sigma_{\text{model}}^2}{\sigma_y^2} = 1 - \frac{\sigma_{\text{error}}^2}{\sigma_y^2}$$

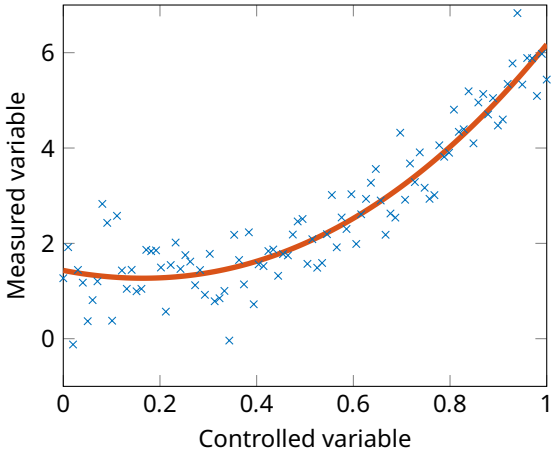
$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}}$$

- SSE: Sum of errors (residuals) squared (difference between data and model)
- SST: Total sum of squares (variance of the data)
- SSR: Sum of squares (model)

# Back to the example

The statistics:

| | Value |
|---|---|
| $N$ | 100 |
| SSE | 32.042 |
| SST | 896.907 |
| SSR | 928.950 |
| $R^2$ | 0.964 |



# Today's outline


- Introduction
- Curve fitting
- Regression
- Fitting numerical models
- Optimization
- Linear programming
- Summary

# Curve fitting: scipy.optimize.curve_fit

Python offers various non-linear parameter and curve fitting tools. The function `curve_fit` from the `scipy.optimize` module allows to fit a model to a given dataset. Again, based on the data generated in slide 554:

```
from poly_regression import generate_random_data
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt
import numpy as np

# Define the model function
def curve_fit_model_1(xdata, a0, a1, a2, a3):
    return a0*xdata**3 + a1*xdata**2 + a2*xdata + a3
```

```
if __name__ == '__main__':
    x,y = generate_random_data()
    a0 = [1, 2, 1, 3] # Initial guess of coefficients

    # Perform fitting, store resulting coeffs in a_fit
    a_fit, _ = curve_fit(curve_fit_model_1, x, y, p0=a0)

    # Run the model once more, with fitted coefficients
    y_model = curve_fit_model_1(x, *a_fit) # unpack coefficients using *
```

# Curve fitting from command line: scipy.optimize.curve_fit

For fitting a polynomial model, this function works as well as **polyfit**. But it also allows other (much more complex) types of model to be defined:

```
def curve_fit_model_2(xdata, a0, a1, a2):
    return a0*np.exp(a1*xdata) + a2

# Initial guess of coefficients
a0 = [1,1,1]

# Perform fitting of the data to another model
a_fit, _ = curve_fit(curve_fit_model_2, x, y, p0=a0)

# Run the model once more, with fitted coefficients
y_model = curve_fit_model_2(x, *a_fit)
```

The model functions take individual parameters separately, we use the unpacking syntax (`*`) to pass the fitted parameters when generating the `y_model` data.

# Dynamic fitting of non-linear equations

You may encounter situations where the model data is slightly more complicated to obtain (e.g., a numerical model based on ODEs where coefficients are unknown), or you want to perform fitting of multiple functions/coefficients, or just want to automate things via scripts. Python's Scipy library gives access to powerful functions such as `least_squares` and `curve_fit`.

# General use of scipy.optimize.least_squares

```
from scipy.optimize import least_squares

result = least_squares(fun, k0, bounds=(lb, ub), xtol=1.0E-6, max_nfev=1000)
```

- `fun` is a function handle to the fit criterion (e.g., `myFitCrit`). The fit criterion function `myFitCrit` should return the residuals vector, e.g., $d_i = (y_i - \hat{y}_i)$. Here, $y_i$ would again be the measurement data and $\hat{y}$ the solution computed by a model.
- `k0` is the initial guess for the fitting coefficient (or: array of initial guesses when fitting multiple coefficients).
- `lb` and `ub` are the lower and upper boundaries for `k0`. These should both be the size of the `k0`-array.
- Use arguments such as `xtol` and `max_nfev` for more fine-grained control on the fit procedure.

# General use of scipy.optimize.curve_fit

```
from scipy.optimize import curve_fit

popt, pcov = curve_fit(fun, xdata, ydata, p0=k0, bounds=(lb, ub))
```

- `fun` is the model function that you want to fit to your data. It takes the independent variable as the first argument and the parameters to fit as separate remaining arguments.
- `xdata` and `ydata` are the data points that you are fitting the model function to.
- `k0` is the initial guess for the parameters to be fitted.
- `lb` and `ub` are the lower and upper bounds for the parameters, respectively.
- `popt` will contain the optimized parameters, and `pcov` will contain the covariance matrix, which can give you an idea of the uncertainties of the estimates.

# Example use of scipy.optimize.curve_fit

We have experimental data stored in a file, possibly in a .csv or .txt format, containing $T$ and $U$ data. We want to fit a model with coefficients $k_1$ and $k_2$ with the following structure:

$$\frac{du}{dt} = -k_1 u + k_2$$

- First, we define a function that describes our model:

```
import scipy as sp
import numpy as np

def simpleode(t, u, k1, k2):
    dudt = -k1*u + k2
    return dudt
```

Note that we supply the coefficients $k_1$ and $k_2$ as arguments to the function.

- We create a fit criterion function:

```
def fitcrit(t,k1, k2):
    u0 = [1.0]
    tspan = [0,max(t)]
    sol = sp.integrate.solve_ivp(simpleode, tspan, u0, args=(k1, k2), t_eval=t)
    return sol.y[0]
```

# Example use of scipy.optimize.curve_fit

Now let's make a script that uses `curve_fit` to yield k-values fitted to our dataset:

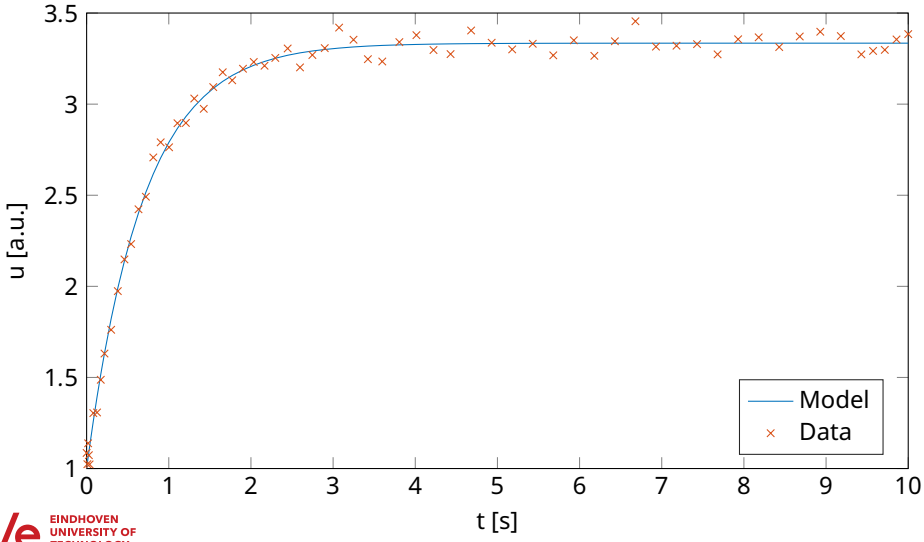
```
# Load your data here (adjust as necessary)
T, U = np.loadtxt('./scripts/optimization/tudataset1.txt',unpack=True, skiprows=1)

# Initial guesses for model parameters
k0 = [1.0, 1.0]

# Perform the curve fitting
params, params_covariance = sp.optimize.curve_fit(fitcrit, T, U, p0=k0)
print('Fitted coefficients:', params)
```

Our fitted coefficients are stored in `params`. The `params_covariance` gives an estimate of the covariance of the estimated parameters, offering an insight into the uncertainty of the fit.

# Example fitted ODE model



# Postprocessing of results

The data returned by `curve_fit` or `least_squares` can be used to obtain the 95% confidence intervals for the fitted parameters. Recall we stored/computed the parameters and covariance matrix:

```
params, params_covariance = curve_fit(fitcrit, T, U, p0=k0)
```

We can use the square root of the diagonal of the covariance matrix, multiplied by a factor from the t-distribution to get the confidence bounds:

```
from scipy import stats
import numpy as np

alpha = 0.05 # 95% confidence interval = 100*(1-alpha)
n = len(U) # number of data points
p = len(params) # number of parameters

dof = max(0, n - p) # number of degrees of freedom
tval = stats.t.ppf(1.0-alpha/2., dof) # t value for the dof and confidence level
sigma = np.sqrt(np.diag(params_covariance))
ci = sigma * tval

print('Confidence intervals:')
print('k1:', params[0] - ci[0], params[0] + ci[0])
print('k2:', params[1] - ci[1], params[1] + ci[1])
```

# Similar case, but using scipy.optimize.least_squares

- First, we define a function that describes our model:

```
def simpleode(t, u, k1, k2):
    dudt = -k1*u + k2
    return dudt
```

- Create a fit criterion function that computes the residuals between the model and the data:

```
def fitcrit(k,expdata):
    t = expdata[0]
    u = expdata[1]
    u0 = [u[0]]
    tspan = [0,max(t)]
    k1,k2 = k
    sol = sp.integrate.solve_ivp(simpleode, tspan, u0, args=(k1, k2), t_eval=t)
    return sol.y[0] - u
```

Note: the function `fitcrit` takes the parameters to fit as a single argument, so we need to unpack them inside the function.

# Similar case, but using scipy.optimize.least_squares

- Perform the fitting:

```
# Initial guesses for model parameters
k0 = [1.0, 1.0]

# Perform the curve fitting
expdata = [T,U]
fit = sp.optimize.least_squares(fitcrit, k0, args=(expdata,))
params = fit.x
params_covariance = fit.jac
print('Fitted coefficients:', params)
```

- Postprocessing of results is similar to the `curve_fit` case.

# Postprocessing of results

The data returned by `curve_fit` or `least_squares` can be used to obtain the 95% confidence intervals for the fitted parameters. Recall we stored/computed the parameters and covariance matrix:

```
params, params_covariance = curve_fit(fitcrit, T, U, p0=k0)
```

We can use the square root of the diagonal of the covariance matrix, multiplied by a factor from the t-distribution to get the confidence bounds:

```
from scipy import stats
import numpy as np

alpha = 0.05 # 95% confidence interval = 100*(1-alpha)
n = len(U) # number of data points
p = len(params) # number of parameters

dof = max(0, n - p) # number of degrees of freedom
tval = stats.t.ppf(1.0-alpha/2., dof) # t value for the dof and confidence level
sigma = np.sqrt(np.diag(params_covariance))
ci = sigma * tval

print('Confidence intervals:')
print('k1:', params[0] - ci[0], params[0] + ci[0])
print('k2:', params[1] - ci[1], params[1] + ci[1])
```

# Today's outline

- Introduction
- Curve fitting
- Regression
- Fitting numerical models
- Optimization
- Linear programming
- Summary

# What is optimization?

Optimization is minimization or maximization of an objective function (also called a performance index or goal function) that may be subject to certain constraints.

- $\min f(x)$: Goal function
- $g(x) = 0$: Equality constraints
- $h(x) \geq 0$: Inequality constraints

# Optimization Spectrum

| Problem | Method | Solvers |
|---|---|---|
| LP | Simplex method | Linprog |
| | Barrier methods | CPLEX (GAMS, AIMMS, AMPL, OPB) |
| NLP<br>QP | Lagrange multiplier method | Fminsearch/fmincon (Matlab) |
| | Successive linear programming | MINOS (GAMS, AMPL) |
| | Quadratic programming | CONOPT (GAMS) |
| MIP<br>MILP<br>MINLP<br>MIQP | Branch and bound | |
| | Dynamic programming | Bintprog (Matlab) |
| | Generalized Benders decomposition | DICOPT (GAMS) |
| | Outer approximation method | BARON (GAMS) |
| | Disjunctive programming | |

# Factors of concern

- Continuity of the functions
- Convexity of the functions
- Global versus local optima
- Constrained versus unconstrained optima

# Today's outline


- Introduction
- Curve fitting
- Regression
- Fitting numerical models
- Optimization
- Linear programming
- Summary

# Linear programming

In linear programming the objective function and the constraints are linear functions.

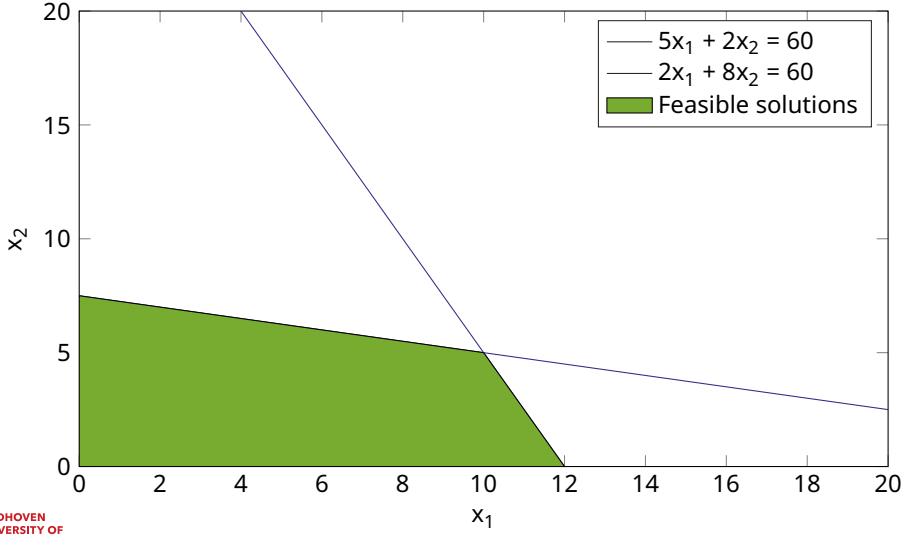
For example:

$$\max z = f(x_1, x_2) = 40x_1 + 88x_2$$
s.t. (subject to)
$$2x_1 + 8x_2 \leq 60$$
$$5x_1 + 2x_2 \leq 60$$
$$x_1 \geq 0$$
$$x_2 \geq 0$$

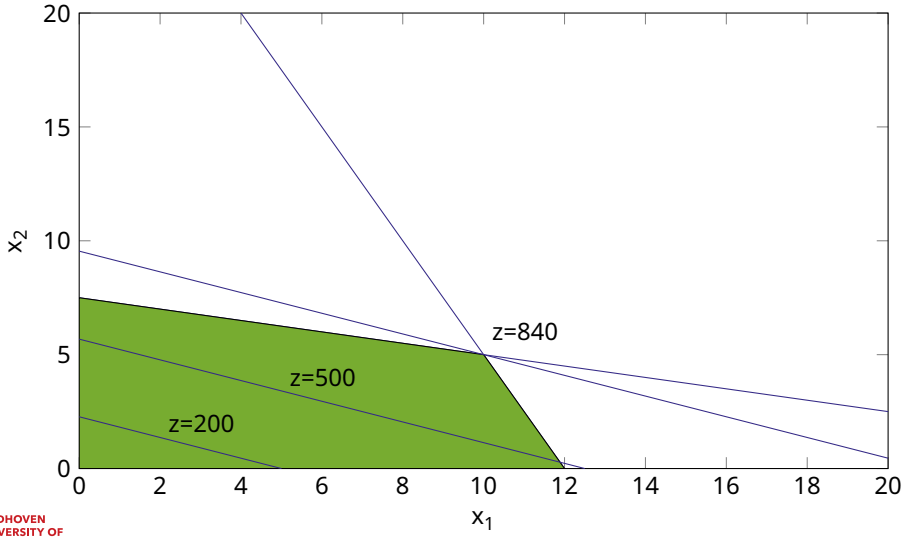
If the constraints are satisfied, but the objective function is not maximized/minimized we speak of a feasible solution.

If also the objective function is maximized/minimized, we speak of an optimal solution!

# Plotting the constraints



# Plotting the constraints



# Normal form of an LP problem

$$\max z = f(x_1, x_2) = 40x_1 + 88x_2$$
s.t.
$$2x_1 + 8x_2 \leq 60$$
$$5x_1 + 2x_2 \leq 60$$
$$x_1 \geq 0$$
$$x_2 \geq 0$$

$$\max z = f(x) = 40x_1 + 88x_2$$
s.t.
$$2x_1 + 8x_2 + x_3 = 60$$
$$5x_1 + 2x_2 + x_4 = 60$$
$$x_i \geq 0 \qquad i \in 1, 2, 3, 4$$

$x_3$ and $x_4$ are called slack variables, they are non auxiliary variables introduced for the purpose of converting inequalities in to equalities

# The simplex method

We can formulate our earlier example to the normal form and consider it as the following augmented matrix with $T_0 = [z \quad x_1 \quad x_2 \quad x_3 \quad x_4 \quad b]$:

$$T_0 = \begin{bmatrix} 1 & -40 & -88 & 0 & 0 & 0 \\ 0 & 2 & 8 & 1 & 0 & 60 \\ 0 & 5 & 2 & 0 & 1 & 60 \end{bmatrix}$$

This matrix is called the (initial) simplex table. Each simplex table has two kinds of variables, the basic variables (columns having only one nonzero entry) and the nonbasic variables

# The simplex method

$$T_0 = \begin{bmatrix} 1 & -40 & -88 & 0 & 0 & 0 \\ 0 & 2 & 8 & 1 & 0 & 60 \\ 0 & 5 & 2 & 0 & 1 & 60 \end{bmatrix}$$

Every simplex table has a feasible solution. It can be obtained by setting the nonbasic variables to zero: $x_1 = 0$, $x_2 = 0$, $x_3 = 60/1$, $x_4 = 60/1$, $z = 0$.

# The optimal solution?

- The optimal solution is now obtained stepwise by pivoting in such way that z reaches a maximum.
- The big question is, how to choose your pivot equation ...

# Step 1: Selection of the pivot column

Select as the column of the pivot, the first column with a negative entry in Row 1. In our example, that's column 2 (-40)

$$T_0 = \begin{bmatrix} 1 & -40 & -88 & 0 & 0 & 0 \\ 0 & 2 & 8 & 1 & 0 & 60 \\ 0 & 5 & 2 & 0 & 1 & 60 \end{bmatrix}$$

# Step 2: Selection of the pivot row

Divide the right sides by the corresponding column entries of the selected pivot column. In our example that is $60/2 = 30$ and $60/5 = 12$.

$$T_0 = \begin{bmatrix} 1 & -40 & -88 & 0 & 0 & 0 \\ 0 & 2 & 8 & 1 & 0 & 60 \\ 0 & 5 & 2 & 0 & 1 & 60 \end{bmatrix}$$

Take as the pivot equation the equation that gives the smallest quotient, so 60/5.

# Step 3: Elimination by row operations

- Row 1 = Row 1 + 8 * Row 3
- Row 2 = Row 2 - 0.4 * Row 3

$$T_1 = \begin{bmatrix} 1 & 0 & -72 & 0 & 8 & 480 \\ 0 & 0 & 7.2 & 1 & -0.4 & 36 \\ 0 & 5 & 2 & 0 & 1 & 60 \end{bmatrix}$$

The basic variables are now $x_1$, $x_3$ and the nonbasic variables are $x_2$, $x_4$. Setting the nonbasic variables to zero will give a new feasible solution: $x_1 = 60/5$, $x_2 = 0$, $x_3 = 36/1$, $x_4 = 0$, $z = 480$.

# The simplex method

- We moved from z = 0 to z = 480. The reason for the increase is because we eliminated a negative term from the eqation, so: elimination should only be applied to negative entries in Row 1, but no others.
- Although we found a feasible solution, we did not find the optimal solution yet (the entry of -72 in our simplex table) $\longrightarrow$ repeat step 1 to 3.

# The simplex method

Another iteration is required:

- Step 1: Select column 3
- Step 2: $36/7.2 = 5$ and $60/2 = 30 \longrightarrow$ select 7.2 as the pivot
- Elimination by row operations:
  - Row 1 = Row 1 + 10*Row 2
  - Row 3 = Row 3 - (2/7.2)*Row 2

$$T_2 = \begin{bmatrix} 1 & 0 & 0 & 10 & 4 & 840 \\ 0 & 0 & 7.2 & 1 & -0.4 & 36 \\ 0 & 5 & 0 & -1/36 & 1/0.9 & 50 \end{bmatrix}$$

- The basic feasible solution: $x_1 = 50/5$, $x_2 = 36/7.2$, $x_3 = 0$, $x_4 = 0$, $z = 840$ (no more negative entries: so this solution is also the optimal solution)

# Using Python for LP problems

We are going to solve the following LP problem:

$$\min f(x) = -5x_1 - 4x_2 - 6x_3$$

s.t.

$$x_1 - x_2 + x_3 \leq 20$$
$$3x_1 + 2x_2 + 4x_3 \leq 42$$
$$3x_1 + 2x_2 \leq 30$$

$$x_1 \geq 0$$
$$x_2 \geq 0$$
$$x_3 \geq 0$$

Using the function `linprog` from `scipy.optimize`:

```
from scipy.optimize import linprog

c = [-5, -4, -6]
A = [[1, -1, 1], [3, 2, 4], [3, 2, 0]]
b = [20, 42, 30]
bounds = [(0, None), (0, None), (0, None)]

res = linprog(c, A_ub=A, b_ub=b, bounds=bounds)
```

Gives (after accessing appropriate attributes of the result object):

```
x = res.x
fun = res.fun
slack = res.slack
success = res.success
```

# Summary

- Curve fitting: Manual procedures for polynomial fitting in Python
- Curve fitting: Python's SciPy library for curve fitting
- Curve fitting: Python's non-linear least-squares solver `least_squares`
- Curve fitting: Python's non-linear least-squares solver `curve_fit`
- Optimization: An introduction to the Simplex method in Python
- Optimization: Use of the `linprog` function in the SciPy library